AEGIS-SOC-GRC

An Integrated Security Operations and Governance, Risk & Compliance Platform

PROJECT DOCUMENTATION

A Comprehensive Technical Documentation of the Design, Development, Implementation and Testing of the AEGIS-SOC-GRC Platform

Prepared By

Pallabi Poria

Master of Computer Applications (MCA)

Area of Study: Cybersecurity

Project Focus: Security Operations & Governance, Risk and Compliance

TECHNOLOGY STACK

FastAPI • React • TypeScript • PostgreSQL • Docker

DEVELOPMENT YEAR

2026

2. PROJECT INFORMATION

2.1 Project Details

Information	Details
Project Name	AEGIS-SOC-GRC
Project Title	An Integrated Security Operations and Governance, Risk & Compliance Platform
Project Type	Practical Cybersecurity Project
Area of Study	Cybersecurity
Project Focus	Security Operations and Governance, Risk & Compliance
Author	Pallabi Poria
Program	Master of Computer Applications (MCA)
Development Year	2026
Document Version	1.0
Document Status	Final

2.2 Technology Stack

Component	Technology
Frontend	React + TypeScript
Backend	FastAPI
Database	PostgreSQL
Containerization	Docker
API Architecture	REST API
Security Simulation	AEGIS Scenario Engine
Evidence Integrity	SHA-256
Development Environment	Visual Studio Code / Windows
Database Environment	PostgreSQL running through Docker

2.3 Purpose of This Documentation

This document provides a detailed technical record of the design, development, implementation, testing, troubleshooting, and final validation of the AEGIS-SOC-GRC platform.

The purpose of the documentation is to explain not only what was built, but also why individual technologies, architectural approaches, workflows, component names, and implementation decisions were selected.

The document also records significant development challenges encountered during implementation, the investigation performed to identify their causes, the solutions applied, and the testing performed to confirm the final behaviour.

2.4 Professional Purpose of the Project

AEGIS-SOC-GRC was developed as a practical cybersecurity project to demonstrate the application of Security Operations Center (SOC), Governance, Risk and Compliance (GRC), security monitoring, detection engineering, incident management, risk management, evidence handling, compliance, and security auditing concepts in a working software platform.

The project is intended to demonstrate practical implementation skills rather than represent a production enterprise security platform. The system uses controlled and synthetic security activity for simulation and testing purposes.

2.5 Documentation Approach

A key objective of this documentation is to record the project as a **development journey**, rather than documenting only the final version of the application.

For each major feature, the documentation will explain:

- **What is it?**
- **Why was it needed?**
- **Why was this approach chosen?**
- **Why was it given this name?**
- **How was it designed?**
- **How was it implemented?**
- **Which files/components were involved?**
- **How does the data flow?**
- **What problems occurred?**
- **How was the problem investigated?**

- **What was the root cause?**
- **What was changed?**
- **How was it tested?**
- **What was the final result?**
- **What screenshot demonstrates it?**
- **What limitations remain?**
- **What could be improved in the future?**

3. PROJECT BACKGROUND

3.1 Project Details

Cybersecurity is not limited to identifying malicious activity. A complete security process involves monitoring activity, detecting suspicious behaviour, investigating security events, managing incidents, assessing associated risks, implementing controls, performing remediation, collecting evidence, verifying the effectiveness of controls, and maintaining an auditable record of important activities.

While learning cybersecurity concepts, I wanted to build a project that could demonstrate this broader lifecycle in a practical and connected manner rather than implementing only an individual security tool or isolated feature.

The initial idea was therefore to develop a security-focused application that could simulate security activity, detect suspicious behaviour, generate alerts, and provide a structured workflow for investigating and managing the resulting security information.

As the project evolved, the scope was expanded beyond traditional SOC functionality to include risk management, controls, remediation, evidence, compliance, and auditing. This resulted in the development of **AEGIS-SOC-GRC**, an integrated platform connecting security operations with governance, risk, and compliance workflows.

3.2 Why I Decided to Build This Project

The project was developed to gain practical experience in connecting different areas of cybersecurity within one working application.

Instead of creating separate demonstrations for topics such as:

- Security monitoring
- Detection engineering
- Incident management
- Risk management
- Security controls
- Evidence management
- Compliance

- Auditing

the objective was to understand how these activities can relate to one another in a realistic security workflow.

The project therefore became an opportunity to move from learning individual cybersecurity concepts to implementing a connected system around them.

3.3 The Initial Idea

The initial concept focused primarily on the **SOC side** of cybersecurity.

The basic idea was:

Security Activity
↓
Security event
↓
Detection
↓
Alert
↓
Investigation

This would provide a practical demonstration of how suspicious activity could be generated and detected.

However, this workflow ended at investigation and did not answer an important question:

> **What happens after a security issue has been identified?**

A real security process may require the organization to understand the associated risk, determine which control addresses it, perform remediation, collect supporting evidence, verify the remediation, and maintain records for governance and auditing.

This observation became an important turning point in the project.

3.4 Expansion from SOC to SOC + GRC

The project was therefore expanded from a primarily SOC-oriented application into an integrated **SOC + GRC platform**.

The resulting workflow became:

Scenario
↓
Security Telemetry
↓
Detection Rule
↓
Alert
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
Evidence Integrity
↓
Compliance
↓
Audit Trail

This expansion changed the project from a simple security monitoring demonstration into a broader security lifecycle platform**.**

3.5 Project Development Philosophy

The project followed a **build-and-validate** approach.

Rather than attempting to implement every feature at once, the platform was developed incrementally.

A simplified representation of the development approach is:

Idea
↓
Architecture
↓
Environment Setup

↓
Database
↓
Backend APIs
↓
Security Telemetry
↓
Detection
↓
Scenario Engine
↓
Alerts
↓
Incidents
↓
Risk
↓
Controls
↓
Remediation
↓
Evidence
↓
Integrity Verification
↓
Compliance
↓
Audit Trail
↓
Reports & UI Refinement
↓
Testing

Each stage introduced new functionality while also creating dependencies for later stages.

For example, the Evidence Integrity functionality was not treated as an isolated feature. It became part of the evidence workflow and subsequently contributed to the verification and audit process.

3.6 Core Objective

The central objective of the project became:

> **To build a practical cybersecurity platform that demonstrates how security operations can be connected with risk management, remediation, evidence, compliance, and auditing through a traceable workflow.**

This objective influenced the architecture, database relationships, backend APIs, frontend pages, simulation engine, evidence workflow, and audit system developed throughout the project.

3.7 What the Project Is Intended to Demonstrate

AEGIS-SOC-GRC is intended to demonstrate practical understanding of:

Security Operations

- Security event generation
- Security telemetry
- Detection rules
- Alert management
- Incident management
- Security investigation workflows

Risk Management

- Risk identification
- Risk scoring
- Risk status
- Relationship between incidents and risks

Security Controls

- Control identification
- Control assignment
- Control testing
- Verification

Remediation

- Remediation tracking
- Remediation status
- Verification of remediation effectiveness

Evidence

- Evidence collection

- Artifact management
- Evidence metadata
- Evidence review
- Evidence integrity

Security Integrity

- SHA-256 hashing
- Artifact verification
- Integrity status tracking

Governance and Compliance

- Compliance records
- Control relationships
- Evidence supporting controls
- Verification activities

Auditing

- Platform-wide audit events
- State changes
- Traceability of important security and GRC activities

3.8 Project Scope and Practical Limitation

AEGIS-SOC-GRC is a **self-developed practical cybersecurity project** intended to demonstrate concepts and implementation skills.

It is not intended to replace a production enterprise SIEM, SOAR, GRC platform, or enterprise security operations environment.

The security activity used by the Scenario Engine is controlled and synthetic. The simulation layer generates intentional telemetry for testing detection and workflow behaviour rather than performing real attacks.

This distinction is important because it allows the project to demonstrate security detection and response workflows in a controlled environment without requiring offensive activity against real systems.

3.9 Result of the Initial Concept

The project ultimately evolved from a basic idea of:

> **“Build a security monitoring application.”**

into:

> **“Build a connected security lifecycle platform where simulated security activity can be traced from detection through incident management, risk treatment, remediation, evidence, compliance, and auditing.”**

This evolution became the foundation for the final AEGIS-SOC-GRC architecture.

4. PROJECT NAME AND NAMING RATIONALE

4.1 Why the Project Was Named AEGIS-SOC-GRC

The name **AEGIS-SOC-GRC** was intentionally constructed from three concepts that represent the overall purpose of the platform:

> **AEGIS + SOC + GRC**

Each part has a specific meaning within the project.

4.2 Why "AEGIS" Was Chosen

The word **AEGIS** represents protection, defense, and safeguarding.

The project was designed around the idea of creating a protective security lifecycle in which security activity can be monitored, detected, investigated, managed, remediated, verified, and audited.

Therefore, **AEGIS** was selected as the primary project identity because it represents the overall protective purpose of the platform.

The name also reflects the fact that the platform is not limited to a single cybersecurity function.

It connects multiple stages:

Detection → Investigation → Risk Treatment → Remediation → Evidence → Compliance → Audit

Key idea

> **AEGIS represents the protective layer connecting the technical and governance sides of the security lifecycle.**

4.3 Why "SOC" Was Included

SOC stands for **Security Operations Center**.

The SOC portion of the project represents the operational security side of AEGIS-SOC-GRC.

It includes activities such as:

- Security telemetry
- Security event generation
- Detection rules
- Alert creation
- Alert investigation
- Incident management
- Security scenario simulation

The term **SOC** was therefore included because the project contains a complete simulated security-operations workflow rather than functioning only as a generic cybersecurity application.

The SOC workflow begins with security activity and continues through detection and investigation.

Security Scenario → Security Telemetry → Detection Rule → Alert → Incident

4.4 Why "GRC" Was Included

GRC represents:

- Governance
- Risk
- Compliance

The project was expanded beyond SOC functionality because detecting a security problem is only one part of the security lifecycle.

A security issue may need to be:

- Assessed as a risk
- Mapped to a control
- Addressed through remediation
- Supported by evidence
- Tested
- Verified
- Associated with compliance requirements
- Recorded for auditing

Therefore, GRC became an important part of the project architecture.

The GRC workflow can be represented as:

Risk → Control → Remediation → Evidence → Verification → Compliance → Audit

4.5 Why the Complete Name Is AEGIS-SOC-GRC

The three terms together describe the overall purpose:

Name	Meaning	Role in Project
AEGIS	Protection / defense	Overall project identity
SOC	Security Operations Center	Detection and operational security
GRC	Governance, Risk & Compliance	Risk, controls, remediation, evidence and compliance

Therefore:

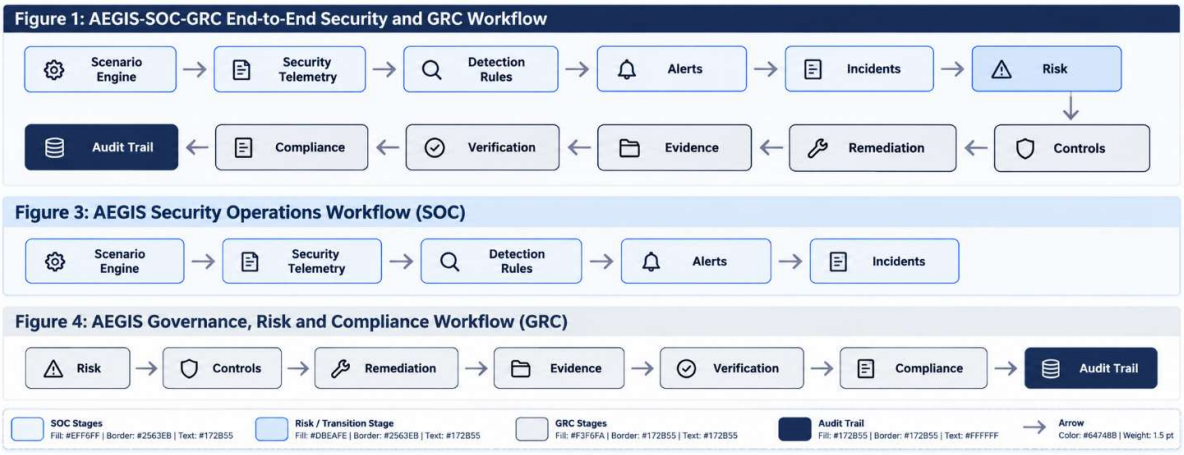
> **AEGIS-SOC-GRC represents a security platform that combines operational security monitoring and detection with governance, risk, compliance, remediation, evidence, and auditing workflows.**

This naming structure also reflects the project's central architectural concept: **connecting SOC activity with GRC outcomes**.

4.6 Naming of the Major Components

The component names were selected to represent their actual responsibility within the security lifecycle.

The naming sequence follows the movement of security information through the platform.



Each name was selected based on the role performed by that component.

4.7 Scenario Engine

Why this name?

The term **Scenario Engine** was selected because this component does more than generate individual security events.

It provides reusable security scenarios that represent different types of security activity and can generate a sequence of synthetic events.

The Scenario Engine acts as the controlled starting point of the AEGIS security workflow.

Its purpose is to create intentional synthetic telemetry that can subsequently be processed by the detection workflow.

The implementation was specifically designed as a **synthetic telemetry layer** rather than an attack-execution framework. It does not execute real attacks, malware, network scans, phishing activity, or similar offensive operations.

Name rationale

> **Scenario** = a predefined security situation
>
> **Engine** = the component responsible for executing the scenario logic

Therefore, **Scenario Engine** accurately describes its role.

4.8 Security Telemetry

Why this name?

The term **Security Telemetry** was chosen for the information generated by security activity.

Telemetry represents the observable information produced by an event or activity.

Examples include:

- Failed authentication attempts
- Successful authentication
- Suspicious process activity
- Network activity
- Web application activity
- Other simulated security events

The term was preferred over simply calling the data "logs" because the platform uses the generated information as security signals for subsequent detection and analysis.

4.9 Detection Rules

Why this name?

The component is called **Detection Rules** because it contains the logical conditions used to identify potentially suspicious activity within security telemetry.

A detection rule evaluates incoming security information and determines whether a particular security condition has occurred.

For example:

> Multiple failed login attempts → potential brute-force activity.

The term **Detection Rules** therefore directly describes the function performed by this component.

4.10 Alerts

Why this name?

An **Alert** represents a security condition that has been identified by a detection rule and requires attention.

The term follows common SOC terminology.

The distinction is important:

Security Event → Detection Rule → Alert

The event is the observed activity.

The detection rule evaluates that activity.

The alert represents the resulting security finding.

4.11 Incidents

Why this name?

The term **Incident** represents a security situation that requires investigation and management beyond an individual detection.

An alert can provide an initial indication of suspicious activity, while an incident provides a broader operational context for handling the security issue.

Therefore, the workflow distinguishes between:

Alert → Incident

rather than treating every alert as an incident.

4.12 Risk

Why this name?

The term **Risk** represents the potential security impact associated with a security issue.

The project uses risk management to connect operational security findings with the potential consequences they represent.

A risk can therefore provide the bridge between:

> **What happened technically**

and

> **What needs to be addressed from a security and governance perspective.**

4.13 Controls

Why this name?

Controls represent security measures intended to reduce or manage identified risks.

Once a risk is identified, an appropriate control can be associated with it.

This creates a relationship:

Risk → Control

The term **Controls** was chosen because this is standard GRC terminology and accurately represents the security measures used to address risk.

4.14 Remediation

Why this name?

Remediation represents the actions taken to address a security weakness, risk, incident-related requirement, or control deficiency.

The term was selected because remediation describes the process of correcting or improving the identified security condition.

The workflow therefore becomes:

This creates a relationship:

Risk → Control → Remediation

This allows the platform to represent not only what security problem exists, but also what action is being taken to address it.

4.15 Evidence

Why this name?

Evidence represents the information or artifacts collected to demonstrate that a control, remediation activity, or security requirement has been addressed.

The Evidence component provides a connection between an implemented action and its verification.

For example:

Remediation → Evidence → Control Testing → Verification

The term **Evidence** was chosen because it directly represents the supporting material required to validate a security or GRC activity.

4.16 Evidence Integrity

Why this name?

Evidence integrity was given its own concept because simply storing an evidence artifact does not establish that the artifact has remained unchanged.

AEGIS therefore uses **SHA-256 hashing** to verify the integrity of collected evidence artifacts.

The workflow is:

Evidence Artifact → SHA-256 Hash → Stored Hash → Recalculate Hash
↓
Verified / Failed ← Compare

This allows the platform to determine whether the current artifact matches its recorded cryptographic hash.

The term Evidence Integrity was therefore chosen to describe the verification of whether collected evidence remains consistent with its recorded state.

4.17 Compliance

Why this name?

The **Compliance** component represents the governance side of security requirements.

It connects controls and supporting evidence with compliance-related requirements.

The purpose is not simply to display a compliance status, but to demonstrate how technical security activities can support governance requirements.

4.18 Audit Trail

Why this name?

The term **Audit Trail** was selected because the component provides a historical record of important activities performed within the platform.

Examples include:

- Alert creation
- Alert status changes
- Evidence integrity verification
- Other security and GRC state changes

The Audit Trail provides a historical sequence that helps answer:

> **What happened, to which entity, when, and what changed?**

This makes the Audit Trail the final traceability layer of the AEGIS-SOC-GRC workflow.

4.19 Naming Philosophy

The names of the components were intentionally kept understandable and aligned with commonly used cybersecurity and GRC terminology.

The naming philosophy can be summarized as:

> **Use familiar security terminology for operational concepts, while giving the overall platform a distinct identity through the AEGIS name.**

This approach makes the application easier to understand while also giving the project a recognizable identity.

5. PROJECT OBJECTIVES AND SCOPE

5.1 Project Objectives

The primary objective of AEGIS-SOC-GRC is to develop a practical cybersecurity platform that demonstrates how security operations and Governance, Risk and Compliance activities can be connected within a single workflow.

The project was designed with the following objectives:

- To simulate realistic cybersecurity events using controlled and synthetic security scenarios.
- To generate security telemetry that can be consumed by detection logic.
- To implement detection rules for identifying suspicious or security-relevant activity.
- To convert detected activity into actionable alerts.
- To support the investigation and management of security incidents.
- To connect security incidents with identified organizational risks.
- To associate risks with appropriate security controls.
- To support remediation activities for identified risks and security issues.
- To maintain evidence associated with controls and remediation activities.
- To verify evidence integrity using SHA-256 hashing.
- To support compliance-related tracking through controls, evidence, and verification.
- To maintain an audit trail of important security and governance activities.
- To provide reports and a centralized interface for viewing security and GRC information.

> **Key Objective:**
> **“The objective of AEGIS-SOC-GRC is not only to detect security events, but to demonstrate how a detected security issue can progress through investigation, risk management, remediation, evidence verification, compliance and auditing.”**

5.2 Functional Scope

The functional scope of the project covers the complete security and GRC lifecycle implemented within the platform.

Area	Scope
Security Simulation	Controls synthetic security scenarios
Security Telemetry	Generation and recording of security events
Detection	Rule-based identification of suspicious activity
Alerts	Creation and management of detected security conditions
Incidents	Investigation and tracking of security incidents
Risk Management	Identification and assessment of security risks
Controls	Association and management of security controls
Remediation	Tracking corrective security actions
Evidence	Collection and management of supporting artifacts
Integrity	SHA-256 based evidence integrity verification
Compliance	Control, evidence and verification tracking
Audit Trail	Historical recording of important platform activities
Reports	Presentation of security and GRC information
Frontend	Centralized React-based management interface
Backend	REST APIs implemented using FastAPI
Database	Persistent PostgreSQL data storage

5.3 Technical Scope

The technical scope includes the development and integration of the major software components required to operate the platform.

The **frontend** provides the user interface for viewing and managing security and GRC information, while the **FastAPI backend** provides REST APIs for communication between the interface and database.

PostgreSQL is used for persistent storage of security events, alerts, incidents, risks, controls, evidence, audit records and related information. **Docker** is used to provide a consistent database environment during development.

The **AEGIS Scenario Engine** provides controlled security simulations that generate synthetic telemetry without executing real attacks, malware, network scans or other harmful activity.

5.4 Security Scope

The security scope focuses on demonstrating the lifecycle of security monitoring and response rather than implementing a production-grade SOC.

The project demonstrates:

Scenario → Telemetry → Detection → Alert → Incident → Risk → Control → Remediation → Evidence → Verification → Compliance → Audit Trail

This allows the project to demonstrate how operational security activity can be connected to governance and risk management processes.

5.5 Project Boundaries

AEGIS-SOC-GRC was intentionally developed as a controlled cybersecurity project.

It does **not** attempt to replace enterprise SIEM, SOAR, GRC or EDR platforms. The simulation layer uses synthetic security activity so that detection and GRC workflows can be tested safely in a development environment.

The project therefore focuses on **demonstrating architecture, integration, workflow implementation and practical cybersecurity concepts**, rather than claiming production-level scalability or enterprise deployment readiness

5.6 Expected Outcome

The expected outcome was a functional platform in which security activity could be followed from its initial simulation through detection, investigation, risk management, remediation, evidence handling, verification, compliance and auditing.

The completed implementation provides this connected workflow and demonstrates how SOC and GRC processes can be integrated rather than treated as completely separate activities.

6. REQUIREMENTS ANALYSIS

6.1 Purpose of Requirements Analysis

Before implementation, the project requirements were identified based on the intended SOC and GRC workflow. The requirements were divided into **functional requirements**, which define what the platform should do, and **non-functional requirements**, which define how the platform should behave.

6.2 Functional Requirements

The major functional requirements identified for AEGIS-SOC-GRC were:

6.2.1 Security Scenario Simulation

The platform should provide controlled scenarios capable of generating synthetic security activity.

The scenarios should represent different categories of security events, such as authentication activity, network activity, endpoint activity, phishing-related activity and web application activity.

6.2.2 Security Telemetry

The system should record simulated security events in a structured format so that they can be processed by the detection layer.

Each event should contain relevant information required for analysis and investigation.

6.2.3 Detection Rules

The platform should evaluate security telemetry against predefined detection rules.

A detection rule should identify a specific suspicious condition and produce a meaningful security result when its conditions are satisfied.

6.2.4 Alert Management

When a detection condition is identified, the platform should create an alert.

Alerts should contain information such as severity, status, detection rule, description and associated security information.

The system should also support updating alert status and recording important status changes.

6.2.5 Incident Management

Security alerts should be capable of progressing into incidents for investigation and tracking.

The incident workflow should provide a structured way to manage security situations identified by the SOC layer.

6.2.6 Risk Management

The platform should allow security issues to be represented as risks.

Risk records should support information required for assessing and managing the potential impact of a security issue.

6.2.7 Control Management

Risks should be associated with appropriate security controls.

Controls should provide a governance mechanism for addressing identified risks.

6.2.8 Remediation Management

The platform should support corrective actions associated with identified security risks or incidents.

Remediation should allow the project to demonstrate the transition from identifying a problem to addressing it.

6.2.9 Evidence Management

The system should allow supporting evidence to be associated with controls and remediation activities.

Evidence should contain sufficient metadata to identify its purpose, source, type and relationship to the relevant security or GRC record.

6.2.10 Evidence Integrity Verification

The platform should provide a mechanism to verify whether an evidence artifact remains consistent with its recorded integrity value.

SHA-256 hashing is used for this purpose.

6.2.11 Compliance Tracking

The platform should connect controls and evidence with compliance-related information.

This allows the project to demonstrate how technical security activities can provide supporting information for governance and compliance processes.

6.2.12 Audit Trail

Important platform activities should be recorded in an audit trail.

The audit records should capture information such as:

- Timestamp
- Actor
- Action
- Entity type
- Entity ID
- Description
- Previous value
- New value
- Source

This provides traceability for important changes and verification activities.

6.3 Non-Functional Requirements

6.3.1 Usability

The interface should provide a clear and consistent way to navigate between SOC and GRC modules.

6.3.2 Maintainability

The backend, frontend and database components should be separated into logical modules so that individual features can be developed and modified independently.

6.3.3 Reliability

The platform should provide appropriate error handling and should maintain consistent relationships between related security and GRC records.

6.3.4 Data Persistence

Important security and governance information should be stored persistently in PostgreSQL rather than being maintained only in frontend state.

6.3.5 Traceability

The system should maintain relationships between security activity and subsequent GRC actions wherever applicable.

This allows an analyst to follow the progression from a security event toward remediation, evidence and verification.

6.3.6 Security and Integrity

The platform should use controlled synthetic activity for simulation and provide integrity verification for collected evidence.

6.4 Development Environment Requirements

The development environment required the following major components:

Requirement	Technology / Component
Operating environment	Windows
Code editor	Visual Studio Code
Backend	Python + FastAPI
Frontend	React + TypeScript
Database	PostgreSQL
Database environment	Docker
API Communication	REST
Security Simulation	AEGIS Scenario Engine
Evidence Integrity	SHA-256
Browser	Modern Web Browser

6.5 Requirement-to-Implementation Mapping

The requirements were subsequently translated into individual project modules.

Requirement	Implemented Component
Security Simulation	Scenario Engine
Security Event Generation	Security Telemetry
Threat Identification	Detection Rules
Security Notification	Alerts
Investigation	Incidents
Security Assessment	Risk Management
Risk Treatment	Controls
Corrective Action	Remediation
Supporting Documentation	Evidence
Artifact Verification	SHA-256 Integrity
Governance Tracking	Compliance
Activity Traceability	Audit Trail
Information Presentation	Reports and Dashboard

> **Requirement Design Principle:**
> Each major requirement was mapped to a specific functional component so that the final platform could be developed incrementally while maintaining a clear connection between project objectives and implementation.

7. TECHNOLOGY SELECTION AND JUSTIFICATION

7.1 Overview

The technology stack for AEGIS-SOC-GRC was selected based on the requirements of the project, the need for a clear separation between frontend and backend components, database persistence, API-based communication, controlled security simulation, and maintainable development.

The selected technologies were not chosen only because they are commonly used; each technology was selected for a specific role within the overall architecture.

7.2 Technology Stack

Layer	Technology	Purpose
Frontend	React	User interface and component-based application structure
Frontend Language	typescript	Type-safe frontend development
Backend	FastAPI	REST API development and backend business logic
Backend Language	Python	Backend implementation and security simulation
Database	PostgreSQL	Persistent storage of project data
Containerization	Docker	Consistent database development environment
API Architecture	REST API	Communication between frontend and backend
Security Simulation	AEGIS Scenario Engine	Controlled synthetic security activity
Integrity Verification	SHA-256	Evidence artifact integrity verification
Development Environment	Visual Studio Code	Project development and source-code management

7.3 React

React was selected for the frontend because the platform contains multiple independent modules such as Dashboard, Alerts, Incidents, Risks, Controls, Remediation, Evidence, Compliance, Reports and Audit Trail.

A component-based frontend makes it possible to organize these modules separately while maintaining a consistent interface.

React also supports dynamic rendering, which is useful for displaying changing security records, filtering data, expanding details and interacting with backend APIs.

7.4 TypeScript

TypeScript was used together with React to provide stronger type checking during frontend development.

The application contains structured objects such as alerts, incidents, risks, evidence records and audit logs. Defining interfaces for these objects helps maintain consistency between the frontend and the backend API responses.

This was particularly useful as the project expanded and additional modules were introduced.

7.5 FastAPI

FastAPI was selected as the backend framework for AEGIS-SOC-GRC.

The backend is responsible for:

- Exposing REST API endpoints.
- Processing frontend requests.
- Performing database operations.
- Running detection-related logic.
- Managing security and GRC records.
- Handling evidence-related operations.
- Recording audit activities.

FastAPI also provides automatic API documentation, which was useful during development and testing of individual endpoints.

7.6 Python

Python was selected as the primary backend programming language.

Python was suitable for the project because it supports both web application development and cybersecurity-oriented development.

The same language could therefore be used for backend services, security-event processing, detection logic and the AEGIS Scenario Engine.

7.7 PostgreSQL

PostgreSQL was selected as the project's relational database.

The platform contains multiple related entities, including:

- Security events
- Alerts
- Incidents
- Assets
- Risks
- Controls
- Remediation records
- Evidence
- Compliance information
- Audit logs

A relational database was appropriate because these entities have relationships that need to remain consistent and traceable.

7.8 Docker

Docker was used to provide the PostgreSQL database in a controlled containerized environment.

Instead of installing and managing the database environment entirely on the host system, the project used a PostgreSQL Docker container.

This simplified database setup and provided a more consistent development environment.

7.9 REST API

A **REST-based API architecture** was used to connect the React frontend with the FastAPI backend.

The general communication pattern is:

React Frontend → REST API → FastAPI Backend → PostgreSQL

For example, when the Alerts page requires alert information, the frontend sends an API request to the backend. The backend retrieves the required records from PostgreSQL and returns structured data to the frontend.

7.10 AEGIS Scenario Engine

The **AEGIS Scenario Engine** was developed specifically for this project.

Its purpose is to generate controlled, synthetic security activity that can be processed by the platform's detection workflow.

The engine does not execute real attacks. Instead, it uses predefined scenarios to generate telemetry representing security situations such as authentication failures, network reconnaissance, endpoint activity, phishing activity and web application attacks.

This provides a safe way to demonstrate detection and investigation workflows.

7.11 SHA-256

SHA-256 was selected for evidence integrity verification.

When an evidence artifact is collected, its SHA-256 hash can be stored with the evidence record.

During verification, the artifact can be hashed again and the newly calculated value can be compared with the stored value.

The basic process is:

Evidence Artifact → SHA-256 Hash → Stored Hash → Recalculate → Compare → Integrity Result

This allows the project to demonstrate whether the collected artifact remains consistent with its recorded integrity value.

7.12 Visual Studio Code

Visual Studio Code was used as the primary development environment.

It provided a single environment for working with:

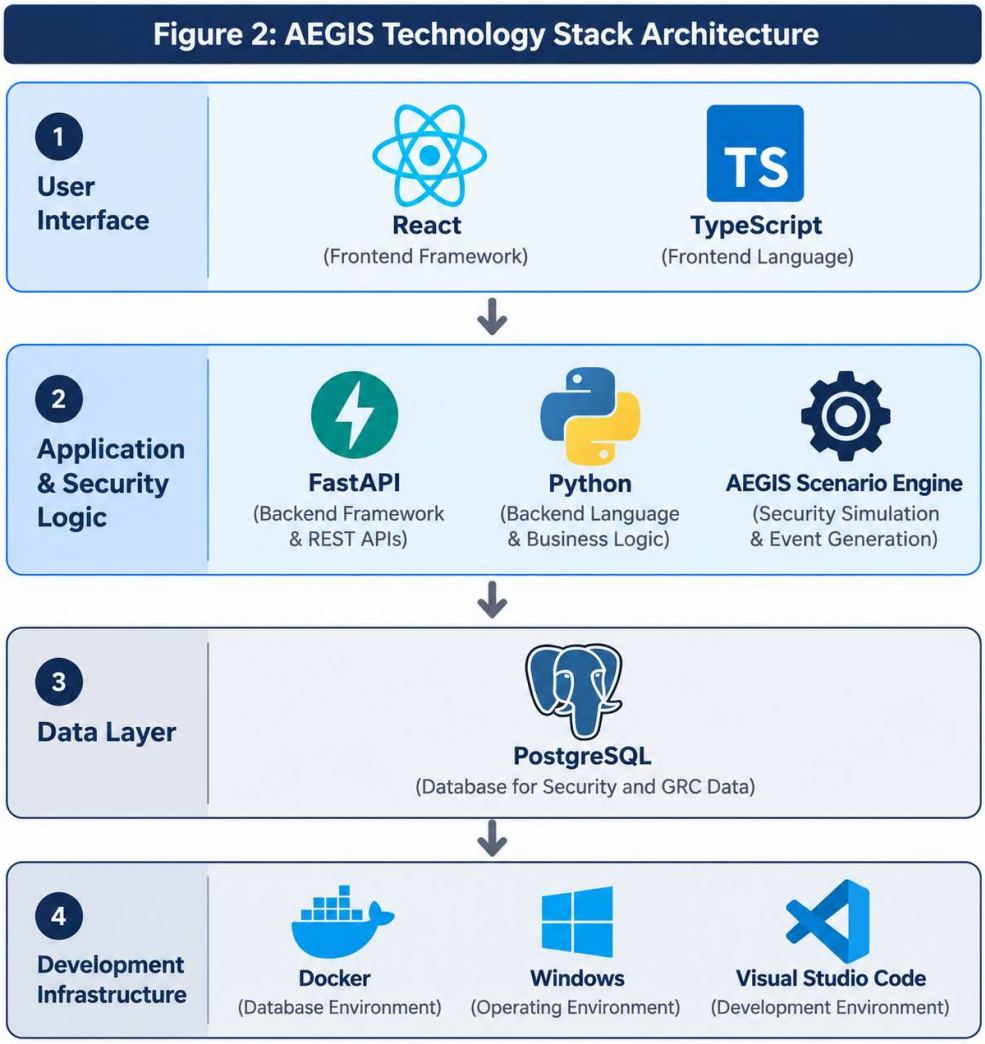
- Python backend files
- React and TypeScript files
- Configuration files
- Database-related scripts
- Simulation scripts
- CSS
- Project documentation and supporting files

7.13 Why This Technology Combination

The selected technologies form a practical full-stack architecture:

> **React + TypeScript** provides the interactive security dashboard, **FastAPI + Python** provides the backend and security logic, **PostgreSQL** provides persistent storage, **Docker** provides the database environment, and the **AEGIS Scenario Engine** connects controlled security simulation with the detection workflow.

This combination allowed the project to remain modular while supporting the complete SOC-to-GRC workflow.


Figure 2: AEGIS Technology Stack Architecture
1
User Interface
React
(Frontend Framework)
TS
TypeScript
(Frontend Language)
2
Application & Security Logic
FastAPI
(Backend Framework & REST APIs)
Python
(Backend Language & Business Logic)
AEGIS Scenario Engine
(Security Simulation & Event Generation)
3
Data Layer
PostgreSQL
(Database for Security and GRC Data)
4
Development Infrastructure
Docker
(Database Environment)
Windows
(Operating Environment)
Visual Studio Code
(Development Environment)

8. SYSTEM ARCHITECTURE

8.1 Architecture Overview

AEGIS-SOC-GRC follows a **layered full-stack architecture** in which the frontend, backend, security processing, database and simulation components are separated into logical layers.

The architecture was designed to keep the system modular while allowing security operations and GRC activities to exchange information through the backend APIs.

The overall architecture can be represented as:

User Interface → API Layer → Application & Security Logic → Data Layer

The AEGIS Scenario Engine operates within the application and security layer and generates controlled security telemetry that enters the SOC workflow.

8.2 Major Architectural Layers

The system consists of the following major layers:

8.2.1 Presentation Layer

The presentation layer is implemented using **React and TypeScript**.

It provides the user interface through which security and GRC information can be viewed and managed.

Major interface areas include:

- Dashboard
- Alerts
- Incidents
- Detection Rules
- Scenarios
- Assets
- Risks

- Controls
- Remediation
- Evidence
- Compliance
- Reports
- Audit Trail

8.2.2 API Layer

The API layer is implemented using **FastAPI**.

It acts as the communication boundary between the frontend and backend services.

The frontend sends HTTP requests to the appropriate API endpoints, while the backend processes the requests and returns structured responses.

This separation prevents the frontend from directly accessing the database.

8.2.3 Application and Security Logic Layer

This layer contains the core processing logic of the platform.

It includes:

- Security event processing
- Detection logic
- Alert creation
- Incident processing
- Risk management
- Control management
- Remediation workflows
- Evidence processing
- Integrity verification

- Compliance-related processing
- Audit event recording

The **AEGIS Scenario Engine** is also part of this layer and provides controlled synthetic security scenarios for testing the SOC workflow.

8.2.4 Data Layer

PostgreSQL provides persistent storage for the platform.

The database stores the information required by both the SOC and GRC portions of the system.

Examples include:

- Security events
- Detection-related information
- Alerts
- Incidents
- Assets
- Risks
- Controls
- Remediation records
- Evidence
- Compliance information
- Audit logs

8.2.5 Development Infrastructure Layer

The development infrastructure provides the environment required to run and develop the platform.

The project uses:

- **Windows** as the operating environment

- **Docker** for the PostgreSQL database environment
- **Visual Studio Code** as the primary development environment

8.3 High-Level Data Flow

The high-level interaction between the major components can be represented as:

User → React + TypeScript Frontend → FastAPI REST API → Application & Security Logic → PostgreSQL Database

The security simulation workflow follows a separate but connected path:

AEGIS Scenario Engine → Synthetic Security Telemetry → Detection Rules → Alerts → Incidents → Risk / GRC Workflow

8.4 SOC-to-GRC Integration

One of the main architectural characteristics of AEGIS-SOC-GRC is that the SOC and GRC functions are connected.

The SOC side identifies and investigates security activity, while the GRC side provides a structured mechanism for managing the resulting security risks.

The relationship can be represented as:

Security Activity → Detection → Alert → Incident → Risk → Control → Remediation → Evidence → Verification → Compliance → Audit Trail

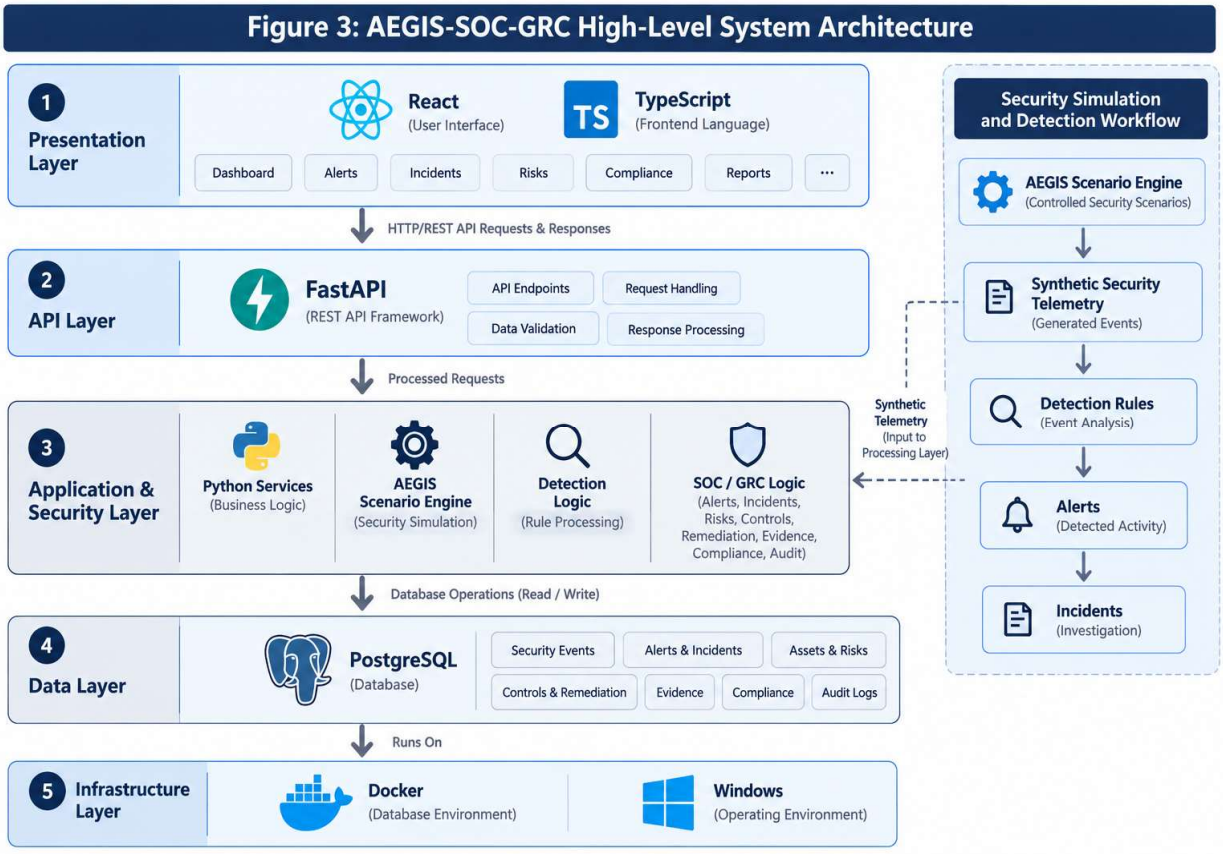
This architecture allows the project to demonstrate a continuous flow from technical security monitoring to governance and accountability.

8.6 Architectural Design Principle

The architecture was designed around a simple principle:

> **Security activity should be processed through clearly separated layers while maintaining traceability across the complete SOC and GRC lifecycle.**

This approach allowed the project to grow incrementally, with individual modules being added without requiring the entire application architecture to be redesigned.



9. DATABASE ARCHITECTURE

9.1 Database Overview

PostgreSQL was selected as the primary database for AEGIS-SOC-GRC because the platform contains multiple structured entities that need to be stored persistently and related to one another.

The database acts as the central storage layer for both the **Security Operations Center (SOC)** and **Governance, Risk and Compliance (GRC)** components of the platform.

The database was designed around the project's end-to-end security lifecycle rather than treating each module as an isolated feature.

9.2 Purpose of the Database

The database provides persistent storage for information generated throughout the platform.

Its main purposes are:

- Store security telemetry and security events.
- Store detection-related information.
- Maintain alerts generated by detection rules.
- Maintain incidents created for investigation.
- Store assets involved in security activity.
- Record identified security risks.
- Maintain security controls.
- Track remediation activities.
- Store collected evidence and related metadata.
- Maintain compliance information.
- Record evidence integrity information.
- Maintain the audit trail of important system activities.

9.3 Major Database Entities

The major entities implemented within the project include:

Entity	Purpose
Security Events	Stores generated security telemetry
Detection Rules	Defines conditions used for security detection
Alerts	Stores detected security conditions
Incidents	Represents security situations requiring investigation
Assets	Stores information about monitored assets
Risks	Records identified security risks
Controls	Represents measures used to address risks
Remediation	Tracks corrective actions
Evidence	Stores supporting security and GRC artifacts
Compliance	Stores compliance-related information
Audit Logs	Records important platform activities

9.4 SOC Database Flow

The SOC portion of the database follows the movement of security information from telemetry toward investigation.

Security Events → Detection Rules → Alerts → Incidents → Assets

Security events provide the underlying activity, while detection rules evaluate that activity.

When a rule identifies a relevant condition, an alert can be created. Alerts can subsequently be associated with incidents for investigation.

Assets provide information about the systems or resources involved in the security activity.

9.5 GRC Database Flow

The GRC portion extends the workflow beyond incident investigation.

Incident → Risk → Control → Remediation → Evidence → Verification → Compliance → Audit Log

This structure allows technical security activity to be connected with governance and risk-management activities.

9.6 Evidence and Integrity Data

Evidence records contain information required to identify and manage collected artifacts.

The evidence workflow also includes integrity information associated with the artifact.

The integrity process follows:

Evidence Artifact → SHA-256 Hash → Stored Integrity Value → Verification → Verified / Failed

The resulting verification activity can also generate an audit record, providing traceability for the integrity-check operation.

9.7 Audit Log Data

The `audit_logs` entity provides a historical record of important activities.

An audit record can contain:

- Unique audit record ID
- Timestamp
- Actor
- Action
- Entity type
- Entity ID
- Description
- Previous value
- New value
- Source

For example, when an alert changes from one status to another, the previous and new values can be recorded in the audit trail.

This provides an additional layer of traceability beyond the current state stored in the primary entity.

9.8 Database Relationships

The database relationships were designed to support traceability between related SOC and GRC records.

A simplified relationship can be represented as:

Security Event
↓
Detection Rule
↓
Alert → Asset
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
Compliance

Audit logs operate as a historical traceability layer for important activities occurring across these entities.

9.9 Database Environment

PostgreSQL is executed through a **Docker container** during development.

The architecture therefore separates the application environment from the database environment:

AEGIS Application
|
↓ **Database connection**

PostgreSQL
↓
Docker Container

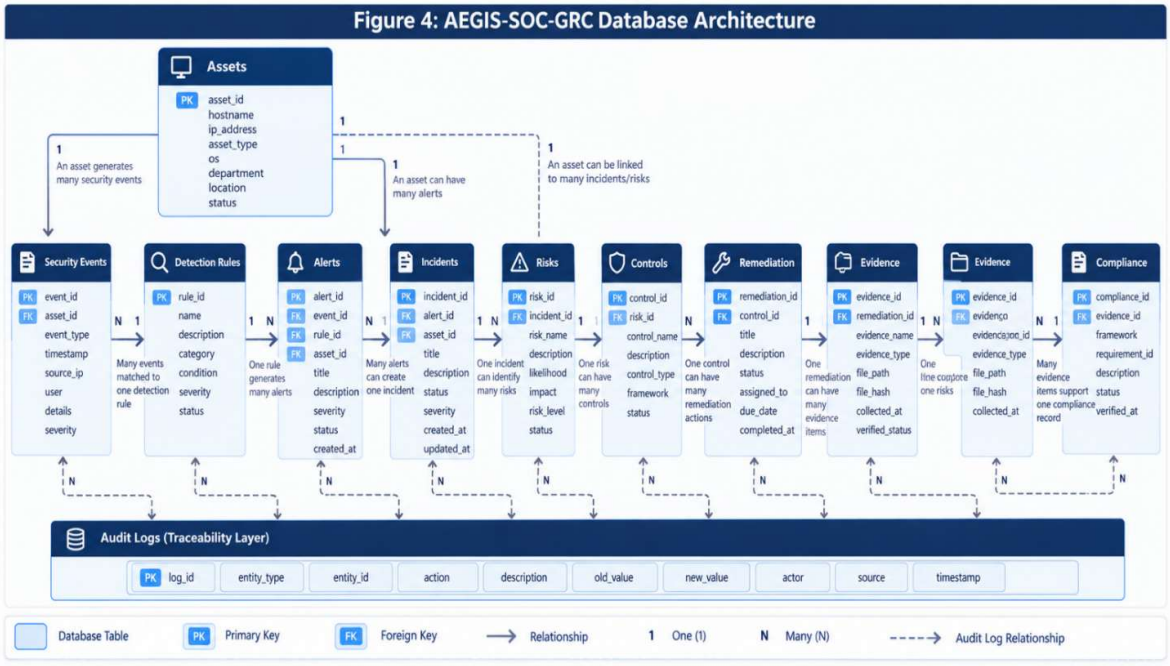
This approach allowed the database environment to be managed independently from the application code.

9.10 Database Design Principle

The database was designed around the following principle:

> **Each major security or GRC concept should have a structured representation that can be connected to related records and traced through the overall security lifecycle.**

This design supports the central objective of AEGIS-SOC-GRC: connecting security operations with risk management, remediation, evidence, compliance and auditing.



10. SECURITY OPERATIONS CENTER (SOC) WORKFLOW

10.1 Overview

The SOC component represents the operational security side of AEGIS-SOC-GRC. Its purpose is to demonstrate how security activity can be generated, observed, analyzed, detected and progressed into an incident for investigation.

The SOC workflow is centered around five major stages:

Scenario Engine → Security Telemetry → Detection Rules → Alerts → Incidents

This workflow provides the foundation for the later GRC process, where identified security issues can be assessed as risks and connected with controls and remediation activities.

10.2 Scenario Generation

The SOC workflow begins with the **AEGIS Scenario Engine**.

The Scenario Engine provides predefined security scenarios that represent different types of security activity. These scenarios are intentionally controlled and synthetic so that the detection workflow can be tested without performing real attacks.

Examples of scenario categories include:

- Authentication activity
- Network reconnaissance
- Endpoint activity
- Phishing activity
- Web application activity

Each scenario contains information such as its scenario ID, category, severity, description, detection rules and expected outcome.

10.3 Security Telemetry

When a scenario is executed, it generates synthetic security events.

These events represent observable activity that a SOC platform could receive from monitored systems.

Telemetry can contain information such as:

- Event type
- Timestamp
- User
- Source IP
- Destination information
- Asset information
- Event-specific details

The generated telemetry becomes the input for the detection layer.

10.4 Detection Rules

The detection layer evaluates security telemetry against predefined detection rules.

A detection rule represents a logical condition that identifies a particular security pattern.

For example, repeated failed authentication attempts can be evaluated to identify potential password-guessing activity.

The detection process therefore follows:

Telemetry → Detection Rule → Detection Result

The purpose of this layer is to convert raw security activity into meaningful security findings.

10.5 Alert Creation

When a detection rule identifies a relevant condition, the platform can create an **alert**.

An alert provides a structured representation of the detected security condition.

Important alert information can include:

- Alert ID
- Detection rule
- Title
- Severity
- Status
- Description
- Username
- Source information
- Related security activity

Alerts provide the SOC analyst with a starting point for investigation.

10.6 Incident Creation

Some detected security conditions can progress from alerts into **incidents**.

An incident represents a broader security situation that requires investigation and management.

The incident layer provides additional context around the detected activity and allows the security issue to be tracked as part of the response process.

10.7 Example SOC Detection Flow

A simplified example of the implemented concept is:

Brute-Force Scenario
↓
Multiple Failed Login Events
↓
Authentication Detection Rule
↓
AUTH-001 / AUTH-002
↓

High-Severity Alert
↓
Security Incident

This demonstrates how a simulated security scenario can travel through multiple layers instead of remaining as an isolated event.

10.8 SOC-to-GRC Transition

The SOC workflow does not represent the end of the security lifecycle.

Once a security issue has been investigated, the identified condition can become relevant to the GRC workflow.

The transition is:

Incident → Risk → Control → Remediation

This connection is one of the central design principles of AEGIS-SOC-GRC.

> **SOC identifies and investigates the security problem; GRC provides the structure for managing its risk and remediation.**

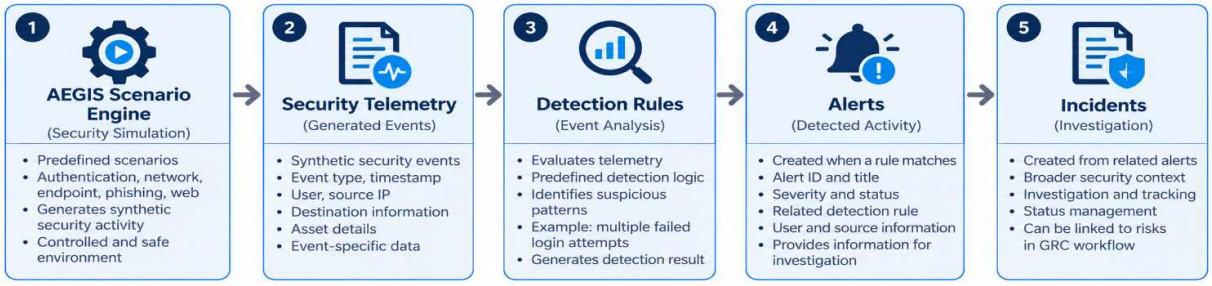
10.9 SOC Workflow Design Principle

The SOC workflow was designed to maintain a clear distinction between:

- **Events** — observable security activity.
- **Detection Rules** — logic used to identify meaningful patterns.
- **Alerts** — detected security conditions.
- **Incidents** — security situations requiring investigation.

Keeping these concepts separate makes the system easier to understand and allows each stage to have a specific responsibility.

Figure 5: AEGIS Security Operations Workflow



11. DETECTION RULES

11.1 Overview

Detection rules form the analytical layer of the AEGIS SOC workflow. Their purpose is to examine security telemetry and identify patterns that may represent suspicious or security-relevant activity.

The detection layer sits between **Security Telemetry** and **Alerts**:

Security Telemetry → Detection Rules → Alerts

This separation allows raw security events to remain distinct from the logic used to interpret those events.

11.2 Purpose of Detection Rules

The detection rules were implemented to:

- Analyze generated security events.
- Identify predefined suspicious patterns.
- Apply consistent detection logic.
- Associate detected activity with a specific rule.
- Generate actionable detection results.
- Provide the basis for alert creation.
- Support investigation through meaningful security context.

11.3 Detection Rule Structure

Each detection rule represents a specific security condition.

A rule can contain information such as:

Attribute	Purpose
Rule ID	Unique identification of the rule
Rule Name	Human-readable rule name
Description	Explanation of the detected condition
Severity	Importance of the detected activity
Detection Logic	Condition used for identification
Category	Security activity category

MITRE Technique	Associated ATT&CK technique where applicable
Expected Outcome	Expected result when the condition is detected

11.4 Example: Authentication Detection

One implemented example is the detection of multiple failed authentication attempts.

A simplified representation is:

Failed Login
↓
Failed Login
↓
Failed Login
↓
Repeated Attempts Detected
↓
AUTH-001
↓
Alert

The detection rule evaluates the generated authentication telemetry and identifies repeated failed login activity.

This provides a practical example of how multiple low-level events can be interpreted as a meaningful security condition.

11.5 Detection Rules and MITRE ATT&CK

Where applicable, detection scenarios are associated with **MITRE ATT&CK techniques**.

For example, the authentication brute-force scenario is associated with:

T1110.001 — Password Guessing

The use of ATT&CK references provides additional context about the type of behaviour represented by the scenario and detection rule.

11.6 Detection Result

When a detection condition is satisfied, the detection process produces a result that can be used by the alert-management layer.

The general flow is:

Security Event
↓
Detection Logic
↓
Condition Satisfied
↓
Detection Result
↓
Alert Creation

This ensures that an alert is based on an identifiable detection condition rather than being created independently of the security telemetry.

11.7 Detection Rule Categories

The project includes detection concepts covering multiple security areas.

These include:

- Authentication
- Network activity
- Endpoint activity
- Phishing activity
- Web application activity

This allows the Scenario Engine and detection layer to demonstrate different types of security monitoring rather than relying on a single event category.

11.8 Relationship with the Scenario Engine

The Scenario Engine and detection rules were designed as separate components.

The **Scenario Engine generates the security activity**, while the **detection layer determines whether that activity matches a known security condition**.

This separation is important because the same detection logic can potentially be tested against different telemetry without changing the detection concept itself.

The relationship is:

Scenario → Synthetic Events → Detection Rule → Detection Result

11.9 Relationship with Alerts

Detection rules provide the foundation for alert creation.

When a rule identifies suspicious activity, the resulting alert can contain the corresponding detection rule information.

This creates traceability between:

Security Event → Detection Rule → Alert

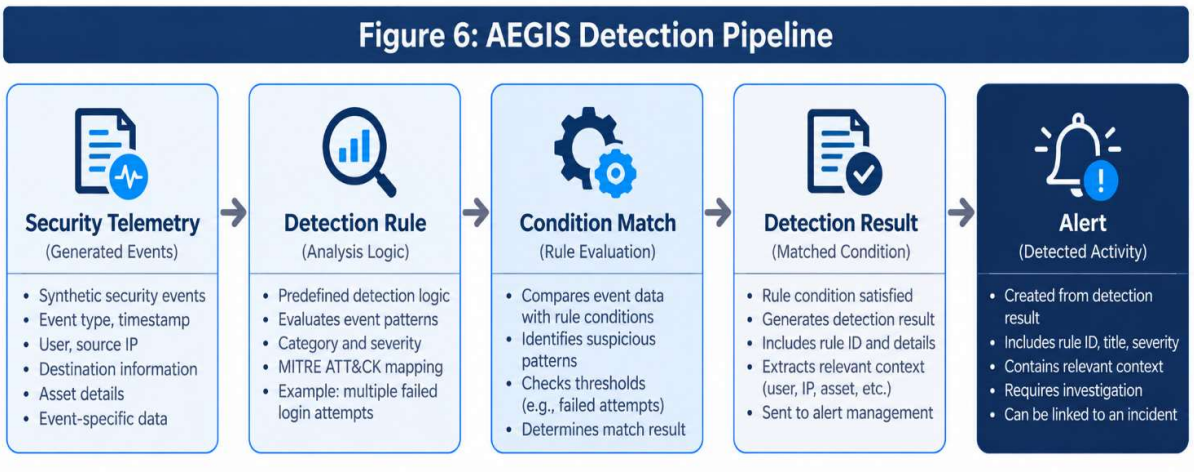
An analyst can therefore understand not only that an alert exists, but also which detection logic identified the activity.

11.10 Detection Design Principle

The detection layer follows the principle:

> **Raw telemetry should be transformed into meaningful security findings through explicit and traceable detection logic.**

This makes the SOC workflow easier to investigate, test and extend with additional detection scenarios.



12. ALERT MANAGEMENT

12.1 Overview

Alerts represent the point where the AEGIS SOC workflow converts a detected security condition into an actionable security record.

The alert workflow follows:

Detection Result → Alert → Investigation → Incident

An alert provides the SOC analyst with structured information about the activity identified by a detection rule.

12.2 Purpose of Alerts

The alert-management component was developed to:

- Record detected security conditions.
- Preserve information about the detection rule that generated the alert.
- Assign an appropriate severity.
- Provide context for investigation.
- Track the current alert status.
- Support progression from alert to incident.
- Maintain traceability of important alert changes through the audit trail.

12.3 Alert Information

An alert can contain several pieces of information that help an analyst understand the detected activity.

Information	Purpose
Alert ID	Uniquely identifies the alert
Title	Provides a short description of the detection
Detection Rule	Identifies the rule responsible for the alert
Severity	Indicates the importance of the alert

Status	Shows the current state of the alert
Description	Provides additional security context
User	Identifies the relevant user where applicable
Source Information	Provides information about the originating activity
Asset	Identifies the affected or associated asset
Created At	Records when the alert was created
Updated At	Records the latest update

12.4 Alert Severity

Severity provides a way to prioritize detected security activity.

The project uses severity levels to distinguish the relative importance of alerts.

For example:

- **Low** — Activity requiring limited attention.
- **Medium** — Activity requiring review.
- **High** — Significant suspicious activity requiring investigation.
- **Critical** — Highly significant activity requiring immediate attention.

Severity is stored as part of the alert rather than being represented only visually in the frontend.

12.5 Alert Status

Alert status represents the current state of an alert throughout its lifecycle.

The status can change as the security condition is reviewed and handled.

A simplified lifecycle is:

Open

↓

Under Investigation

↓

Resolved

The actual status management is handled through the backend, while the frontend provides the interface for viewing and updating the status.

12.6 Alert Creation

Alerts are created as a result of detection processing.

The overall process is:

Security Telemetry

↓

Detection Rule

↓

Condition Match

↓

Detection Result

↓

Alert Created

This ensures that alerts originate from identifiable detection activity.

For example, a brute-force simulation can generate repeated failed-login events. When the relevant authentication detection rule identifies the pattern, an alert such as **Multiple Failed Login Attempts** can be created.

12.7 Alert Investigation

The Alerts interface provides analysts with information required to understand the detected condition.

The interface supports viewing details such as:

- Alert title
- Severity
- Status
- Detection rule

- Description
- User
- Source information
- Associated asset
- Related security context

The design separates the summary information from expanded details so that the analyst can quickly scan alerts while still being able to inspect individual records.

12.8 Alert Status Audit

Important alert changes are also connected to the Audit Trail.

For example, when an alert status changes, the system records:

Old Status → New Status

along with information about the action, entity, timestamp and source.

This creates a historical record of important alert-state changes rather than storing only the latest state.

12.9 Relationship with Incidents

An alert represents a detected security condition, while an incident represents a broader security situation requiring investigation and management.

The relationship can therefore be represented as:

Detection → Alert → Incident

Multiple related alerts may contribute to the understanding of an incident.

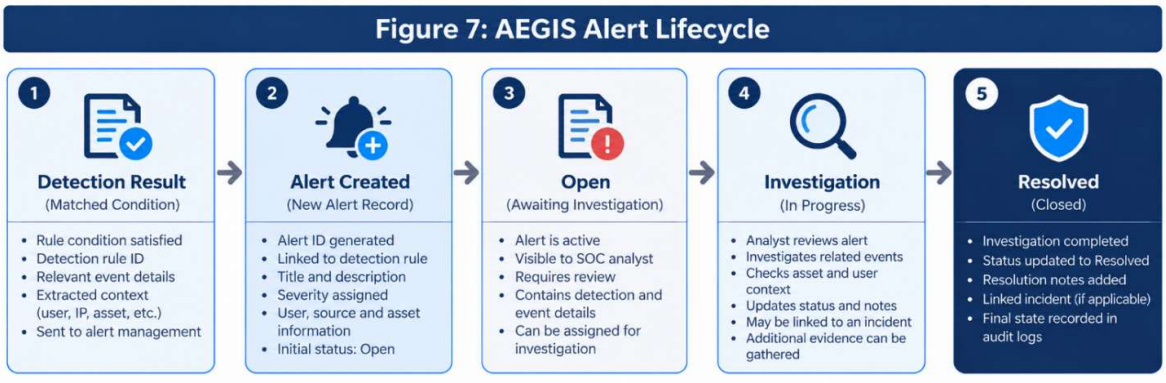
This distinction helps prevent the system from treating every individual detection as an independent incident.

12.10 Alert Design Principle

The alert-management component follows the principle:

> **A security detection should produce an actionable and traceable record that provides sufficient context for investigation and can progress into the incident-management workflow.**

This makes alerts the operational bridge between the detection layer and incident response.



13. INCIDENT MANAGEMENT

13.1 Overview

Incident management represents the next stage after alert detection in the AEGIS-SOC-GRC security workflow.

While an **alert** represents a detected security condition, an **incident** provides a broader context for investigating and managing a security situation.

The relationship is:

Alert → Incident → Investigation → Risk

13.2 Purpose of Incident Management

The incident-management component was designed to:

- Track security situations requiring investigation.
- Provide broader context around related alerts.
- Maintain information about the affected security activity.
- Support investigation and status tracking.
- Connect operational security activity with the GRC workflow.
- Provide a transition point from SOC investigation to risk management.

13.3 Alert-to-Incident Relationship

An alert identifies a specific security condition detected by a rule.

An incident represents the larger security situation that may contain or be associated with one or more alerts.

Detection Rule → Alert → Related Security Activity → Incident → Investigation

This distinction prevents the system from treating every individual detection as an independent security incident.

13.4 Incident Information

An incident can contain structured information used for investigation and tracking.

Typical information includes:

Information	Purpose
Incident ID	Unique incident identification
Title	Short description of the incident
Description	Detailed security context
Severity	Indicates incident importance
Status	Represents current incident state
Related Alert	Connects incident with detected activity
Asset	Identifies affected or associated asset
Timeline	Provides investigation context
Created At	Records incident creation
Updated At	Records latest modification

13.5 Incident Investigation

Once an incident is created, the analyst can investigate the associated security activity.

The investigation may involve reviewing:

- Related alerts.
- Security events.
- Detection rules.
- Affected assets.
- Users involved in the activity.
- Source information.
- Supporting evidence.
- Risk implications.

This provides a broader view than examining the original alert alone.

13.6 Incident Status

Incident status provides a way to track the progress of an investigation.

A simplified lifecycle can be represented as:

Open

↓

Under Investigation

↓

Resolved / Closed

The status allows the current state of an incident to be distinguished from its historical activity.

13.7 Incident and Risk Relationship

An important purpose of incident management in AEGIS-SOC-GRC is to provide a transition into risk management.

After an incident is investigated, its underlying security issue may indicate a broader organizational risk.

The workflow therefore becomes:

Incident → Risk

For example, repeated authentication failures may not only represent a single security event but may also highlight weaknesses in authentication monitoring or account protection.

These weaknesses can then be represented as risks and addressed through appropriate controls and remediation.

13.8 SOC-to-GRC Transition

The incident layer acts as the transition point between the operational SOC workflow and the governance workflow.

SOC
Scenario
↓
Telemetry
↓
Detection
↓
Alert
↓
Incident
|
▼
GRC
Risk
↓
Control
↓
Remediation

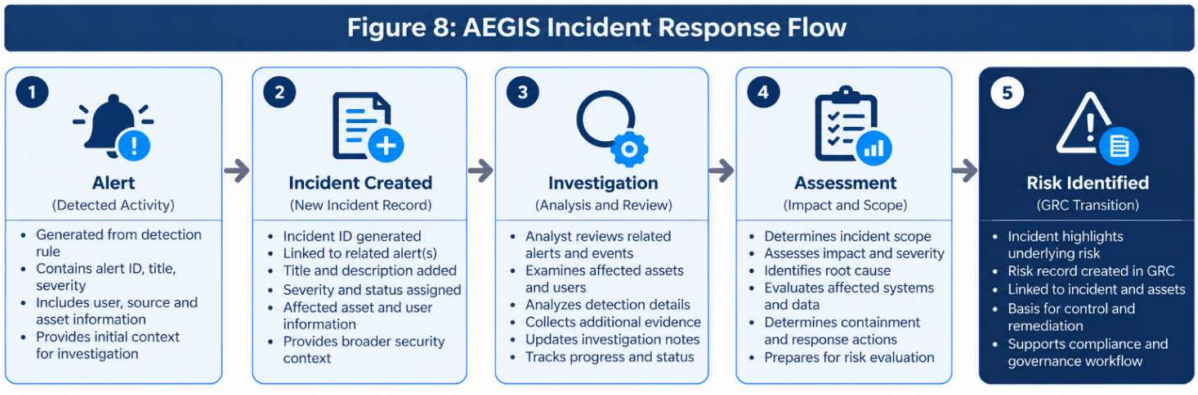
This connection is central to the design of AEGIS-SOC-GRC.

13.9 Incident Management Design Principle

The incident-management component follows the principle:

> **A detected security condition should be capable of being investigated as a broader security situation before its implications are evaluated from a risk and governance perspective.**

This maintains a clear distinction between detection, investigation and risk management.



14. RISK MANAGEMENT

14.1 Overview

Risk management represents the point where the AEGIS-SOC-GRC workflow moves from **security investigation into governance and risk management**.

After an incident has been investigated and its impact has been assessed, the underlying security weakness or exposure can be represented as a risk.

The workflow is:

Incident → Risk → Control → Remediation

14.2 Purpose of Risk Management

The risk-management component was developed to:

- Record security risks identified during investigation.
- Connect risks with their originating incidents.
- Associate risks with affected assets where applicable.
- Describe the underlying security concern.
- Assess likelihood and potential impact.
- Assign an overall risk level.
- Track the current risk status.
- Provide the basis for selecting appropriate controls.
- Connect technical security findings with GRC activities.

14.3 Risk Identification

A risk is identified when an investigated security issue indicates a condition that could negatively affect the organization's systems, information or operations.

The project therefore distinguishes between the immediate security event and its broader implication.

For example:

Repeated Authentication Failures

↓

Security Alert

↓

Incident Investigation

↓

Weak Authentication Protection Identified

↓

Security Risk

This allows the project to demonstrate that risk management is based on an identified security condition rather than being an isolated GRC activity.

14.4 Risk Information

Risk records contain structured information used to describe and assess the identified risk.

Information	Purpose
Risk ID	Unique risk identification
Risk Name	Short description of the risk
Description	Detailed explanation
Incident	Links the risk to its security origin
Asset	Identifies the affected asset where applicable
Likelihood	Estimates probability of occurrence
Impact	Represents potential consequence
Risk Score	Provides an overall risk measurement
Risk Level	Categorizes the severity
Status	Tracks the current risk state

14.5 Risk Assessment

Risk assessment considers two important dimensions:

Likelihood — how likely the risk is to occur or continue.

Impact — how significant the consequences could be if the risk materializes.

These factors contribute to the overall risk assessment.

A simplified representation is:

Likelihood + Impact

↓

Risk Assessment

↓

Risk Score

↓

Risk Level

The resulting risk level helps prioritize the issue for treatment.

14.6 Risk Levels

The platform uses risk levels to distinguish the relative significance of identified risks.

A typical interpretation is:

- **Low** — Limited security concern requiring routine management.
- **Medium** — Requires planned treatment and monitoring.
- **High** — Significant security concern requiring prioritized remediation.
- **Critical** — Severe security exposure requiring urgent treatment.

The risk level provides a concise representation of the assessed security concern.

14.7 Risk Status

Risk status represents the current stage of risk treatment.

A risk can progress through stages such as:

Identified

↓

Under Treatment

↓

Mitigated

↓

Closed

The exact status depends on the progress of the associated control and remediation activities.

14.8 Risk-to-Control Relationship

Once a risk has been identified and assessed, an appropriate security control can be associated with it.

The relationship is:

Risk → Control

The control represents the security measure intended to reduce the likelihood, impact or exposure associated with the risk.

For example:

Risk:

Weak Authentication Protection

↓

Control:

Authentication Monitoring & Account Lockout

This creates a direct connection between the identified risk and the security measure intended to address it.

14.9 Risk-to-Remediation Relationship

Controls provide the governance mechanism for addressing risks, while remediation represents the corrective action performed to implement or improve the control.

The resulting chain is:

Risk → Control → Remediation → Evidence → Verification

This creates a traceable path from identifying a security weakness to demonstrating that corrective action was performed and verified.

14.10 Risk Detail and Traceability

The AEGIS interface provides a detailed view of individual risks.

The risk detail view can present:

- Risk level
- Risk score
- Risk status
- Description
- Related incident
- Related asset
- Associated controls
- Remediation information
- Supporting evidence
- Verification information

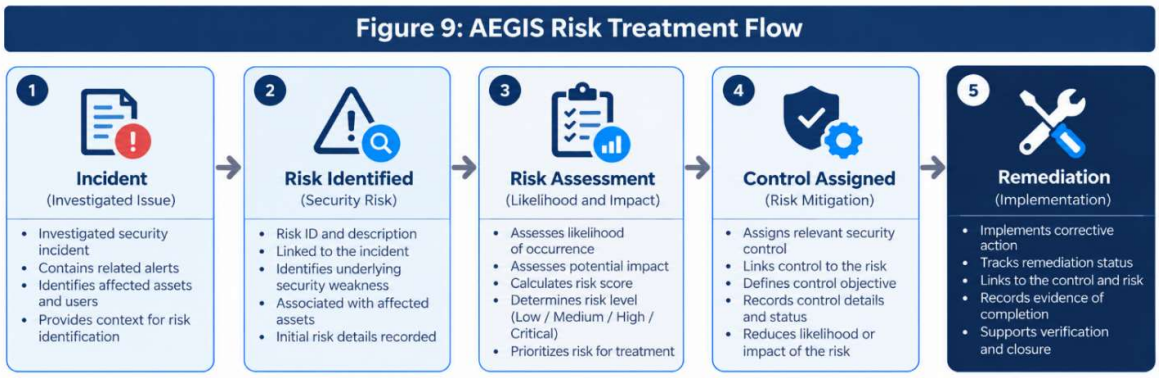
This allows the analyst to understand the risk without losing its connection to the rest of the security lifecycle.

14.11 Risk Management Design Principle

The risk-management component follows the principle:

> **Security incidents should be translated into measurable and manageable risks that can be addressed through defined controls and remediation activities.**

This makes risk management the central transition between the operational SOC workflow and the GRC treatment lifecycle.



15. SECURITY CONTROLS

15.1 Overview

Security controls represent the measures used to address risks identified during the GRC workflow.

After a risk has been assessed, an appropriate control can be associated with it to define how the risk should be managed or reduced.

The workflow is:

Risk → Control → Remediation → Evidence → Verification

15.2 Purpose of Security Controls

The Controls component was developed to:

- Associate security measures with identified risks.
- Define the objective of a control.
- Describe how the control addresses a security concern.
- Track the implementation status of controls.
- Connect controls with remediation activities.
- Provide a basis for collecting supporting evidence.
- Support compliance and audit activities.

15.3 Control Information

A control record can contain structured information such as:

Information	Purpose
Control ID	Unique control identification
Control Name	Identifies the security control
Description	Explains the purpose of the control

Control type	Categorizes the control
Framework	Identifies the associated framework
Risk	Connects the control to an identified risk
Status	Tracks implementation state
Objective	Defines what the control is intended to achieve

15.4 Risk-to-Control Mapping

The most important relationship is the association between a risk and its corresponding control.

Identified Risk

↓

Control Requirement

↓

Security Control

↓

Control Implementation

For example, an authentication-related risk may be addressed through a control requiring authentication monitoring and account lockout mechanisms.

This creates a direct relationship between the identified weakness and the security measure intended to address it.

15.5 Control Types

Controls can be categorized according to their purpose or implementation characteristics.

Examples include:

- Preventive controls
- Detective controls
- Corrective controls
- Technical controls
- Administrative controls

Within AEGIS-SOC-GRC, the control structure is primarily used to demonstrate the relationship between identified risks, security measures and subsequent remediation activities.

15.6 Control Status

Control status provides an indication of the current implementation state.

A simplified control lifecycle is:

Planned

↓

In Progress

↓

Implemented

↓

Verified

The status can be used alongside evidence and verification information to determine whether the control has been adequately implemented.

15.7 Control and Remediation

A control defines **what security measure is required**, while remediation represents the corrective work performed to implement or improve that measure.

Therefore:

Control → Remediation

For example:

Control:

Authentication Monitoring & Account Lockout

↓

Remediation:

Implement / configure the required monitoring

and account lockout mechanism

This separation allows the project to distinguish the governance requirement from the actual corrective action.

15.8 Control and Evidence

Once remediation has been performed, evidence can be collected to demonstrate that the control has been implemented.

The resulting relationship is:

Control → Remediation → Evidence → Control Test → Verification

This provides a structured path from defining a security requirement to demonstrating its implementation.

15.9 Control and Compliance

Controls also provide the foundation for compliance tracking.

A compliance requirement can be supported by one or more controls and their associated evidence.

This creates the relationship:

Compliance Requirement → Control → Evidence → Verification

The approach demonstrates how technical security activities can produce supporting information for governance and compliance purposes.

15.10 Example Control Workflow

A simplified example from the project can be represented as:

Authentication Risk
↓
CTRL-AUTH-001
↓
Authentication Monitoring and Account Lockout
↓
Remediation

↓

Configuration Evidence

↓

Control Test

↓

Verification

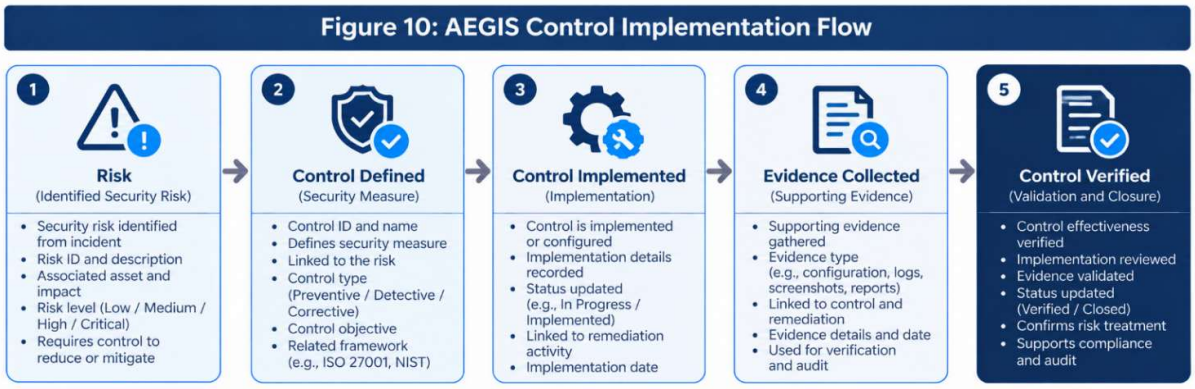
This example demonstrates the complete transition from a security weakness to a documented and verified control.

15.11 Control Design Principle

The Controls component follows the principle:

> **Every significant security risk should be capable of being associated with a defined security measure whose implementation can be demonstrated and verified.**

This makes controls the central bridge between risk identification and measurable remediation.



16. REMEDIATION MANAGEMENT

16.1 Overview

Remediation represents the corrective-action stage of the AEGIS-SOC-GRC workflow.

After a security risk has been identified and an appropriate control has been assigned, remediation records the actions required to address the identified security weakness.

The workflow is:

Risk → Control → Remediation → Evidence → Verification

16.2 Purpose of Remediation

The Remediation component was developed to:

- Record corrective actions associated with security risks.
- Connect corrective actions with their relevant controls.
- Track remediation progress.
- Record implementation and completion information.
- Provide a basis for collecting supporting evidence.
- Support subsequent control testing and verification.
- Contribute to risk closure and compliance activities.

16.3 Remediation Information

A remediation record can contain information such as:

Information	Purpose
Remediation ID	Unique identification
Control	Identifies the control being addressed
Title	Short description of the corrective action
Description	Explains the remediation activity

Status	Tracks remediation progress
Assigned To	Identifies the responsible party
Due Date	Defines the expected completion date
Completed At	Records completion
Evidence	Provides supporting implementation evidence

16.4 Remediation Lifecycle

A simplified remediation lifecycle is:

Identified
↓
Planned
↓
In Progress
↓
Completed
↓
Verified

The lifecycle provides visibility into whether a corrective action has merely been identified, is currently being implemented, or has been completed and subsequently verified.

16.5 Relationship with Controls

A control defines the security requirement, while remediation defines the action taken to satisfy or improve that requirement.

For example:

Control

Authentication Monitoring

↓

Remediation

Configure authentication monitoring

and account lockout mechanisms

This distinction allows governance requirements and implementation activities to be represented separately.

16.6 Remediation and Evidence

Completion of a remediation activity should be supported by appropriate evidence.

The workflow therefore becomes:

Control

↓

Remediation

↓

Evidence Collected

↓

Evidence Integrity Verification

The evidence provides supporting information that the corrective action was actually implemented.

16.7 Remediation Verification

After remediation evidence has been collected, the implementation can be reviewed and tested.

The verification process can include:

- Reviewing the collected evidence.
- Confirming evidence integrity.
- Performing a control test.
- Recording the test result.
- Assessing whether the remediation addressed the original requirement.
- Recording the verification result.

This creates a stronger relationship between remediation completion and actual control effectiveness.

16.8 Example Remediation Workflow

An example from the authentication-related workflow can be represented as:

Authentication Risk

↓

CTRL-AUTH-001

↓

Authentication Monitoring

and Account Lockout

↓

Remediation Activity

↓

Configuration Evidence

↓

SHA-256 Integrity Verification

↓

Control Test

↓

Verification

This demonstrates that remediation is not treated simply as a status change; it becomes part of a traceable implementation and verification process.

16.9 Remediation and Risk Closure

Successful remediation contributes to the treatment of the original risk.

The overall relationship is:

Risk → Control → Remediation → Verification → Risk Treatment

Once the corrective action has been implemented and the associated control has been successfully verified, the risk can be considered for mitigation or closure according to its status and assessment.

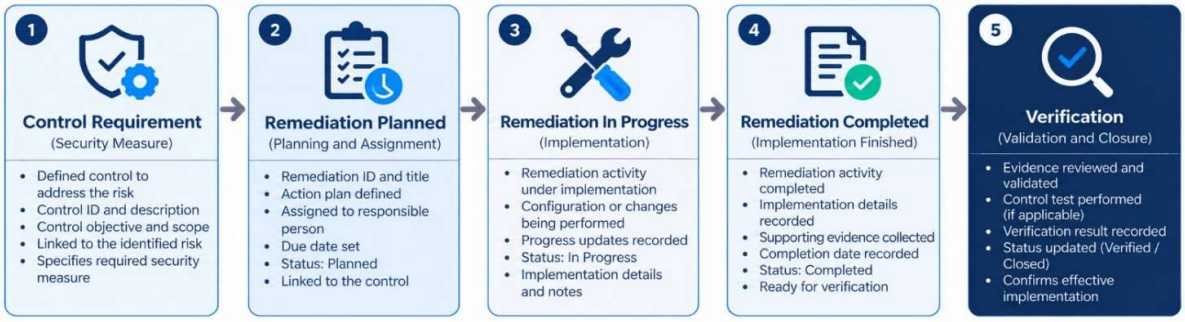
16.10 Remediation Design Principle

The Remediation component follows the principle:

> **A security control should be supported by a clearly defined corrective action whose implementation can be documented, evidenced and independently verified.**

This makes remediation the implementation bridge between **GRC requirements and measurable security improvement**.

Figure 11: AEGIS Remediation Lifecycle



17. EVIDENCE MANAGEMENT

17.1 Overview

Evidence management represents an important stage in the AEGIS-SOC-GRC workflow because it provides supporting information for remediation, control implementation and verification.

After a remediation activity has been completed, evidence can be collected to demonstrate what was implemented and provide a basis for subsequent testing.

The workflow is:

Remediation → Evidence → Integrity Verification → Control Test → Verification

17.2 Purpose of Evidence Management

The Evidence component was developed to:

- Collect supporting artifacts related to remediation and controls.
- Associate evidence with the relevant control.
- Record the source and type of evidence.
- Store evidence metadata.
- Maintain information about the collected artifact.
- Calculate and store a SHA-256 integrity value.
- Support analyst review of collected evidence.
- Support control testing.
- Provide evidence for verification and compliance activities.

17.3 Evidence Information

Each evidence record can contain information such as:

Information	Purpose
Evidence ID	Unique evidence identification
Title	Identifies the evidence
Description	Explains what the evidence demonstrates
Evidence Type	Categorizes the artifact
Control	Identifies the associated control
Source	Identifies where the evidence originated
Artifact	Identifies the collected file
File Path	Stores the artifact location
SHA-256	Records the artifact integrity value
Review Status	Records analyst review
Reviewer	Identifies the reviewer
Review Notes	Records review observations
Test Result	Records control-test outcome
Verification Result	Records final verification

17.4 Evidence Collection Process

Evidence can be collected as part of a remediation workflow.

The general process is:

Remediation Completed

↓

Evidence Identified

↓

Evidence Record Created

↓

Artifact Uploaded / Collected

↓

SHA-256 Calculated

↓

Evidence Ready for Review

This creates a structured connection between the corrective action and the artifact supporting its implementation.

17.5 Evidence Review

After evidence is collected, it can be reviewed by the responsible analyst.

The review records whether the artifact is considered suitable for supporting the associated control or remediation activity.

The review process can include:

- Reviewing the evidence title and description.
- Confirming the evidence is relevant to the associated control.
- Checking the artifact contents.
- Recording the reviewer.
- Recording review notes.
- Assigning an appropriate review status.

17.6 Example Evidence Record

A practical example implemented in the project is the authentication monitoring and account lockout configuration evidence.

The evidence is associated with:

`CTRL-AUTH-001`

and supports the remediation workflow related to the authentication security requirement.

The artifact documents the relevant authentication monitoring and account lockout configuration reviewed during the remediation process.

17.7 Control Testing

Evidence is also used as the basis for control testing.

The testing process can be represented as:

Evidence
↓
Control Test Procedure
↓
Test Result
↓
Findings
↓
Tester Notes

A control test can determine whether the collected evidence provides sufficient support for the associated control requirement.

17.8 Verification Assessment

After the evidence and control test have been reviewed, a final verification assessment can be performed.

The verification considers:

- Whether the evidence is relevant.
- Whether evidence integrity has been verified.
- Whether the control test passed.
- Whether the remediation addresses the original requirement.
- Whether sufficient information exists to support closure.

The resulting process is:

Evidence Review
↓
Integrity Verification
↓
Control Test
↓
Verification Assessment
↓
Remediation Effectiveness

17.9 Evidence and Compliance

Evidence provides the supporting material required to demonstrate that security controls have been implemented and tested.

The relationship is:

Compliance Requirement → Control → Evidence → Verification

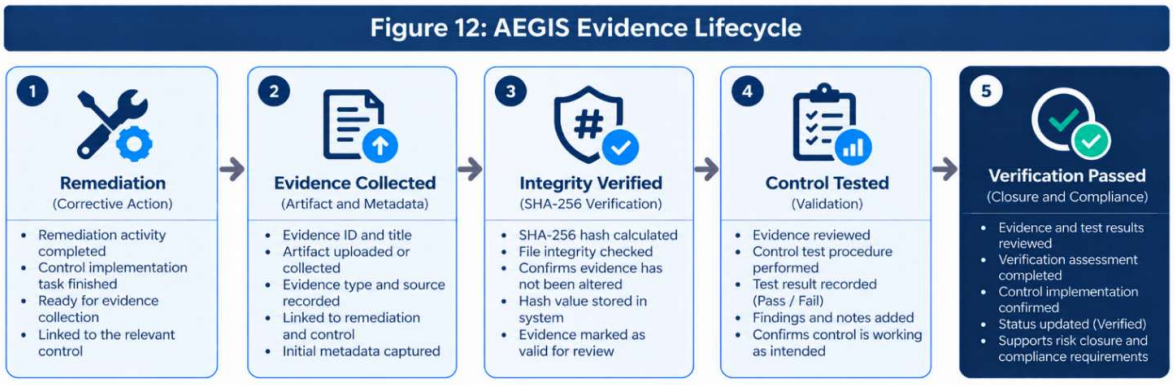
Therefore, evidence is not simply a stored file; it acts as a supporting artifact connecting technical implementation with governance and compliance activities.

17.10 Evidence Design Principle

The Evidence component follows the principle:

> **Security and compliance claims should be supported by identifiable, reviewable and verifiable evidence rather than relying only on recorded status values.**

This makes evidence a critical component of the AEGIS-SOC-GRC traceability model.



18. EVIDENCE INTEGRITY AND SHA-256 VERIFICATION

18.1 Overview

Evidence integrity is an important part of the AEGIS-SOC-GRC evidence workflow. Collecting an artifact alone is not sufficient; the platform should also provide a mechanism to determine whether the artifact remains consistent with the integrity value recorded when it was collected.

AEGIS-SOC-GRC uses **SHA-256 hashing** for this purpose.

The integrity workflow is:

Evidence Artifact → SHA-256 Hash → Stored Hash → Recalculate → Compare → Integrity Result

18.2 Why Evidence Integrity Is Required

Evidence may be used to support:

- Security remediation.
- Control implementation.
- Control testing.
- Verification assessments.
- Compliance activities.
- Audit activities.

Because these activities depend on the evidence being trustworthy and unchanged, the project includes an integrity-verification mechanism.

The purpose is not to prove that an artifact is inherently authentic. Instead, the mechanism verifies whether the current artifact produces the same SHA-256 value as the previously recorded value.

18.3 SHA-256

SHA-256 is a cryptographic hash function that produces a fixed-length hash value from input data.

For the project, the evidence artifact is processed to calculate its SHA-256 hash.

Conceptually:

Evidence File

↓

SHA-256 Calculation

↓

Hash Value

↓

Stored with Evidence Record

The resulting value acts as an integrity reference for the artifact.

18.4 Integrity Verification Process

When verification is requested, the system recalculates the SHA-256 hash of the current artifact.

The newly calculated value is then compared with the previously stored value.

Stored SHA-256

↓ **Compare**

Current Artifact → Recalculate SHA-256

↓

Match / Mismatch

If the values match, the artifact is considered **verified**.

If the values do not match, the integrity check is considered **failed**.

18.5 Integrity Status

The verification process produces an integrity status.

The relevant outcomes are:

- **Verified** — the calculated hash matches the stored hash.

- **Failed** — the calculated hash does not match the stored hash.

This result is stored with the evidence record and can be viewed through the Evidence interface.

18.6 Implementation in AEGIS-SOC-GRC

The integrity-verification process is implemented in the backend evidence service.

The backend:

1. Retrieves the selected evidence record.
2. Locates the associated artifact.
3. Reads the artifact content.
4. Calculates its SHA-256 hash.
5. Compares it with the stored hash.
6. Updates the integrity status.
7. Returns the updated evidence information.

This keeps the actual integrity calculation within the backend rather than relying on the frontend.

18.7 Example Verification

During project testing, an evidence artifact associated with the authentication monitoring and account lockout control was verified.

The verification process compared the stored SHA-256 value with a newly calculated value from the artifact.

The values matched, resulting in:

Integrity Status: Verified

This demonstrated that the artifact used for the subsequent control-testing workflow remained consistent with its recorded integrity value.

18.8 Integrity and Audit Trail

Evidence verification is also connected to the Audit Trail.

When an integrity verification is performed, an audit record can capture:

- Actor
- Action
- Evidence entity
- Description
- Previous integrity state
- New integrity state
- Source
- Timestamp

For example:

EVIDENCE_INTEGRITY_VERIFIED

↓

Evidence

↓

unverified → verified

↓

Audit Log

This provides traceability for the verification activity itself.

18.9 Integrity and Control Verification

SHA-256 verification is performed before the evidence is relied upon for the broader control-testing and verification workflow.

The resulting sequence is:

Evidence Collected

↓

SHA-256 Integrity Verification

↓

Control Test

↓

Verification Assessment

↓

Remediation Effectiveness

This adds an integrity-checking step between evidence collection and control validation.

18.10 Important Limitation

SHA-256 verification demonstrates **artifact consistency**, not complete evidence authenticity.

A matching hash indicates that the current artifact produces the same hash as the recorded reference. It does not independently establish who originally created the artifact, whether the original information was accurate, or whether the evidence was trustworthy before the hash was recorded.

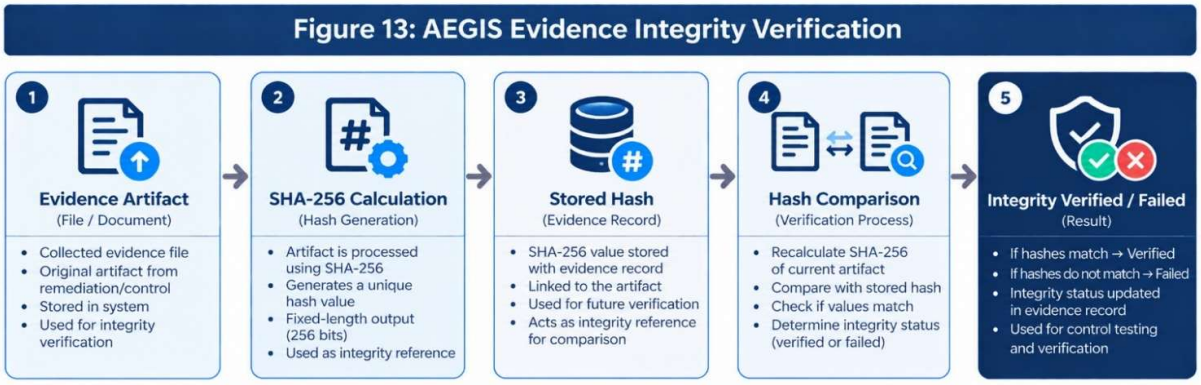
This distinction is important when describing the security capabilities of the project.

18.11 Design Principle

The Evidence Integrity component follows the principle:

> **Evidence used for security and governance decisions should be checked for consistency before it is relied upon for control testing and verification.**

This adds an additional integrity layer to the AEGIS-SOC-GRC evidence lifecycle.


Figure 13: AEGIS Evidence Integrity Verification
1
Evidence Artifact
(File / Document)
Collected evidence file
Original artifact from remediation/control
Stored in system
Used for integrity verification
2
SHA-256 Calculation
(Hash Generation)
Artifact is processed using SHA-256
Generates a unique hash value
Fixed-length output (256 bits)
Used as integrity reference
3
Stored Hash
(Evidence Record)
SHA-256 value stored with evidence record
Linked to the artifact
Used for future verification
Acts as integrity reference for comparison
4
Hash Comparison
(Verification Process)
Recalculate SHA-256 of current artifact
Compare with stored hash
Check if values match
Determine integrity status (verified or failed)
5
Integrity Verified / Failed
(Result)
If hashes match → Verified
If hashes do not match → Failed
Integrity status updated in evidence record
Used for control testing and verification

19. COMPLIANCE MANAGEMENT

19.1 Overview

Compliance management represents the governance layer of AEGIS-SOC-GRC. It connects implemented security controls and supporting evidence with compliance-related requirements.

The compliance workflow is:

Compliance Requirement → Control → Evidence → Verification → Compliance Status

This allows the platform to demonstrate how technical security activities can support governance and compliance objectives.

19.2 Purpose of Compliance Management

The Compliance component was developed to:

- Record compliance-related requirements.
- Associate requirements with relevant security controls.
- Connect controls with supporting evidence.
- Track verification of implemented controls.
- Provide visibility into compliance status.
- Support audit and governance activities.
- Maintain traceability between technical implementation and compliance requirements.

19.3 Compliance Information

A compliance record can contain information such as:

Information	Purpose
Compliance ID	Unique compliance record
Framework	Identifies the compliance framework
Requirement ID	Identifies the specific requirement

Description	Explains the requirement
Control	Identifies the supporting security control
Evidence	Provides supporting documentation
Status	Indicates the current compliance state
Verification	Records validation information

19.4 Compliance and Controls

Controls form the primary connection between security implementation and compliance requirements.

The relationship is:

Compliance Requirement

↓

Security Control

↓

Control Implementation

For example, a compliance requirement related to authentication security can be supported by an authentication monitoring and account lockout control.

19.5 Compliance and Evidence

A compliance requirement should be supported by appropriate evidence demonstrating that the associated control has been implemented.

The relationship is:

Compliance Requirement

↓

Control

↓

Evidence

↓

Verification

This ensures that compliance information is connected to actual implementation evidence rather than existing only as a status value.

19.6 Verification and Compliance Status

Verification results provide supporting information for determining whether a control satisfies the associated requirement.

The simplified process is:

Control Implemented

↓

Evidence Collected

↓

Evidence Integrity Verified

↓

Control Tested

↓

Verification Result

↓

Compliance Status

A successful verification therefore contributes to the overall compliance assessment.

19.7 Compliance and Audit Trail

Compliance-related activities can also be connected with the Audit Trail.

Important activities such as evidence verification and control-related changes can be recorded as audit events.

This provides historical traceability for governance activities.

Compliance Activity

↓

Verification / Change

↓

Audit Event

↓

Historical Record

19.8 Example Compliance Mapping

A simplified AEGIS compliance mapping can be represented as:

Compliance Requirement

↓

CTRL-AUTH-001

↓

Authentication Monitoring

and Account Lockout

↓

Configuration Evidence

↓

Control Test: PASS

↓

Verification: PASS

This demonstrates how a technical control can provide supporting evidence for a governance requirement.

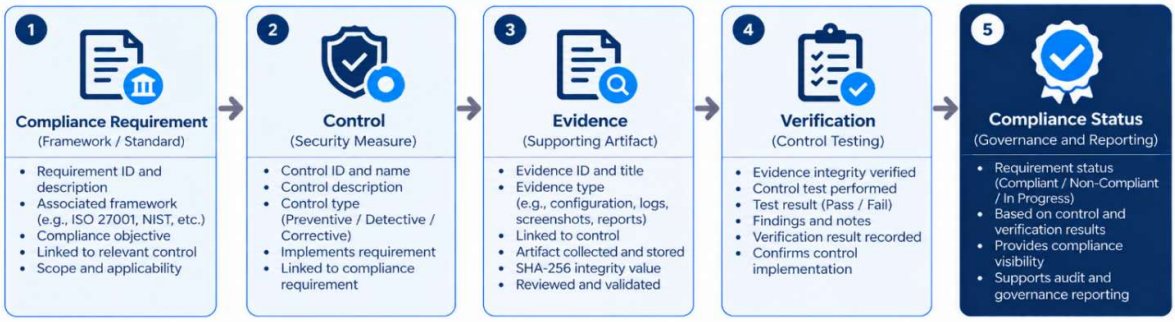
19.9 Compliance Design Principle

The Compliance component follows the principle:

> **Compliance status should be supported by implemented controls, relevant evidence and verification results so that governance decisions remain traceable to technical activity.**

This makes compliance a connected part of the AEGIS-SOC-GRC lifecycle rather than a separate reporting function.

Figure 14: AEGIS Compliance Traceability Flow



20. AUDIT TRAIL AND ACTIVITY TRACEABILITY

20.1 Overview

The Audit Trail represents the final traceability layer of the AEGIS-SOC-GRC platform.

While the other modules primarily represent the **current state** of security and GRC records, the Audit Trail preserves a historical record of important activities and changes performed within the platform.

The overall relationship is:

Security Activity → GRC Activity → Verification → Audit Trail

20.2 Purpose of the Audit Trail

The Audit Trail was developed to:

- Record important security and governance activities.
- Maintain historical traceability of changes.
- Identify the actor responsible for an activity.
- Record when an activity occurred.
- Identify the affected entity.
- Record previous and new values when applicable.
- Identify the source of the activity.
- Support security investigations and governance reviews.
- Provide supporting information for auditing and compliance.

20.3 Audit Log Information

Each audit record can contain:

Information	Purpose
Audit ID	Uniquely identifies the audit record
Timestamp	Records when the activity occurred
Actor	Identifies who or what performed the action
Action	Describes the activity
Entity Type	Identifies the affected entity
Entity ID	Identifies the specific record
Description	Provides additional context
Old Value	Records the previous value where applicable
New Value	Records the resulting value where applicable
Source	Identifies the originating module

20.4 Why an Audit Trail Was Added

During development, it became important to distinguish between **what the system currently says** and **what happened previously**.

For example, an alert may currently have the status resolved. Without an audit record, the current alert record does not necessarily provide a complete historical explanation of how that status was reached.

The Audit Trail can instead record:

Alert Status Changed

Open → Resolved

along with the timestamp, actor, entity and source.

This provides historical context in addition to the current database state.

20.5 Audit Event Generation

Important operations can generate audit events.

The simplified process is:

Platform Activity
↓
Important State Change / Action
↓
Audit Service
↓
Audit Log Created
↓
Audit Trail

The audit service provides a centralized mechanism for recording these activities.

20.6 Alert Audit Example

When an alert status changes, the system records an audit event.

For example:

```
Entity: Alert
Action: ALERT_STATUS_CHANGED
Actor: system

Old Value: open
New Value: resolved
Source: alerts
```

This allows the status transition to be reviewed later.

20.7 Evidence Integrity Audit Example

Evidence integrity verification is another important operation recorded by the Audit Trail.

For example:

Entity: Evidence

Action: EVIDENCE_INTEGRITY_VERIFIED

Actor: SOC Analyst

Old Value: unverified

New Value: verified

Source: evidence

This creates traceability for the integrity-verification operation itself.

20.8 Audit Trail Interface

The frontend Audit Trail page provides a centralized view of recorded activities.

The interface includes:

- Total event count.
- Entities tracked.
- Visible event count.
- Activity register.
- Search functionality.
- Action filtering.
- Entity filtering.
- Expandable audit records.
- Detailed descriptions.
- Previous and new values.
- Source information.
- Refresh functionality.

This allows an analyst to review historical activity without directly querying the database.

20.9 Audit Trail and SOC

The Audit Trail complements the SOC workflow by preserving important operational activity.

For example:

Detection

↓

Alert

↓

Incident

↓

Status / Investigation Changes

↓

Audit Trail

This provides historical visibility into how security records progressed during investigation.

20.10 Audit Trail and GRC

The same concept applies to the GRC workflow.

Risk → Control → Remediation → Evidence → Verification → Audit Trail

This makes the Audit Trail useful for demonstrating that governance activities were actually performed and recorded.

20.11 Audit Trail as the Final Traceability Layer

The complete AEGIS-SOC-GRC lifecycle can therefore be represented as:

Scenario
↓
Telemetry
↓
Detection
↓
Alert
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
Integrity Verification
↓
Compliance
↓
Audit Trail

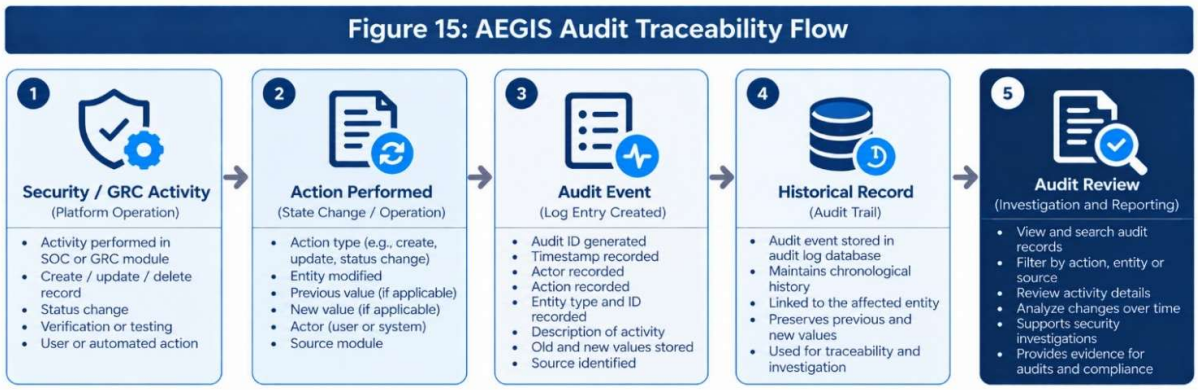
The Audit Trail does not replace the individual modules. Instead, it provides a historical layer across important activities performed within them.

20.12 Design Principle

The Audit Trail follows the principle:

> **Important security and governance activities should leave a traceable historical record so that changes can be reviewed, investigated and demonstrated during governance or audit activities.**

This completes the traceability objective of AEGIS-SOC-GRC by connecting operational activity with historical accountability.



21. REPORTS AND ANALYTICS

21.1 Overview

Reports and analytics provide a consolidated view of information generated across the AEGIS-SOC-GRC platform.

While individual modules are designed for specific operational activities, the Reports component brings relevant information together so that security and GRC information can be reviewed at a higher level.

The reporting layer therefore supports:

Security Data → Analysis → Summary → Reporting

21.2 Purpose of Reports and Analytics

The Reports component was developed to:

- Provide a consolidated view of project information.
- Summarize security and GRC activity.
- Support analysis of platform records.
- Present information in a more readable format.
- Help identify trends and important security information.
- Provide a convenient way to review generated records.
- Support management and governance-oriented visibility.

21.3 Reporting Areas

The reporting functionality can present information originating from different areas of the platform.

Examples include:

- Security events

- Alerts
- Incidents
- Risks
- Controls
- Remediation activities
- Evidence
- Compliance information
- Audit activity

This allows reporting to reflect the connected SOC and GRC workflow rather than focusing only on individual security events.

21.4 Relationship with the Dashboard

The Dashboard provides a high-level operational overview of the platform.

Reports provide a more detailed view of selected information and allow records to be examined beyond the high-level summary.

The relationship can be represented as:

Platform Data → Dashboard → Reports → Detailed Analysis

This provides two levels of visibility:

- **Dashboard** — quick operational overview.
- **Reports** — deeper review of available information.

21.5 Report Information

A report can contain information such as:

Information	Purpose
Report Title	Identifies the report
Report Type	Defines the reporting category

Description	Explains the report
Reporting Data	Provides the underlying information
Generated Date	Identifies when the report was generated
Status	Indicated the report state
Details	Provides expanded information

21.6 Report Detail View

The platform includes a detailed report view that allows an individual report to be examined without leaving the main reporting context.

The detail view provides a focused interface for reviewing the selected report and its associated information.

The implementation uses a dedicated overlay/panel approach so that the report details appear above the main application rather than becoming part of the normal page layout.

21.7 Report Filtering and Review

Reporting becomes more useful when information can be narrowed to relevant records.

The platform therefore provides mechanisms for reviewing report information according to the available report data and context.

The objective is to allow the user to move from:

Summary → Selected Report → Detailed Information

without having to manually search through unrelated platform records.

21.8 Reporting and GRC

Reports also support the governance side of the platform.

GRC information can be reviewed through summarized information about:

- Identified risks.
- Control implementation.

- Remediation progress.
- Evidence collection.
- Verification results.
- Compliance status.
- Audit activity.

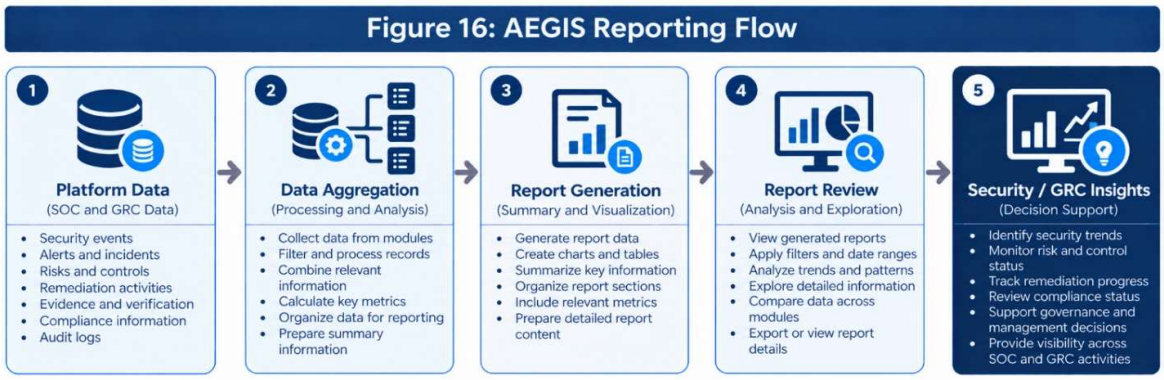
This makes the reporting layer useful not only for SOC monitoring but also for governance and management review.

21.9 Reporting Design Principle

The Reports component follows the principle:

> **Operational and governance data should be presented in a consolidated format so that security activity, risk treatment and compliance information can be reviewed efficiently.**

The reporting layer therefore acts as a presentation and analysis layer over the underlying SOC and GRC data.



22. FRONTEND DESIGN AND USER INTERFACE

22.1 Overview

The frontend of AEGIS-SOC-GRC was developed using **React and TypeScript** to provide a centralized interface for interacting with the SOC and GRC modules.

The interface was designed around the project's security lifecycle so that related functions remain accessible from a common application rather than being separated into independent tools.

22.2 Frontend Design Objectives

The frontend was designed to provide:

- Clear navigation between SOC and GRC modules.
- Consistent visual presentation across pages.
- Easy access to security and governance information.
- Search and filtering capabilities where required.
- Expandable detail views for individual records.
- Clear status and severity indicators.
- Responsive layouts for different screen sizes.
- Error and loading-state feedback.
- A professional security-platform appearance.

22.3 Application Navigation

The application uses a centralized navigation structure to provide access to the major platform modules.

The navigation includes areas such as:

- Dashboard

- Alerts
- Incidents
- Detection Rules
- Scenarios
- Assets
- Risks
- Controls
- Remediation
- Evidence
- Compliance
- Reports
- Audit Trail

This organization reflects the relationship between the operational SOC functions and the GRC workflow.

22.4 Dashboard Design

The Dashboard acts as the initial overview of the platform.

It provides a high-level representation of important platform information so that the user can quickly understand the current state of the environment.

The Dashboard is intended for **overview and orientation**, while individual pages provide deeper investigation and management capabilities.

22.5 Table-Based Data Presentation

Many AEGIS modules use structured tables to present records.

Tables are suitable for security and GRC information because they allow multiple records to be compared using consistent fields.

Examples include:

- Alert severity and status.
- Incident information.
- Risk levels and scores.
- Control status.
- Evidence status.
- Audit activity.

22.6 Detail Views

For records requiring additional context, the frontend provides detailed views rather than displaying every field directly in the main table.

This approach keeps the primary interface compact while still allowing the user to inspect individual records.

A typical interaction is:

Record List

↓

Select Record

↓

Detail View

↓

Related Information

22.7 Search and Filtering

Search and filtering features were implemented where they provide practical value.

They allow users to narrow large sets of records and locate relevant security or GRC information more efficiently.

The Audit Trail, for example, supports searching and filtering by audit activity and entity-related information.

22.8 Status and Severity Visualization

Security information often requires quick visual interpretation.

The frontend therefore uses visual indicators for concepts such as:

- Alert severity.
- Alert status.
- Incident status.
- Risk level.
- Risk status.
- Control status.
- Evidence status.
- Verification results.

These indicators allow important information to be recognized without requiring the user to read every field.

22.9 Loading and Error States

The frontend includes loading and error handling for API-driven pages.

The general interaction is:

Page Opened

↓

API Request

↓

Loading State

↓

Successful Response

OR

Error State

This prevents the interface from appearing unresponsive while backend information is being retrieved.

22.10 API Integration

The React frontend communicates with the FastAPI backend through REST API calls.

The basic interaction is:

User Interaction

↓

React Component

↓

API Request

↓

FastAPI Endpoint

↓

Database / Service Logic

↓

API Response

↓

React State

↓

Updated UI

This separation allows the frontend to focus on presentation and interaction while the backend handles data processing and persistence.

22.11 Page State Persistence

During development, an issue was identified where the selected page returned to the Dashboard after a browser refresh.

The behaviour was improved by storing the active page in browser localStorage.

The resulting flow is:

Selected Page

↓

localStorage

↓

Browser Refresh

↓

Restore Previous Page

This improved usability by allowing the application to retain the user's current navigation context after a refresh.

22.12 Report Detail Overlay

The Reports module presented another frontend implementation challenge.

The report-detail panel initially appeared within the normal page layout instead of behaving as a true overlay.

The implementation was improved by rendering the detail panel through a React portal attached to the document body.

The resulting structure is:

Reports Page

↓

Select Report

↓

Portal-Based Detail Overlay

↓

Focused Report Information

This allowed the detail panel to appear above the dashboard and other page content as intended.

22.13 Consistent Visual Language

The frontend uses a consistent visual language across the platform.

The major visual characteristics include:

- Dark navy primary elements.
- AEGIS blue accents.
- Light blue information areas.
- Status badges.
- Rounded panels.
- Structured tables.
- Clear section headings.
- Consistent spacing.
- Expandable information areas.

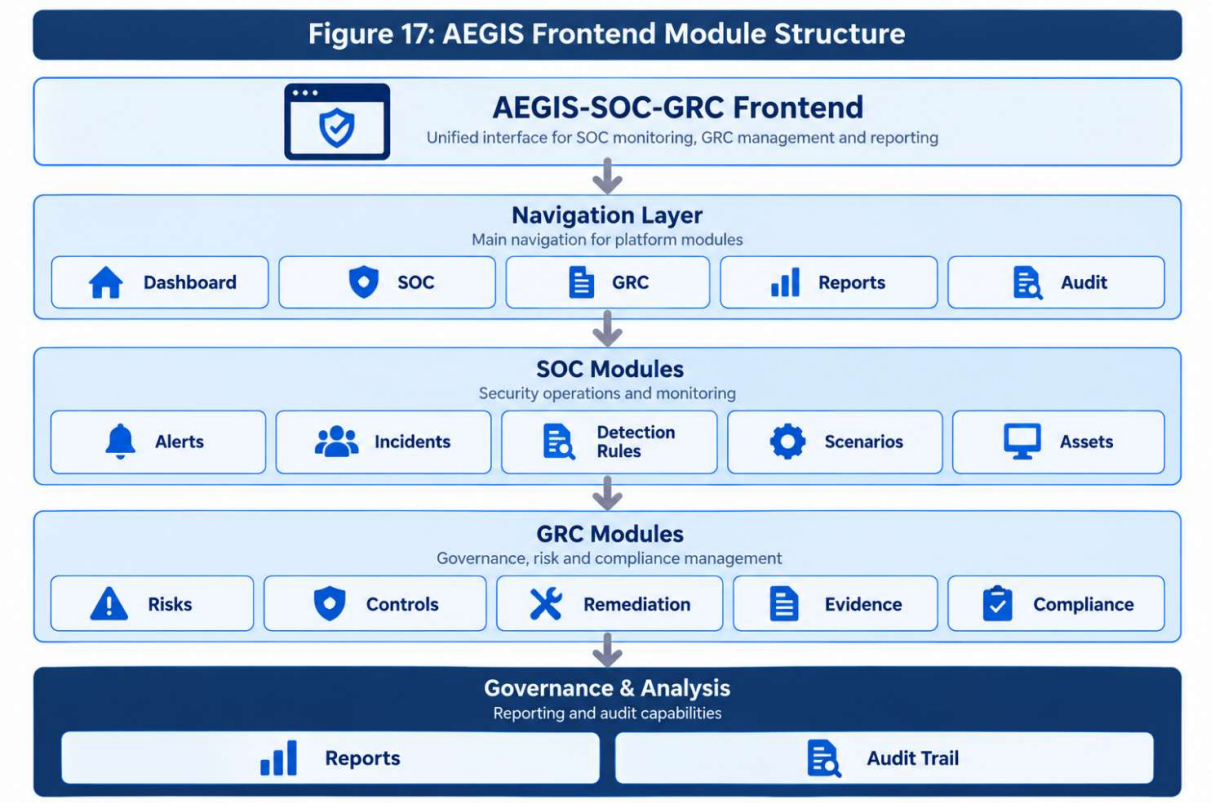
This consistency helps make the application feel like one integrated platform rather than a collection of unrelated pages.

22.14 Frontend Design Principle

The frontend follows the principle:

> **Security information should be easy to scan at a high level while remaining accessible in sufficient detail for investigation, management and verification.**

The interface therefore balances **summary information, detailed investigation and workflow management** within a single application.


Figure 17: AEGIS Frontend Module Structure
AEGIS-SOC-GRC Frontend
Unified interface for SOC monitoring, GRC management and reporting
Navigation Layer
Main navigation for platform modules
Dashboard
SOC
GRC
Reports
Audit
SOC Modules
Security operations and monitoring
Alerts
Incidents
Detection Rules
Scenarios
Assets
GRC Modules
Governance, risk and compliance management
Risks
Controls
Remediation
Evidence
Compliance
Governance & Analysis
Reporting and audit capabilities
Reports
Audit Trail

23. DEVELOPMENT TIMELINE AND IMPLEMENTATION JOURNEY

23.1 Overview

AEGIS-SOC-GRC was developed incrementally rather than being implemented as a single complete system from the beginning.

The project evolved as new requirements and relationships were identified. The initial focus was on security monitoring and detection, which gradually expanded into a connected SOC and GRC platform.

The development journey can be summarized as:

Idea → Architecture → Environment Setup → Database → Backend → SOC → GRC → Evidence → Compliance → Audit → Frontend Refinement → Testing

23.2 Initial Project Concept

The project initially focused on the idea of building a practical security monitoring and detection platform.

The initial objective was to demonstrate how security events could be generated, processed and detected.

At this stage, the core concept was:

Security Activity → Detection → Alert

23.3 Architecture and Environment Setup

After defining the initial concept, the application architecture and development environment were established.

The major activities included:

- Selecting the frontend and backend technologies.
- Establishing the PostgreSQL database environment.

- Configuring Docker.
- Setting up the Python development environment.
- Setting up the React and TypeScript frontend.
- Establishing communication between frontend and backend.
- Organizing the project into logical modules.

23.4 Database Development

Once the environment was established, the database layer was developed to provide persistent storage.

The database gradually expanded as additional project requirements were introduced.

The initial security-related entities were later extended to support:

Alerts → Incidents → Risks → Controls → Remediation → Evidence → Compliance → Audit Logs

This allowed the database to evolve together with the project's functional scope.

23.5 Backend API Development

The FastAPI backend was then developed to provide the application interface and business logic.

Backend development included:

- API endpoint creation.
- Database integration.
- Request and response schemas.
- Service-layer logic.
- Security event processing.
- Detection processing.
- Alert management.
- Incident management.

- GRC workflows.
- Evidence operations.
- Integrity verification.
- Audit logging.

23.6 Security Telemetry and Detection

The next stage focused on the SOC functionality.

Security telemetry was introduced so that security activity could be represented as structured events.

Detection logic was then implemented to evaluate these events and identify relevant security conditions.

The resulting workflow became:

Security Telemetry → Detection Rule → Detection Result → Alert

23.7 Scenario Engine Development

The AEGIS Scenario Engine was subsequently developed to provide repeatable and controlled security simulations.

Instead of manually creating individual security events for every test, reusable scenarios were introduced.

The Scenario Engine therefore provided:

- Reusable scenarios.
- Multiple security categories.
- Synthetic telemetry generation.
- Detection-rule associations.
- Expected outcomes.
- Scenario execution and testing.

This made the SOC workflow easier to demonstrate and repeatedly test.

23.8 Alert and Incident Development

After detection was functional, the project expanded into alert and incident management.

The workflow became:

Scenario → Telemetry → Detection → Alert → Incident

This provided a more realistic representation of how security operations progress from observed activity to investigation.

23.9 Expansion into GRC

The next major expansion occurred when the project moved beyond security detection.

The question became:

> **What happens after a security issue has been identified and investigated?**

This led to the addition of:

- Risk management.
- Controls.
- Remediation.
- Evidence.
- Verification.
- Compliance.

The project therefore evolved from a primarily SOC-focused system into the integrated **AEGIS-SOC-GRC** platform.

23.10 Evidence and Integrity Implementation

Evidence management was introduced to provide supporting artifacts for remediation and controls.

The evidence workflow was subsequently extended with SHA-256 integrity verification.

The implementation therefore progressed from:

Evidence Collection → Integrity Verification → Control Testing → Verification

This added a stronger verification layer to the GRC workflow.

23.11 Compliance Implementation

Compliance functionality was added to connect controls and supporting evidence with governance requirements.

The resulting relationship became:

Compliance Requirement → Control → Evidence → Verification

This allowed the project to demonstrate how technical implementation can support governance and compliance activities.

23.12 Audit Trail Implementation

The Audit Trail was introduced as the final traceability layer.

The objective was to record important activities such as:

- Alert status changes.
- Alert creation.
- Evidence integrity verification.
- Other significant platform actions.

This extended the overall workflow to:

```
... → Compliance → Audit Trail
```

23.13 Frontend Refinement

After the core backend workflows were functional, the frontend was progressively refined.

The refinement included:

- Consistent page layouts.
- Improved tables.
- Detail panels.
- Search and filtering.
- Loading states.
- Error handling.
- Status indicators.
- Evidence workflow presentation.
- Audit Trail interface.
- Report detail overlay.
- Page-state persistence.

These changes focused on usability and presentation without changing the underlying security workflow.

23.14 Important Frontend Fixes

Two notable usability issues were identified and resolved during final refinement.

Report Detail Overlay

The report detail panel initially appeared within the normal page layout.

The implementation was changed to use a React portal so that the detail panel is rendered above the main application content as a proper overlay.

Page Persistence

The selected page initially returned to the Dashboard after a browser refresh.

The active page was stored in `localStorage`, allowing the application to restore the previously selected page after refresh.

These fixes demonstrate that the development process included usability testing and iterative improvement rather than only feature implementation.

23.15 Final Development Lifecycle

The complete development journey can now be represented as:

Project Idea
↓
Architecture
↓
Environment Setup
↓
Docker + PostgreSQL
↓
Database
↓
Backend APIs
↓
Security Telemetry
↓
Detection Rules
↓
Scenario Engine
↓
Alerts
↓
Incidents
↓
Risk
↓
Controls
↓
Remediation
↓
Evidence
↓
SHA-256 Integrity
↓
Compliance
↓
Audit Trail
↓
Reports & Frontend Refinement
↓
Testing & Validation

23.16 Development Philosophy

The project followed an **incremental development approach**.

Each major capability was added after the previous layer provided a working foundation. This reduced the need to build the complete platform simultaneously and made it possible to test individual components before connecting them to subsequent stages.

> **The final AEGIS-SOC-GRC platform is the result of progressively connecting individual cybersecurity and GRC capabilities into one traceable lifecycle.**

Figure 18: AEGIS Development Timeline



24. DEVELOPMENT CHALLENGES, ERRORS AND TROUBLESHOOTING

24.1 Overview

The development of AEGIS-SOC-GRC involved several technical challenges across the environment setup, database, backend, security simulation and frontend layers.

These issues were handled through an iterative troubleshooting process rather than being treated as isolated errors.

The general troubleshooting approach followed:

Problem → Observation → Investigation → Root Cause → Fix → Retest → Result

This section documents the major challenges encountered during development and the solutions applied.

24.2 Docker and PostgreSQL Connectivity

Problem

The backend initially experienced difficulties connecting to the PostgreSQL database running through Docker.

Observation

The application produced connection-related errors indicating that the Docker database environment was not always accessible through the expected Docker API or database connection configuration.

Investigation

The Docker Desktop status, container state, port mapping and database connection configuration were checked.

The PostgreSQL container and its exposed port were also verified to determine whether the backend was attempting to connect to the correct database endpoint.

Root Cause

The issue was related to the Docker/database environment and connection configuration rather than the FastAPI application logic itself.

Fix

The Docker environment and PostgreSQL container configuration were corrected, and the backend database connection settings were aligned with the active container configuration.

Retest

The backend was restarted and database-dependent operations were executed again.

Result

The application successfully communicated with the PostgreSQL database.

24.3 Python Environment and Dependency Issues

Problem

Several Python environment and dependency issues occurred during backend development.

Examples included missing Python packages and differences between Python installations and virtual environments.

Observation

Errors such as:

`ModuleNotFoundError`

were produced when required packages were not available in the active Python environment.

Investigation

The active Python interpreter, virtual environment and installed dependencies were checked.

The project also contained references to different Python versions during development, which required additional verification of the environment being used to execute the backend.

Root Cause

The required dependency was not available in the Python environment being used by the application.

Fix

The correct virtual environment and dependencies were configured, and missing packages were installed into the appropriate environment.

Retest

The backend was started again and the affected module was imported and executed.

Result

The dependency errors were resolved and the backend could continue execution.

24.4 Scenario Engine Import Issue

Problem

The Scenario Engine encountered an import-related issue during development.

Observation

The development server detected changes to:

`simulator\scenario_engine.py`

and reloaded while the Scenario Engine was being tested.

The import behaviour was also explicitly checked during troubleshooting.

Investigation

The Scenario Engine module structure, import paths and execution context were examined.

The module was tested independently to determine whether the issue was caused by the Scenario Engine itself or by the way it was being imported and executed.

Fix

The Scenario Engine imports and execution structure were corrected so that the module could be loaded properly within the project environment.

Retest

The Scenario Engine was executed and the scenario catalogue was successfully loaded.

The following functionality was tested:

```
--list
```

```
--scenario
```

```
--scenario ALL
```

Result

The Scenario Engine successfully loaded and executed its controlled scenarios.

24.5 Security Event and Detection Testing

Problem

Initial detection tests did not always produce alerts as expected.

Observation

A detection run could complete successfully while producing:

```
alerts_created: 0
```

even though security events had been generated.

Investigation

The security-event records, detection rules and detection conditions were examined separately.

The database was also queried to confirm whether the expected security events had actually been stored.

Root Cause

The issue was associated with the relationship between the generated telemetry and the conditions expected by the detection logic.

Fix

The event generation and detection workflow were aligned so that the generated telemetry satisfied the intended detection conditions.

Retest

The scenario was executed again and the detection endpoint was called.

The detection results were then checked for newly generated alerts and incidents.

Result

The detection workflow successfully produced alerts from the simulated security activity.

24.6 Database Schema and Relation Issues

Problem

Some database queries initially returned errors indicating that expected relations did not exist.

Examples included references to relations such as:

events

`detection_rules`

Observation

PostgreSQL returned relation-not-found errors when queries were executed against table names that did not correspond to the currently implemented database schema.

Investigation

The actual database tables were inspected and compared with the model and API implementation.

This helped distinguish between:

- Incorrect table names.
- Existing SQLAlchemy model names.
- Actual database table names.
- Tables that had not yet been created.

Fix

Database queries and implementation references were aligned with the actual schema.

Where required, database initialization was also performed again after model changes.

Retest

The database tables were queried again and the relevant records were successfully retrieved.

Result

Database access became consistent with the implemented schema.

24.7 Evidence Integrity Verification

Problem

The evidence-integrity workflow required verification that the stored SHA-256 value actually matched the physical evidence artifact.

Investigation

The artifact was located and its SHA-256 hash was independently calculated.

The calculated value was compared with the value stored in the database.

Fix

The evidence artifact path was corrected so that the backend could access the intended file.

Retest

The integrity endpoint was executed again.

The calculated hash matched the stored value.

Result

The evidence record was updated to:

Integrity Status: Verified

The verification activity was also recorded in the Audit Trail.

24.8 Report Detail Panel Positioning

Problem

The report detail panel initially appeared underneath the dashboard/page content instead of functioning as a proper overlay.

Investigation

The DOM placement and CSS positioning of the report detail panel were examined.

It was determined that the panel was still being rendered within the normal page structure.

Fix

The report detail panel was changed to use a **React portal**, rendering the overlay through `document.body`.

Additional fixed-position overlay styling was applied.

Retest

A report was selected again from the Reports page.

Result

The report detail panel appeared correctly above the application content as a focused modal-style overlay.

24.9 Page Persistence After Refresh

Problem

The application returned to the Dashboard whenever the browser was refreshed, even when another module was currently open.

Investigation

The `activePage` state was examined.

The original implementation initialized the state directly to:

Dashboard

which meant that the selected page was lost when the React application restarted.

Fix

The active page was stored in browser `localStorage`.

The application now reads the saved page during initialization and updates the stored value whenever the active page changes.

Retest

A non-Dashboard page was selected and the browser was refreshed.

Result

The application restored the previously selected page successfully.

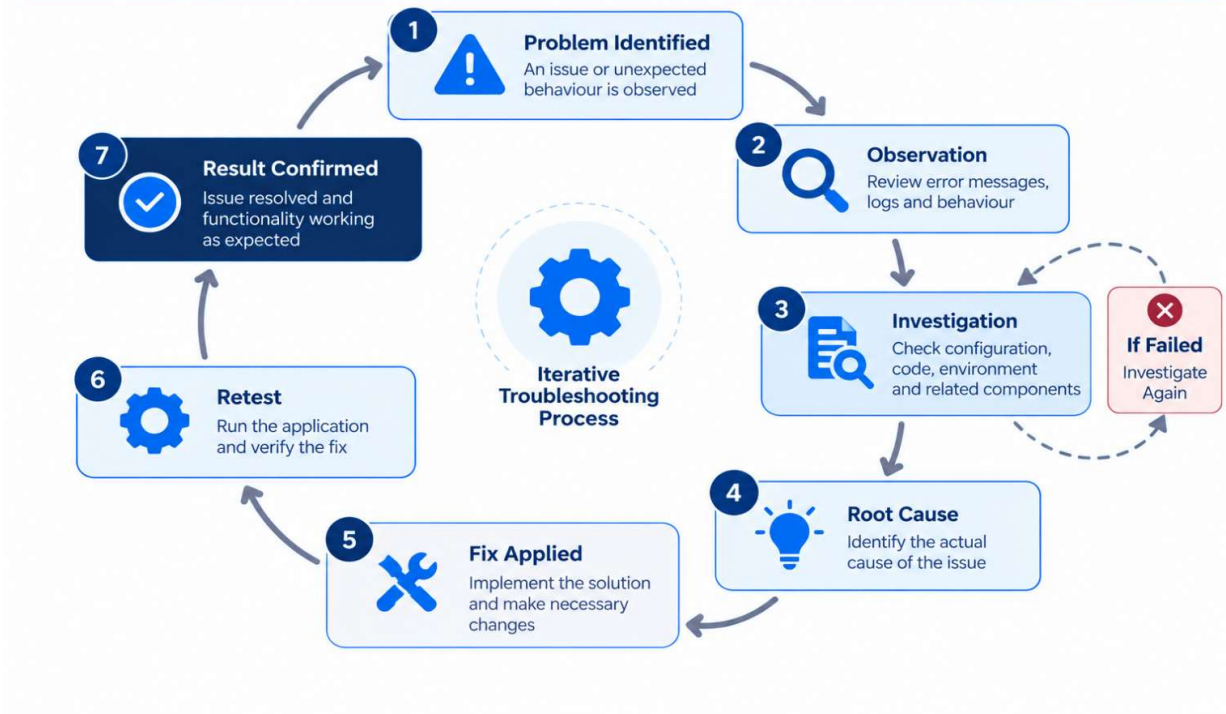
24.10 Lessons from Troubleshooting

The development issues demonstrated several important engineering principles:

- Environment problems should be separated from application-code problems.
- Database connectivity should be verified independently from API logic.
- Detection failures should be investigated from telemetry through detection conditions rather than only from the final alert.
- Database models and actual database relations should be verified together.
- Evidence verification should be tested against the physical artifact.
- Frontend problems should be investigated at both the React and CSS/DOM levels.
- Every significant fix should be followed by a retest.

> **Troubleshooting Principle:**
> "An error message identifies where a problem became visible; investigation is required to determine where the problem actually originated."

Figure 19: AEGIS Troubleshooting Methodology



25. TESTING AND VALIDATION

25.1 Overview

Testing was performed throughout the development of AEGIS-SOC-GRC rather than only after the entire platform had been completed.

Individual components were tested during implementation and were subsequently tested again after integration with other components.

The overall objective was to confirm that the platform behaved correctly from **security simulation through detection, GRC processing, evidence verification and auditing**.

25.2 Testing Objectives

The testing process was designed to verify that:

- Individual modules function correctly.
- Frontend and backend communication works correctly.
- Database records are created and retrieved correctly.
- Security scenarios generate the expected telemetry.
- Detection rules identify the intended conditions.
- Alerts are created correctly.
- Incidents can be associated with detected activity.
- Risks, controls and remediation records maintain the intended relationships.
- Evidence can be collected and reviewed.
- SHA-256 integrity verification works correctly.
- Compliance information remains traceable to controls and evidence.
- Audit events are generated for important activities.
- Frontend interactions behave correctly.
- Important UI fixes remain functional after integration.

25.3 Testing Approach

The project followed an incremental testing approach.

Component Development

↓

Component Testing

↓

Integration Testing

↓

Workflow Testing

↓

Frontend Testing

↓

End-to-End Validation

This approach made it possible to identify problems close to the component where they were introduced.

25.4 Backend API Testing

Backend endpoints were tested individually to confirm that they could:

- Accept valid requests.
- Return the expected response structure.
- Retrieve database records.
- Create new records.
- Update existing records.
- Return appropriate errors when records were not found.
- Maintain relationships between related entities.

API testing was also performed during development through direct endpoint requests and application execution.

25.5 Database Testing

Database testing focused on:

- Table creation.
- Record insertion.
- Record retrieval.
- Record updates.
- Entity relationships.
- Database persistence.
- Verification of generated security records.

Database queries were also used during troubleshooting to confirm whether expected telemetry, alerts, evidence and audit records had actually been stored.

25.6 Scenario Engine Testing

The AEGIS Scenario Engine was tested using its supported execution options.

The scenario catalogue was checked using:

```
python simulator/scenario_engine.py –list
```

Individual scenarios were also executed for testing.

The engine was additionally tested using grouped and complete scenario execution where applicable.

The purpose was to verify that scenarios could generate the expected synthetic security telemetry without executing real attacks.

25.7 Detection Testing

Detection testing verified the relationship between generated telemetry and detection rules.

A representative test followed:

Scenario

↓

Synthetic Security Events

↓

Detection Rule

↓

Detection Result

↓

Alert

The detection endpoint was executed after scenario generation, and the resulting alert information was checked.

Successful tests produced detection results and associated alerts.

25.8 Alert Testing

Alert testing included:

- Alert creation.
- Severity assignment.
- Status display.
- Alert detail expansion.
- Status changes.
- Backend persistence.
- Audit recording of important status changes.

A status-change test also verified that the previous and new values were recorded in the Audit Trail.

25.9 Incident Testing

Incident functionality was tested to confirm that detected security conditions could progress into an investigation context.

The testing focused on:

- Incident creation.
- Incident information.
- Related security activity.
- Status handling.
- Incident-to-risk progression.

25.10 Risk and GRC Testing

The GRC workflow was tested progressively:

Incident

↓

Risk

↓

Control

↓

Remediation

↓

Evidence

↓

Verification

The objective was to confirm that each stage could maintain its relationship with the preceding stage and provide information required by the following stage.

25.11 Evidence Testing

Evidence testing included:

- Creating an evidence record.
- Associating evidence with a control.

- Uploading/collecting an artifact.
- Recording artifact metadata.
- Generating the SHA-256 value.
- Reviewing the evidence.
- Recording control-test information.
- Recording verification information.

A real evidence artifact was used during testing rather than relying only on static frontend data.

25.12 SHA-256 Integrity Testing

Integrity verification was specifically tested by independently calculating the SHA-256 value of the evidence artifact and comparing it with the value stored by the platform.

The test followed:

Evidence Artifact

↓

Calculate SHA-256

↓

Compare with Stored Hash

↓

Match

↓

Integrity: VERIFIED

The successful verification was then reflected in the evidence record and associated audit event.

25.13 Audit Trail Testing

The Audit Trail was tested using real platform activity.

Examples of captured audit activities included:

- Alert creation.

- Alert status changes.
- Evidence integrity verification.

The Audit Trail interface was then checked to confirm that the recorded activity could be retrieved and displayed.

This demonstrated that the Audit Trail was connected to backend activity rather than being populated with static frontend records.

25.14 Frontend Testing

Frontend testing included:

- Navigation between modules.
- API data loading.
- Search functionality.
- Filtering.
- Detail panels.
- Status badges.
- Loading states.
- Error states.
- Refresh operations.
- Responsive behaviour.
- Modal/overlay behaviour.

The frontend was also tested after backend changes to ensure that API response changes did not break the interface.

25.15 Regression Testing

Regression testing was performed after significant fixes and feature additions.

The purpose was to ensure that fixing one component did not introduce problems in another component.

Examples included:

- Testing alerts after audit integration.
- Testing evidence after integrity verification was added.
- Testing reports after the portal-based overlay fix.
- Testing navigation after page persistence was introduced.

25.16 End-to-End Testing

The most important validation was the complete security-to-GRC workflow.

The end-to-end test followed:

Scenario Engine
↓
Security Telemetry
↓
Detection
↓
Alert
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
SHA-256 Integrity Verification
↓
Control Test
↓
Verification
↓
Compliance
↓
Audit Trail

This test validated the central objective of AEGIS-SOC-GRC: connecting security operations with governance, risk management, compliance and auditability.

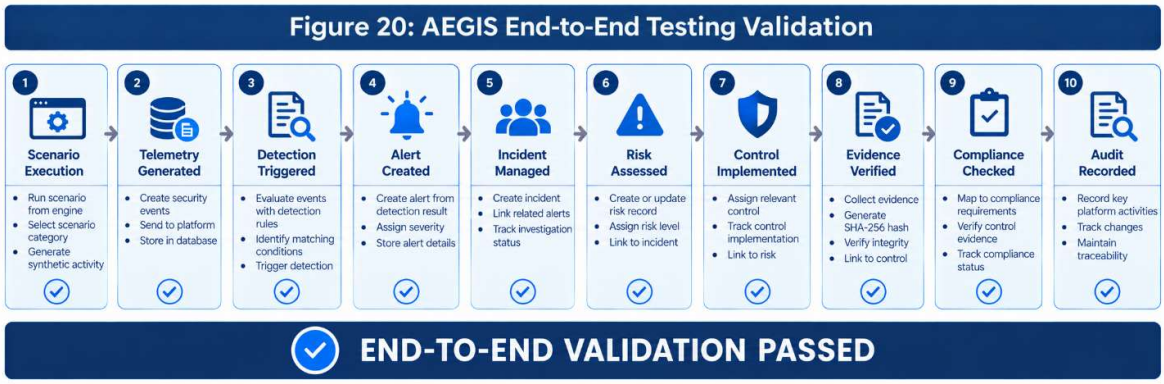
25.17 Testing Result

The testing process confirmed that the major project components could operate together as an integrated platform.

Issues discovered during testing were investigated, corrected and retested using the troubleshooting methodology documented in the previous section.

The final testing stage therefore focused not only on whether individual features worked, but whether the **complete security lifecycle remained connected and traceable**.

> **Validation Principle:**
> "The final system was considered functional when individual components worked correctly and, more importantly, when the complete SOC-to-GRC workflow could be executed and traced from security activity through verification and audit."



26. PROJECT RESULTS AND FINAL OUTCOME

26.1 Overview

The completed AEGIS-SOC-GRC platform demonstrates an integrated approach to security operations and Governance, Risk and Compliance.

The final implementation connects security simulation, telemetry, detection, alerting, incident management, risk treatment, control implementation, evidence verification, compliance and auditing within a single platform.

26.2 Functional Results

The completed project provides the following major capabilities:

- Controlled security scenario simulation.
- Synthetic security telemetry generation.
- Detection-rule processing.
- Alert creation and management.
- Incident management.
- Risk identification and assessment.
- Security control management.
- Remediation tracking.
- Evidence collection.
- SHA-256 evidence-integrity verification.
- Control testing and verification.
- Compliance traceability.
- Audit activity recording.
- Reports and analytics.
- Centralized React-based interface.

26.3 SOC Results

The SOC functionality successfully demonstrates the transformation of simulated security activity into actionable security records.

The implemented flow is:

Scenario → Telemetry → Detection → Alert → Incident

This demonstrates the operational side of the platform and provides a controlled environment for testing detection behaviour.

26.4 GRC Results

The GRC implementation extends the security workflow beyond incident investigation.

The resulting process is:

Incident → Risk → Control → Remediation → Evidence → Verification

This demonstrates how a technical security issue can be converted into a structured risk-management and remediation process.

26.5 Evidence and Verification Results

The evidence-management workflow provides supporting artifacts for control implementation and remediation.

The platform also verifies evidence integrity using SHA-256 before the evidence is relied upon for the subsequent testing and verification process.

The resulting workflow is:

Evidence → Integrity Verification → Control Test → Verification

26.6 Compliance Results

The compliance layer connects governance requirements with implemented controls and supporting evidence.

The resulting relationship is:

Compliance Requirement → Control → Evidence → Verification

This provides traceability between governance requirements and technical implementation.

26.7 Auditability Results

The Audit Trail provides historical traceability for important platform activities.

During testing, real audit events were generated for activities including:

- Alert creation.
- Alert status changes.
- Evidence integrity verification.

This demonstrates that the Audit Trail is connected to backend operations rather than functioning only as a static frontend display.

26.8 End-to-End Result

The most significant result of the project is the successful connection of the complete security lifecycle:

Scenario
↓
Security Telemetry
↓
Detection
↓
Alert
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
Integrity Verification

↓

Compliance

↓

Audit Trail

The workflow demonstrates the primary objective of AEGIS-SOC-GRC: creating a traceable connection between **security operations and governance activities**.

26.9 Practical Skills Demonstrated

The project demonstrates practical implementation experience across several areas:

- Cybersecurity monitoring.
- Detection engineering concepts.
- SOC workflow design.
- Security event processing.
- REST API development.
- FastAPI backend development.
- React and TypeScript frontend development.
- PostgreSQL database management.
- Docker-based infrastructure.
- Security simulation.
- GRC workflow design.
- Evidence management.
- Cryptographic integrity verification.
- Audit logging.
- Troubleshooting and debugging.
- End-to-end application testing.

6.10 Final Project Outcome

AEGIS-SOC-GRC successfully evolved from an initial security-monitoring concept into an integrated SOC and GRC platform.

The final system demonstrates that security operations do not need to end with the creation of an alert or incident. Security findings can be carried forward into **risk management, controls, remediation, evidence, verification, compliance and auditability**.

> **Final Outcome:**
> **AEGIS-SOC-GRC demonstrates a complete, traceable cybersecurity lifecycle from simulated security activity to detection, investigation, risk treatment, evidence-based verification and governance.**

27. PROJECT LIMITATIONS

27.1 Overview

Although AEGIS-SOC-GRC provides a broad demonstration of SOC and GRC concepts, it is important to clearly define its limitations.

The project was designed as a practical cybersecurity platform and demonstration environment rather than as a production enterprise security product.

27.2 Synthetic Security Activity

The Scenario Engine generates **controlled synthetic telemetry**.

It does not perform real attacks, malware execution, phishing campaigns or destructive security operations.

This approach makes the platform safer and more predictable for development and testing, but it also means that the telemetry does not represent the full complexity of production security data.

27.3 Limited Production Telemetry Sources

The project does not currently integrate a large range of production security sources such as:

- Enterprise EDR platforms.
- Cloud security platforms.
- Commercial SIEM feeds.
- Large-scale network sensors.
- Production identity providers.

The Scenario Engine therefore provides the primary controlled source of security activity for demonstration purposes.

27.4 Limited Scale

The project has been developed and tested in a local development environment.

It has not been benchmarked for:

- Large enterprise event volumes.
- Thousands of concurrent users.
- High-availability deployments.
- Distributed processing.
- Multi-region infrastructure.

Therefore, performance characteristics at enterprise scale have not been established.

27.5 Authentication and Authorization

The current project focuses primarily on demonstrating SOC and GRC functionality.

A production deployment would require a significantly stronger identity and access-management architecture, including appropriate authentication, authorization, role management and session security.

27.6 Evidence Integrity Limitation

SHA-256 verification confirms that an artifact matches its previously recorded hash value.

It does **not independently prove the authenticity or correctness of the original artifact**.

Therefore, evidence integrity should be understood as an artifact-consistency mechanism rather than a complete chain-of-custody solution.

27.7 Compliance Scope

The compliance functionality demonstrates the relationship between requirements, controls, evidence and verification.

It should not be interpreted as certification or as proof of organizational compliance with a real regulatory framework.

A production compliance platform would require detailed framework-specific requirements, organizational policies, assessment procedures and formal evidence management.

27.8 Local Development Architecture

The current implementation uses a local development environment involving technologies such as Docker, PostgreSQL, FastAPI, React and Windows.

A production deployment would require additional infrastructure for:

- Secure networking.
- Secrets management.
- Backup and recovery.
- Monitoring.
- High availability.
- Logging infrastructure.
- Access control.
- Deployment automation.

27.9 Limitation Summary

The limitations do not reduce the value of the project as a practical cybersecurity implementation.

Instead, they define the boundary between the current demonstration platform and a production-grade enterprise security system.

> **The project intentionally prioritizes functional demonstration, traceability and practical implementation over enterprise-scale deployment complexity.**

28. FUTURE SCOPE AND POSSIBLE IMPROVEMENTS

28.1 Overview

AEGIS-SOC-GRC provides a functional foundation for integrating SOC operations with GRC activities. The current implementation can be extended significantly to move closer to a production-oriented cybersecurity platform.

Future improvements would focus on **automation, scalability, real-world integrations, intelligence, security hardening and enterprise deployment**.

28.2 Real Security Data Integration

The Scenario Engine currently provides controlled synthetic telemetry.

A future version could integrate real security data sources such as:

- Windows Event Logs.
- Sysmon.
- Linux system logs.
- Firewall logs.
- EDR telemetry.
- Authentication providers.
- Cloud security logs.
- Network monitoring systems.

This would allow AEGIS-SOC-GRC to process a wider variety of real-world security events.

28.3 SIEM Integration

The platform could be extended to integrate with established SIEM technologies such as Splunk, Microsoft Sentinel or Elastic.

A future architecture could support:

Real Security Sources
↓
SIEM / Log Collection
↓
AEGIS Detection Layer

↓

Alerts / Incidents

↓

GRC Workflow

This would allow AEGIS to operate alongside existing security infrastructure rather than relying primarily on synthetic telemetry.

28.4 Advanced Detection Engineering

The current detection model can be expanded with more sophisticated detection capabilities.

Future improvements could include:

- Correlation rules.
- Behaviour-based detection.
- Threshold-based detection.
- Time-window analysis.
- Multi-event correlation.
- Anomaly detection.
- Detection-rule versioning.
- Detection performance metrics.
- False-positive tracking.

This would make the detection layer more representative of real SOC environments.

28.5 Automated Incident Response

The current platform focuses primarily on detection, investigation and GRC progression.

A future version could introduce controlled response automation.

For example:

Detection

↓

Alert

↓
Incident
↓
Automated Response
↓
Containment
↓
Verification

Possible response actions could include disabling a compromised account, isolating an endpoint or triggering an investigation workflow, subject to appropriate authorization and safety controls.

28.6 Threat Intelligence Integration

Threat-intelligence sources could be incorporated to provide additional context for detected activity.

Future functionality could include:

- IP reputation.
- Domain reputation.
- Malware indicators.
- Threat actor information.
- MITRE ATT&CK enrichment.
- Indicator correlation.

This could allow alerts to contain additional contextual information during investigation.

28.7 Advanced GRC Automation

The GRC workflow could be further automated.

Potential improvements include:

- Automatic risk creation from qualifying incidents.
- Automatic control recommendations.

- Remediation task assignment.
- Due-date monitoring.
- Escalation workflows.
- Automated evidence requests.
- Control-test scheduling.
- Risk treatment notifications.

This would reduce manual movement of information between SOC and GRC processes.

28.8 Expanded Compliance Frameworks

The current compliance architecture could be expanded to support multiple frameworks and more detailed control mappings.

Potential frameworks could include:

- ISO 27001.
- NIST Cybersecurity Framework.
- NIST SP 800-53.
- CIS Controls.
- SOC 2.
- PCI DSS.

A future implementation could provide framework-specific mappings between requirements, controls, evidence and verification.

28.9 Stronger Identity and Access Management

A production-oriented version should introduce stronger authentication and authorization capabilities.

Potential improvements include:

- User authentication.

- Role-based access control.
- Administrator permissions.
- Analyst permissions.
- Read-only access.
- Session management.
- Password policies.
- Multi-factor authentication.
- API authorization.

This would provide stronger protection for security and governance information.

28.10 Enterprise Evidence Management

Evidence management could be expanded into a complete evidence repository.

Future functionality could include:

- Evidence versioning.
- Evidence retention policies.
- Chain-of-custody records.
- Secure object storage.
- Access logging.
- Evidence expiration.
- Automated evidence collection.
- Digital signatures.
- Multiple integrity mechanisms.

This would make the evidence lifecycle more suitable for formal audit environments.

28.11 Advanced Reporting and Analytics

The reporting layer could be expanded with more advanced analytics.

Potential additions include:

- SOC performance metrics.
- Mean Time to Detect (MTTD).
- Mean Time to Respond (MTTR).
- Risk trends.
- Control effectiveness trends.
- Remediation aging.
- Compliance dashboards.
- Alert-volume analysis.
- Incident trends.
- Executive reporting.

This would provide stronger decision-support capabilities.

28.12 Cloud Deployment

The current local development architecture could eventually be adapted for cloud deployment.

A future architecture could include:

Cloud Infrastructure
↓
Containerized AEGIS Services
↓
Managed PostgreSQL
↓
Secure API Layer
↓
React Frontend

Cloud deployment would allow improved scalability, availability and remote access when combined with appropriate security controls.

28.13 Scalability Improvements

Future versions could introduce technologies and architectural patterns for higher workloads.

Potential improvements include:

- Asynchronous event processing.
- Message queues.
- Background workers.
- Caching.
- Database optimization.
- Horizontal scaling.
- Load balancing.
- Distributed logging.
- Container orchestration.

These improvements would help the platform handle larger security-event volumes.

28.14 AI-Assisted Security Analysis

AI could potentially be introduced as an analytical assistance layer.

Possible applications include:

- Alert summarization.
- Incident investigation assistance.
- Risk description generation.
- Detection-rule recommendations.
- Threat-context enrichment.
- Evidence classification.
- Compliance mapping assistance.

Any AI capability should remain subject to analyst review rather than replacing security decision-making entirely.

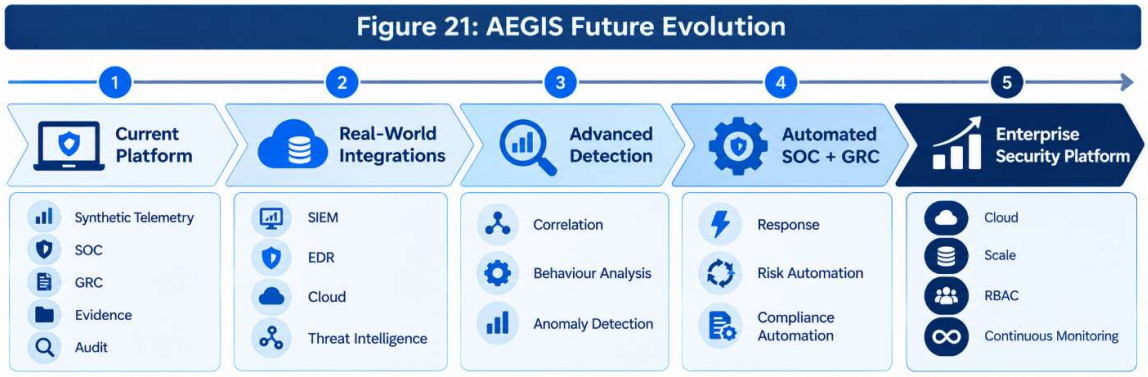
28.15 Future Vision

The long-term direction of AEGIS-SOC-GRC can be represented as:

Real Security Data
↓
Detection & Correlation
↓
SOC Investigation
↓
Automated / Guided Response
↓
Risk Assessment
↓
Control Management
↓
Remediation
↓
Evidence
↓
Verification
↓
Compliance
↓
Audit & Continuous Monitoring

The objective would be to evolve AEGIS from a practical demonstration platform into a more comprehensive security operations and governance environment.

> **Future Direction:**
> **The next evolution of AEGIS-SOC-GRC would focus on connecting real security telemetry, intelligent detection, automated response and continuous GRC management while preserving the platform's core principle of traceability.**


Figure 21: AEGIS Future Evolution
1
2
3
4
5
Current Platform
Real-World Integrations
Advanced Detection
Automated SOC + GRC
Enterprise Security Platform
Synthetic Telemetry
SOC
GRC
Evidence
Audit
SIEM
EDR
Cloud
Threat Intelligence
Correlation
Behaviour Analysis
Anomaly Detection
Response
Risk Automation
Compliance Automation
Cloud
Scale
RBAC
Continuous Monitoring

29. CONCLUSION

29.1 Project Summary

AEGIS-SOC-GRC was developed as a practical cybersecurity platform that integrates **Security Operations Center (SOC)** activities with **Governance, Risk and Compliance (GRC)** processes.

The project began with the objective of demonstrating security monitoring and detection and progressively evolved into a broader security lifecycle platform.

The final implementation connects:

Security Activity → Detection → Investigation → Risk Management → Remediation → Verification → Compliance → Audit

29.2 Achievement of Project Objectives

The project successfully demonstrates the major objectives defined during development:

- Controlled security scenario generation.
- Synthetic security telemetry.
- Detection-rule processing.
- Alert and incident management.
- Risk identification and assessment.
- Security control management.
- Remediation tracking.
- Evidence collection and review.
- SHA-256 evidence-integrity verification.
- Control testing and verification.
- Compliance traceability.
- Audit activity recording.
- Reporting and analytics.

- Integrated frontend and backend architecture.

29.3 Integration of SOC and GRC

One of the most important outcomes of the project is the integration of SOC and GRC concepts.

Traditional security monitoring can identify suspicious activity, but the security lifecycle does not necessarily end with detection or incident investigation.

AEGIS-SOC-GRC extends the workflow by connecting security findings with:

Risk → Controls → Remediation → Evidence → Verification → Compliance

This provides a more complete representation of how a security issue can be managed beyond the initial detection stage.

29.4 Traceability

Traceability is one of the core principles of the project.

The platform is designed so that security and governance information can be connected across multiple stages:

Scenario
↓
Telemetry
↓
Detection
↓
Alert
↓
Incident
↓
Risk
↓
Control
↓
Remediation
↓
Evidence
↓
Integrity
↓
Compliance
↓
Audit

This allows an analyst or reviewer to understand how an identified security condition progressed through the broader lifecycle.

29.5 Practical Development Experience

The development process provided practical experience across multiple areas of cybersecurity and software engineering.

This included:

- Designing a multi-layer security architecture.
- Developing REST APIs.
- Working with PostgreSQL.
- Using Docker for infrastructure.
- Building a React and TypeScript interface.
- Implementing security-event simulation.
- Developing detection logic.
- Designing SOC and GRC workflows.
- Implementing evidence integrity verification.
- Creating audit logging.
- Troubleshooting application and environment issues.
- Performing end-to-end validation.

29.6 Final Perspective

AEGIS-SOC-GRC is not intended to claim the capabilities of a production enterprise SIEM, SOAR or GRC platform.

Instead, it demonstrates how multiple cybersecurity concepts can be implemented and connected within a single practical software project.

The project shows the progression from a simple security-monitoring idea to a complete, traceable SOC-to-GRC lifecycle.

> **Final Statement:**
> **AEGIS-SOC-GRC demonstrates how security operations, risk management, remediation, evidence, compliance and auditing can be connected into one practical and traceable cybersecurity lifecycle.**

30. FINAL PROJECT SUMMARY

Project Identity

Project: AEGIS-SOC-GRC

Focus: Security Operations + Governance, Risk & Compliance

Frontend: React + TypeScript

Backend: FastAPI + Python

Database: PostgreSQL

Infrastructure: Docker

Integrity: SHA-256

Simulation: AEGIS Scenario Engine

Core Lifecycle

Scenario → Telemetry → Detection → Alert → Incident → Risk → Control → Remediation → Evidence → Integrity → Compliance → Audit

Final Development Outcome

A working, integrated cybersecurity platform demonstrating practical SOC and GRC implementation, end-to-end traceability, evidence-based verification and auditability.

31. REFERENCES AND TECHNICAL RESOURCES

31.1 Programming and Development Technologies

1. Python Documentation — Official documentation for the Python programming language and standard library. **3.14.8 Documentation**

2. FastAPI Documentation — Official documentation for FastAPI and REST API development.

FastAPI - FastAPI

3. React Documentation — Official documentation for React components, APIs and frontend development. **React**

4. TypeScript Documentation — Official documentation covering TypeScript language features, types and tooling. **TypeScript: JavaScript With Syntax For Types.**

5. PostgreSQL Documentation — Official documentation for PostgreSQL database administration, SQL and database features. **PostgreSQL: Documentation**

6. Docker Documentation — Official documentation for Docker containers, images and development infrastructure. **Docker Docs**

31.2 Cybersecurity References

1. MITRE ATT&CK — Knowledge base used as a reference for adversary tactics and techniques.

MITRE ATT&CK®

2. MITRE ATT&CK — T1110.001 Password Guessing — Reference used for the authentication brute-force scenario in AEGIS-SOC-GRC. **Brute Force: Password Guessing, Sub-technique T1110.001 - Enterprise | MITRE ATT&CK®**

3. NIST Cybersecurity Framework (CSF) 2.0 — Reference for cybersecurity risk management and governance concepts used in the project. **The NIST Cybersecurity Framework (CSF) 2.0 | NIST**

4. NIST Cybersecurity Framework Resources — Additional NIST resources for cybersecurity risk management and framework implementation. **Frameworks | NIST**

31.3 Cryptographic Reference

1. NIST Secure Hash Standard (FIPS 180-4) — Reference for SHA-256 and the SHA-2 family of cryptographic hash functions. **FIPS 180-4, Secure Hash Standard (SHS) | CSRC**

32. APPENDIX A — PROJECT DIRECTORY STRUCTURE

This appendix documents the major project files and their roles.

32.1 Overall Structure

The project is organized into separate areas for the backend, frontend, simulator and supporting configuration.

A simplified representation is:

```
AEGIS-SOC-GRC/
│
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   ├── database.py
│   │   └── main.py
│   │
│   ├── simulator/
│   ├── init_db.py
│   └── requirements.txt
│
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── services/
│   │   ├──App.tsx
│   │   └──App.css
│   │
│   └── package.json
│
└── README.md
```

32.2 Backend

The backend contains the API, database models, schemas and service logic.

Its primary responsibility is to:

- Process API requests.

- Apply application logic.
- Interact with PostgreSQL.
- Execute detection and GRC operations.
- Manage evidence.
- Record audit activity.

32.3 API Layer

The API directory contains route modules responsible for exposing functionality through REST endpoints.

The API layer provides access to areas such as:

- Security events.
- Alerts.
- Incidents.
- Risks.
- Controls.
- Remediation.
- Evidence.
- Compliance.
- Audit logs.

32.4 Models

The models represent the database entities used by the platform.

Examples include:

- Security events.
- Alerts.

- Incidents.
- Assets.
- Risks.
- Controls.
- Remediation.
- Evidence.
- Audit logs.

These models define the persistence structure used by PostgreSQL.

32.5 Schemas

Schemas define the structures used for API requests and responses.

They provide a controlled interface between the frontend and backend and help ensure that returned data follows the expected structure.

32.6 Services

The service layer contains reusable business logic.

An important example is the evidence service, which handles evidence-related operations including artifact integrity verification.

The audit service provides centralized functionality for recording important platform activities.

32.7 Simulator

The simulator contains the AEGIS Scenario Engine and supporting security-event simulation functionality.

Its purpose is to generate controlled synthetic security activity for testing the SOC workflow.

32.8 Frontend

The frontend contains the React and TypeScript application.

Major frontend responsibilities include:

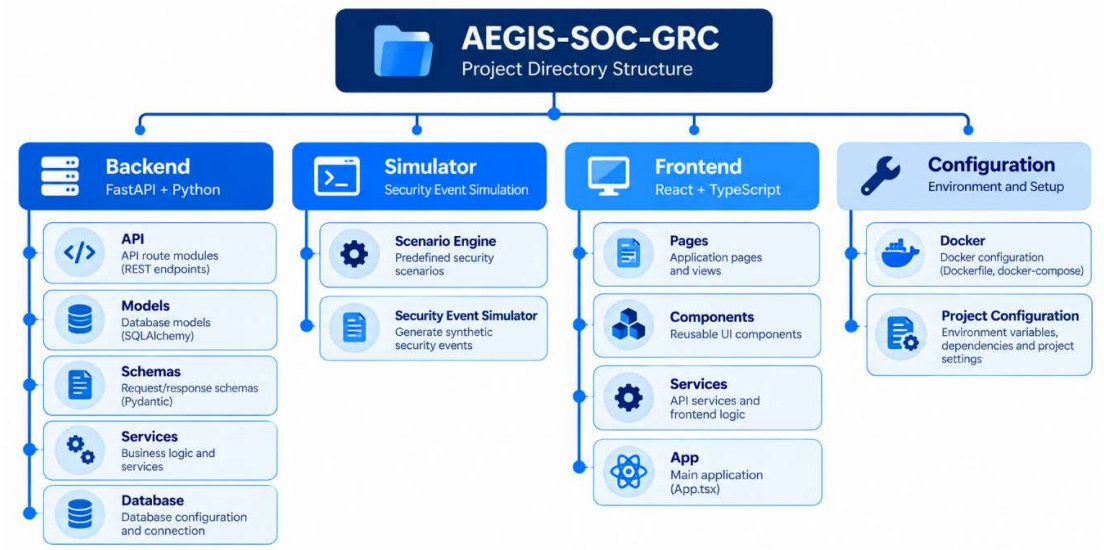
- Navigation.
- Data presentation.
- User interaction.
- API communication.
- Search and filtering.
- Detail views.
- Status visualization.
- Evidence management.
- Reports.
- Audit Trail.

32.9 Project Organization Principle

The project directory structure separates:

Presentation → API → Business Logic → Data → Simulation

This separation makes the project easier to understand, maintain and extend.


AEGIS-SOC-GRC
Project Directory Structure
Backend
FastAPI + Python
API
API route modules (REST endpoints)
Models
Database models (SQLAlchemy)
Schemas
Request/response schemas (Pydantic)
Services
Business logic and services
Database
Database configuration and connection
Simulator
Security Event Simulation
Scenario Engine
Predefined security scenarios
Security Event Simulator
Generate synthetic security events
Frontend
React + TypeScript
Pages
Application pages and views
Components
Reusable UI components
Services
API services and frontend logic
App
Main application (App.tsx)
Configuration
Environment and Setup
Docker
Docker configuration (Dockerfile, docker-compose)
Project Configuration
Environment variables, dependencies and project settings

33. APPENDIX B — KEY API ENDPOINTS

This appendix documents the major REST API endpoints implemented in the AEGIS-SOC-GRC backend. The purpose of this section is to provide a technical reference for the communication between the React frontend and the FastAPI backend.

The API layer acts as the communication boundary between the user interface, security/GRC business logic, and PostgreSQL database.

33.1 API Architecture

AEGIS-SOC-GRC follows a REST-based API architecture.

The general request flow is:

React Frontend → FastAPI API Endpoint → Service / Business Logic → PostgreSQL → JSON Response

The frontend communicates with the backend using HTTP requests such as:

- GET — Retrieve information
- POST — Create a new record or perform an operation
- PATCH — Update an existing record
- PUT — Replace/update resources where applicable
- DELETE — Remove a resource where applicable

The API responses are returned as JSON and consumed by the React/TypeScript frontend.

33.2 Security Event Endpoints

Security events represent the telemetry generated by the AEGIS Scenario Engine and security event simulator.

Method	Endpoint	Purpose
GET	`/events`	Retrieve security events
POST	`/events/`	Create a security event
GET	`/events/{event_id}`	Retrieve a specific event

Example

A simulated authentication failure can be submitted to the backend through:

`POST /events/`

The event can contain information such as:

- Event type
- Username
- Source IP
- Timestamp
- Asset
- Authentication result
- Additional event metadata

These events become the input for the detection workflow.

33.3 Detection Endpoints

The detection layer evaluates security telemetry against configured detection rules.

Method	Endpoint	Purpose
POST	`/detection/run`	Execute detection logic
GET	Detection-related endpoints	Retrieve detection results where applicable

The `/detection/run` endpoint is particularly important because it connects the generated security telemetry with the detection engine.

The simplified workflow is:

Security Events → Detection Engine → Rule Evaluation → Detection Result → Alert

During testing, this endpoint was used to validate scenarios such as repeated failed login attempts.

33.4 Alert Endpoints

Alerts represent security conditions identified by the detection layer.

Method	Endpoint	Purpose
GET	`/alerts`	Retrieve alerts
GET	`/alerts/{alert_id}`	Retrieve a specific alert
PATCH	`/alerts/{alert_id}/status`	Update alert status

Alert records contain information such as:

- Alert ID
- Rule ID
- Title
- Severity
- Status
- Username
- Source information
- Detection information
- Timestamps

Alert Status Tracking

When an alert status changes, the backend also records an audit event.

For example:

Open → Resolved

creates an audit activity describing the status transition.

This provides traceability between operational SOC activity and the Audit Trail module.

33.5 Incident Endpoints

Incidents provide a higher-level investigation record associated with security alerts.

Method	Endpoint	Purpose
GET	`/incidents`	Retrieve incidents
GET	`/incidents/{incident_id}`	Retrieve a specific incident
POST	Incident creation endpoint	Create an incident
PATCH	Incident update endpoint	Update incident information

An incident can contain:

- Incident number
- Title
- Description
- Severity
- Status
- Related alert
- Investigation information
- Risk relationship

The incident layer provides the transition from individual security alerts to structured investigation and risk assessment.

33.6 Risk Management Endpoints

Risk records connect security incidents with GRC activities.

Method	Endpoint	Purpose
GET	`/risks`	Retrieve risks
GET	`/risks/{risk_id}`	Retrieve a specific risk

POST	Risk creation endpoint	Create a risk
PATCH	Risk update endpoint	Update risk information

Risk records can contain:

- Risk ID
- Risk title
- Description
- Likelihood
- Impact
- Risk score
- Risk level
- Status
- Related incident
- Assigned control

The risk layer enables the platform to move from **security detection** into **risk treatment**.

33.7 Security Control Endpoints

Controls represent safeguards or security requirements used to address identified risks.

Method	Endpoint	Purpose
GET	`/controls`	Retrieve controls
GET	`/controls/{control_id}`	Retrieve a specific control
POST	Control creation endpoint	Create a control
PATCH	Control update endpoint	Update control information

A control can be associated with:

- Risk
- Remediation

- Evidence
- Compliance requirement
- Verification result

An example used in the project is:

CTRL-AUTH-001

This control is associated with authentication monitoring and account lockout requirements.

33.8 Remediation Endpoints

Remediation records describe the actions taken to address an identified security or control weakness.

Method	Endpoint	Purpose
GET	`/remediations`	Retrieve remediation controls
GET	`/remediations/{remediation_id}`	Retrieve a specific remediation
POST	Remediation creation endpoint	Create remediation
PATCH	Remediation update endpoint	Update remediation status

Typical remediation lifecycle:

Planned → In Progress → Completed → Verified

Remediation provides the operational bridge between a security risk and the evidence demonstrating that the associated requirement was addressed.

33.9 Evidence Endpoints

Evidence management is one of the important GRC capabilities of the platform.

Method	Endpoint	Purpose
GET	`/evidence`	Retrieve evidence records
GET	`/evidence/{evidence_id}`	Retrieve specific evidence
POST	`/evidence`	Create evidence metadata
POST	`/evidence/{evidence_id}/artifact`	Upload evidence artifact

PATCH	`/evidence/{evidence_id}/integrity`	Verify evidence integrity

The evidence workflow is:

Evidence Metadata → Artifact Upload → SHA-256 Verification → Review → Control Test → Verification

The artifact endpoint accepts the evidence file and associates it with the corresponding evidence record.

33.10 Evidence Integrity Endpoint

The following endpoint was specifically implemented for evidence integrity verification:

`PATCH /evidence/{evidence_id}/integrity`

The backend calculates the SHA-256 hash of the stored artifact and compares it with the previously stored hash.

The result can be:

- `verified`
- `failed`

A successful verification means that the artifact content has remained consistent with the stored integrity value.

The verification activity is also recorded in the Audit Trail.

Example audit action:

EVIDENCE_INTEGRITY_VERIFIED

This creates an additional traceability relationship:

Evidence → Integrity Verification → Audit Event

33.11 Compliance Endpoints

Compliance functionality connects controls and evidence with compliance requirements.

Method	Endpoint	Purpose
GET	/compliance	Retrieve compliance records
GET	/compliance/{id}	Retrieve a compliance record
POST	Compliance creation endpoint	Create a compliance record
PATCH	Compliance update endpoint	Update compliance status

The compliance layer uses information from:

- Controls
- Evidence
- Verification
- Risk treatment
- Audit activity

This allows the project to demonstrate how technical security activity can support governance and compliance processes.

33.12 Audit Trail Endpoints

The Audit Trail API provides historical visibility into important system activities.

Retrieve Audit Logs

```
GET /audit-logs
```

Returns the recorded audit events.

Retrieve a Specific Audit Log

```
GET /audit-logs/{audit_log_id}
```

Returns one audit record based on its unique identifier.

Audit records contain:

- Audit ID
- Timestamp
- Actor

- Action
- Entity type
- Entity ID
- Description
- Previous value
- New value
- Source

Example:

Field	Example
Actor	SOC Analyst
Action	EVIDENCE_INTEGRITY_VERIFIED
Entity type	Evidence
Old Value	unverified
New Value	verified
Source	evidence

The Audit Trail therefore provides historical context rather than only displaying the current state of an object.

33.13 Reports Endpoints

The reporting layer retrieves aggregated information from the platform.

Reports can present information relating to:

- Security events
- Alerts
- Incidents
- Risks
- Controls

- Remediation
- Evidence
- Compliance
- Audit activity

The frontend uses the API data to present report summaries and detailed report views.

33.14 API-to-Frontend Communication

The React frontend uses a common API communication pattern.

A simplified request structure is:

Frontend Page → API Service → FastAPI Endpoint → Database → JSON Response → Frontend State → UI

For example, the Audit Trail page requests:

```
GET /audit-logs
```

The backend retrieves the audit records from PostgreSQL and returns them as JSON.

The frontend then converts the response into the audit activity table.

This approach keeps API communication separate from the visual presentation layer.

33.15 API Design Principles

The AEGIS API was designed around the following principles:

1. **Resource-oriented endpoints**

   Major platform entities have dedicated API routes.

2. **Separation of concerns**

   API routes, services, models and schemas are separated.

3. **Structured JSON responses**

   The frontend receives predictable data structures.

4. **Database-backed operations**

   Important platform information is persisted in PostgreSQL rather than being maintained only in frontend state.

5. **Traceability**

   Important changes can generate Audit Trail records.

6. **Frontend independence**

   The backend API can be tested independently of the React interface.

7. **Extensibility**

   Additional detection rules, GRC resources and integrations can be added without redesigning the entire frontend.

33.16 End-to-End API Relationship

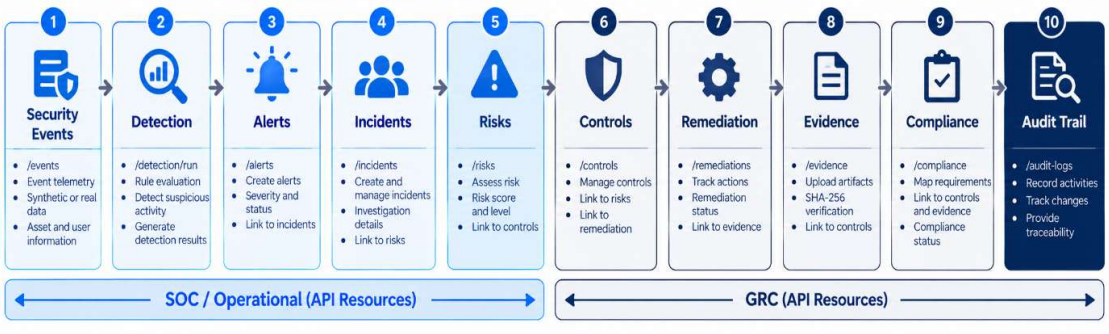
The major API resources support the complete AEGIS workflow:

Events → Detection → Alerts → Incidents → Risks → Controls → Remediation → Evidence → Integrity → Compliance → Audit Logs

This API structure reflects the overall design philosophy of AEGIS-SOC-GRC: security activity should not remain isolated inside a SOC workflow. It should be traceable through the subsequent risk, control, remediation, evidence, compliance and audit stages.

> **The API layer acts as the connective tissue between the SOC operations layer and the GRC management layer, allowing security activity to move through the complete AEGIS lifecycle.**

Figure 23 — Major AEGIS-SOC-GRC API resource flow connecting security operations with GRC and audit activities.



34. APPENDIX C — TESTING EVIDENCE / SCREENSHOTS

This appendix documents the visual evidence collected during the development and validation of AEGIS-SOC-GRC. Screenshots are included to demonstrate that the implemented modules were not only designed but also executed and tested through the working application.

The screenshots should be inserted in the same order as the project workflow so that the documentation tells a continuous technical story from security simulation to final auditability.

34.1 Purpose of Testing Evidence

The purpose of the screenshots is to provide visual confirmation of important implementation and testing results.

The screenshots demonstrate:

- Backend API operation
- Security scenario execution
- Security telemetry generation
- Detection rule execution
- Alert creation
- Incident management
- Risk assessment
- Security control mapping
- Remediation
- Evidence collection
- SHA-256 integrity verification
- Compliance tracking
- Audit Trail generation
- Reports and analytics
- Frontend functionality

Screenshots are therefore treated as supporting evidence rather than as a replacement for functional testing.

> **The screenshots document observable results from the working AEGIS-SOC-GRC implementation and support the technical explanations provided throughout this documentation.**

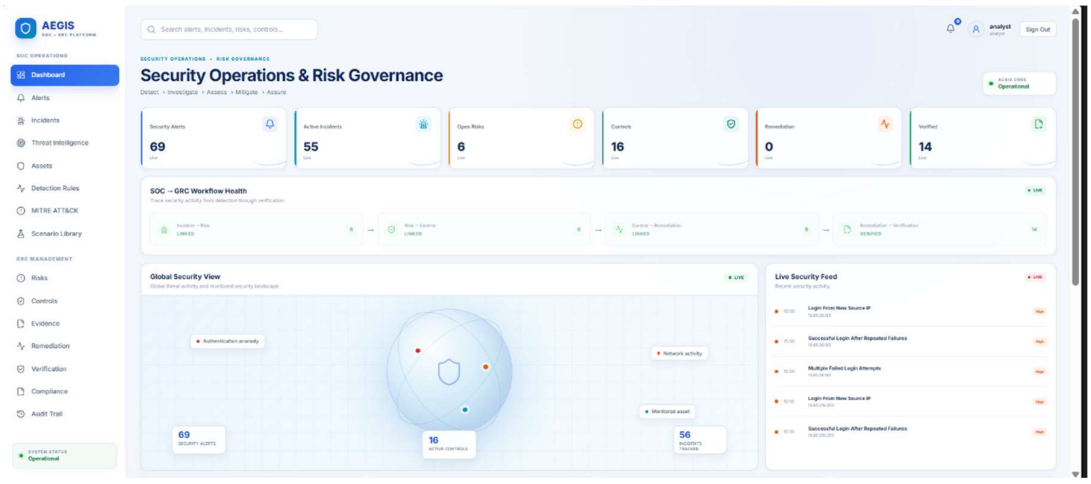
34.2 Recommended Screenshot Sequence

The screenshots should follow the actual AEGIS lifecycle:

Scenario → Telemetry → Detection → Alert → Incident → Risk → Control → Remediation → Evidence → Integrity → Compliance → Audit

This sequence makes it easier for a reviewer to understand how the individual modules are connected.

34.3 Screenshot 1 — AEGIS Dashboard



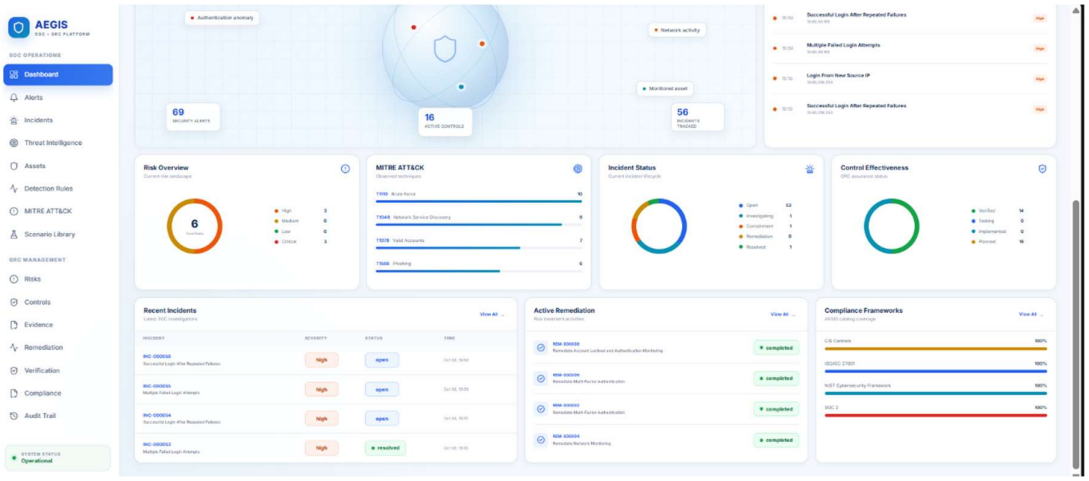


Figure 24 — AEGIS-SOC-GRC dashboard providing an overview of security operations and GRC activity.

The screenshot includes:

- AEGIS-SOC-GRC branding
- Navigation sidebar
- Dashboard statistics
- Security activity
- Alert information
- Risk/GRC information
- Overall application layout

What it demonstrates

This screenshot establishes the overall frontend and provides a visual introduction to the completed platform.

34.4 Screenshot 2 — Scenario Engine Execution

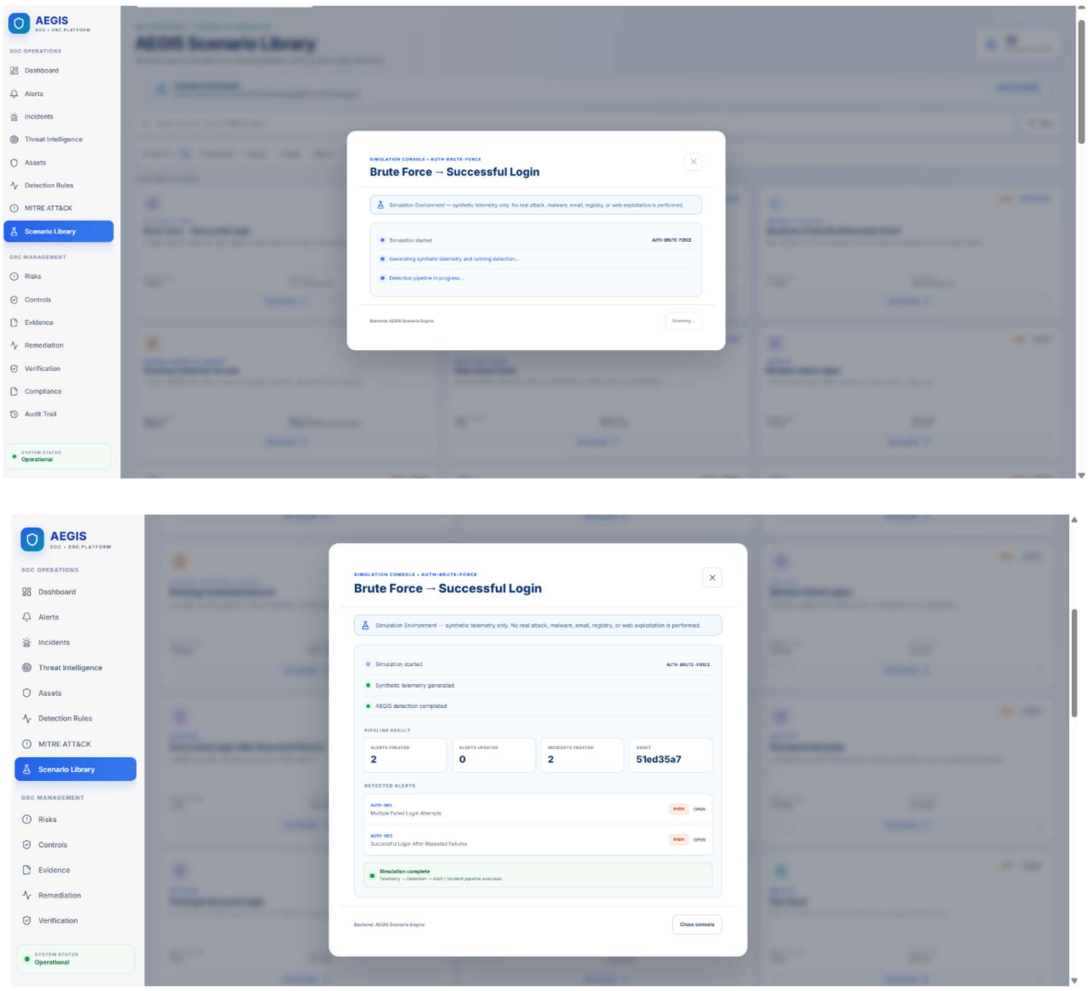


Figure 25 — AEGIS Scenario Engine executing the AUTH-BRUTE-FORCE synthetic security scenario.

A suitable example is:

AUTH-BRUTE-FORCE — Brute Force → Successful Login

The screenshot show relevant information such as:

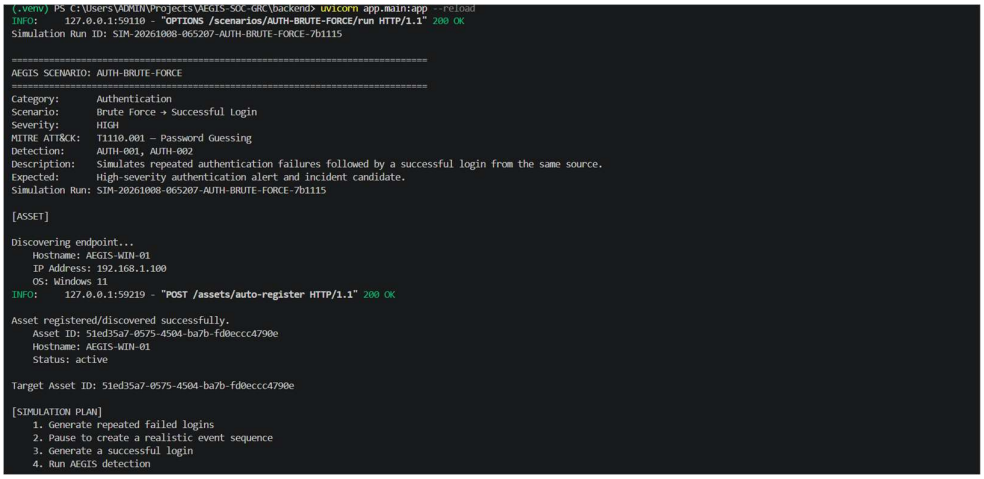
- Scenario ID
- Scenario name
- Category
- Severity
- MITRE ATT&CK technique

- Simulation Run ID
- Asset information
- Generated telemetry

What it demonstrates

This proves that the Scenario Engine can execute controlled synthetic security scenarios rather than relying only on manually entered database records.

34.5 Screenshot 3 — Security Telemetry



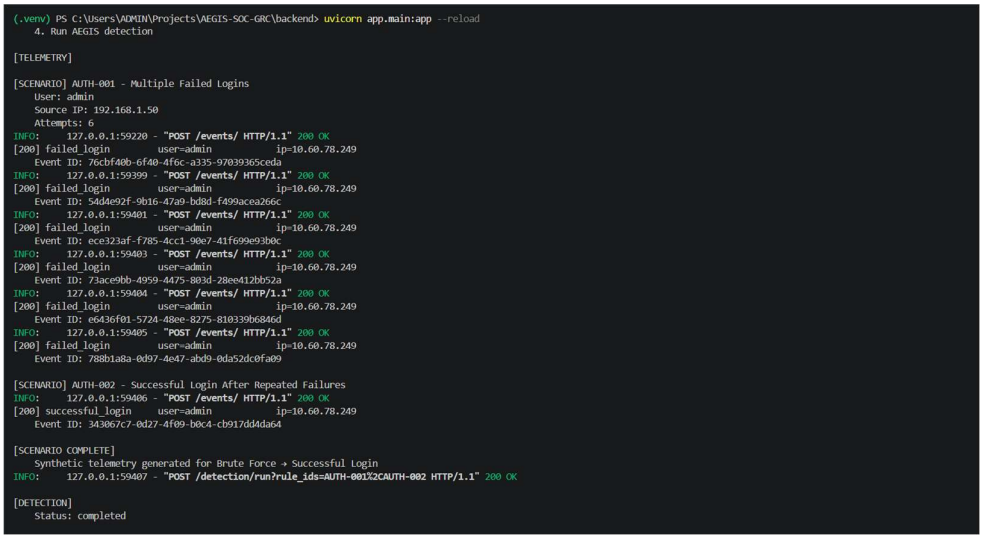




Figure 26 — Synthetic security telemetry generated by the AEGIS security simulation workflow.

For example:

- Failed login events
- Successful login event
- Username
- Source IP
- Event type
- Timestamp
- Asset information

What it demonstrates

This confirms the generation of synthetic telemetry that becomes the input for the detection workflow.

34.6 Screenshot 4 — Detection Result



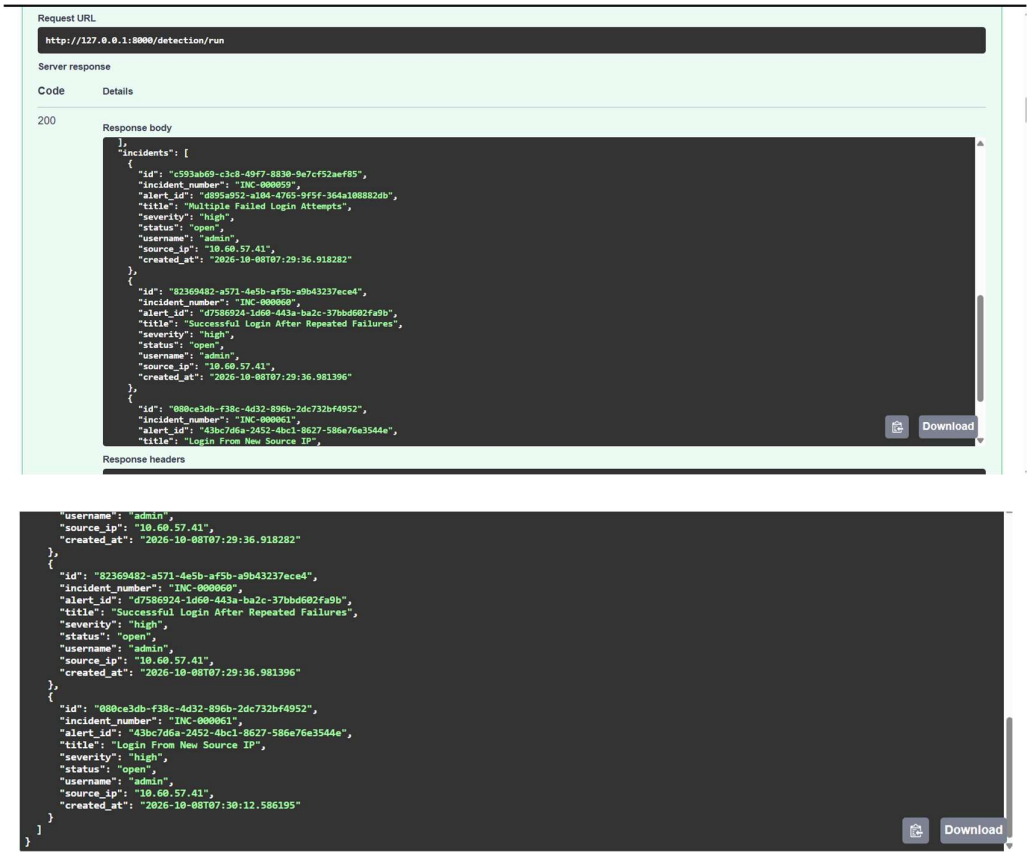


Figure 27 — Detection engine processing security telemetry and generating detection results.

`POST /detection/run`

The screenshot should show relevant information such as:

- Detection status
- Alerts created
- Detection rule IDs
- Matching conditions
- Alert severity
- Detected username or source
- Incident creation where applicable

What it demonstrates

This provides evidence that the generated telemetry was processed by the detection layer.

34.7 Screenshot 5 — Alerts Page

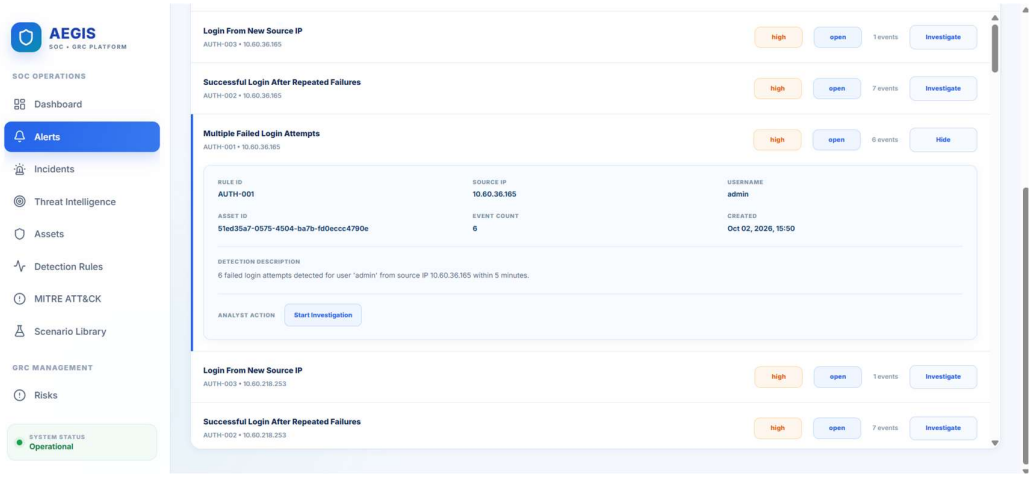


Figure 28 — AEGIS Alerts page displaying detection-generated security alerts.

The screenshot preferably shows:

- Alert ID
- Alert title
- Rule ID
- Severity
- Status
- Detection information
- Relevant timestamps

For the authentication scenario, an example is:

AUTH-001 — Multiple Failed Login Attempts

What it demonstrates

This confirms the transition from detection results to actionable SOC alerts.

34.8 Screenshot 6 — Incident Management

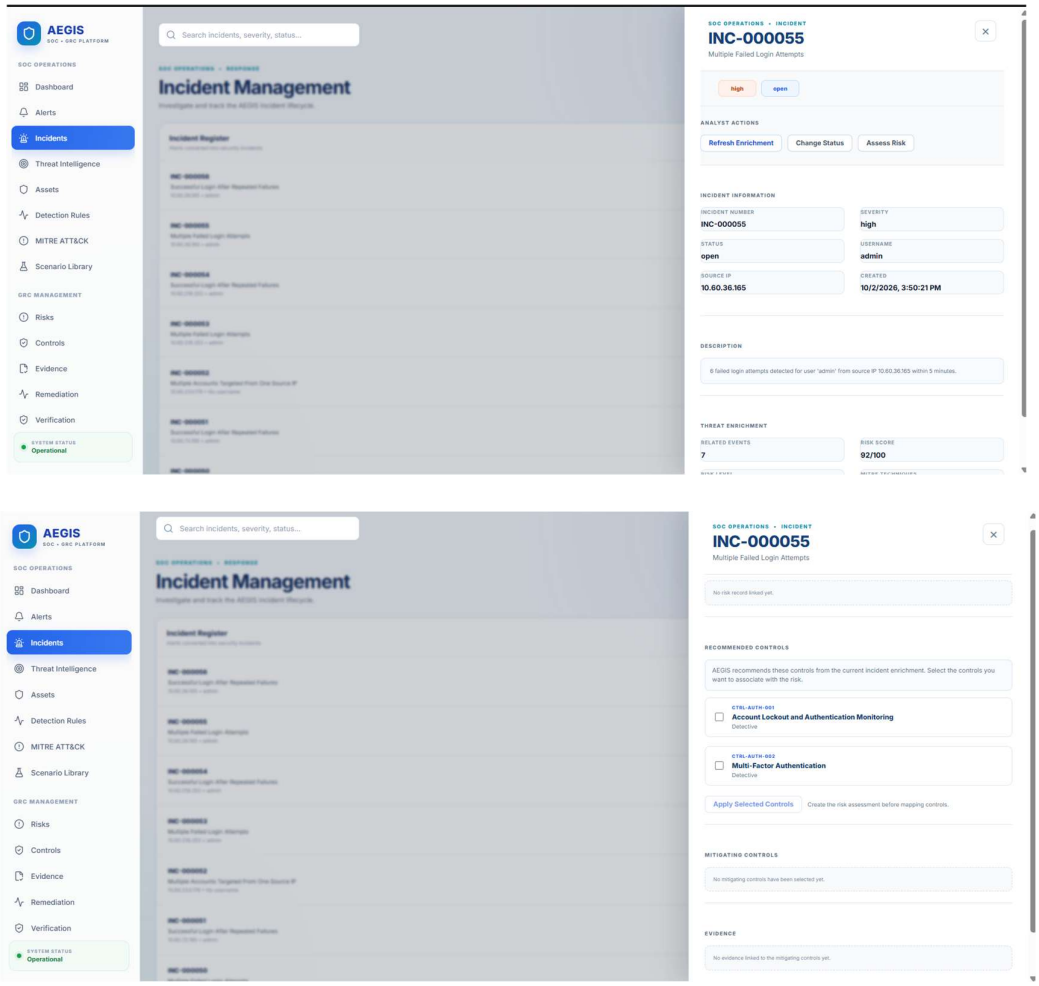


Figure 29 — AEGIS Incident Management interface showing an incident generated from security activity.

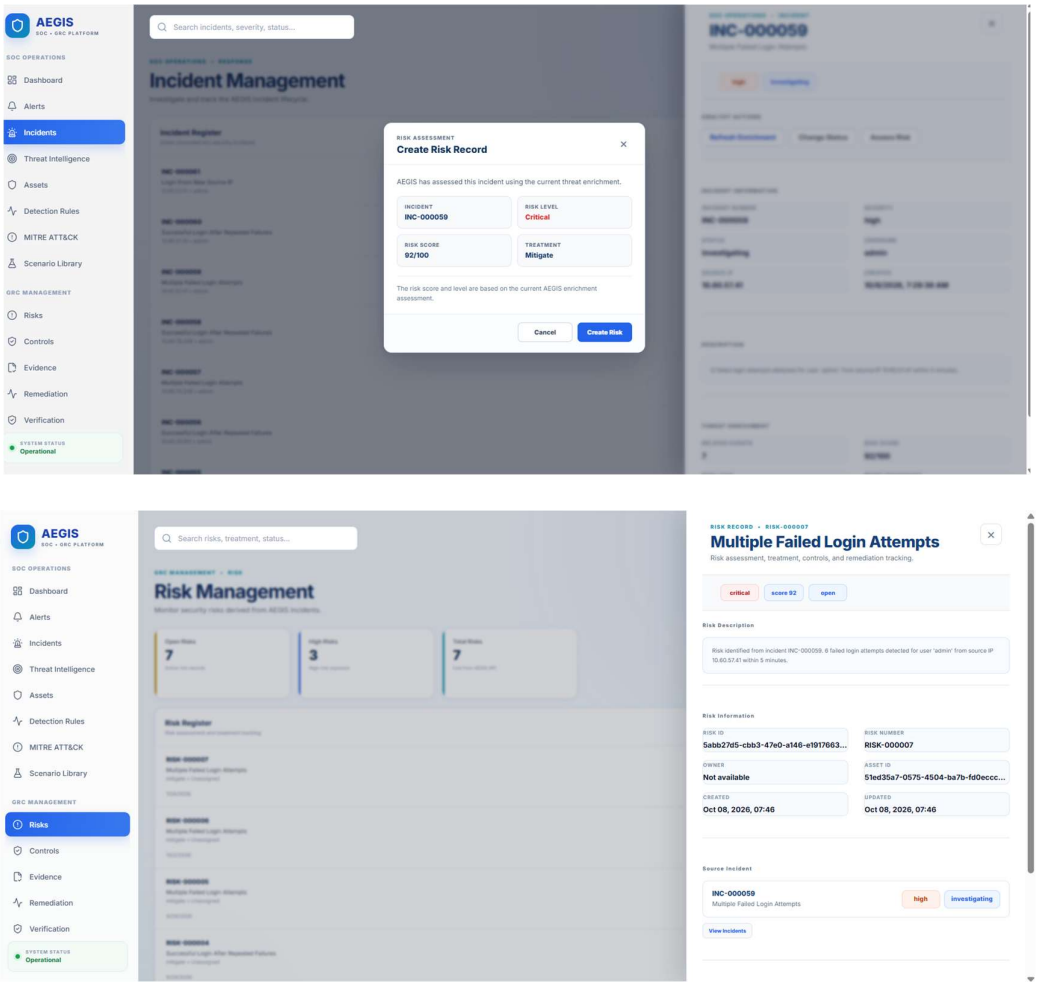
Include information such as:

- Incident number
- Incident title
- Severity
- Status
- Related alert
- Investigation information
- Related risk

What it demonstrates

This proves that detected alerts can be elevated into structured security investigations.

34.9 Screenshot 7 — Risk Management



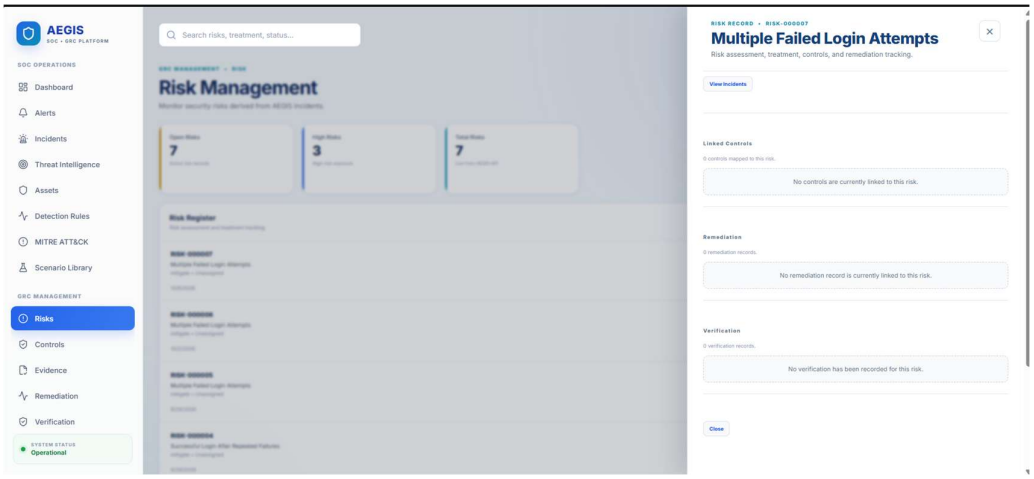


Figure 30 — AEGIS Risk Management interface showing risk assessment and treatment information.

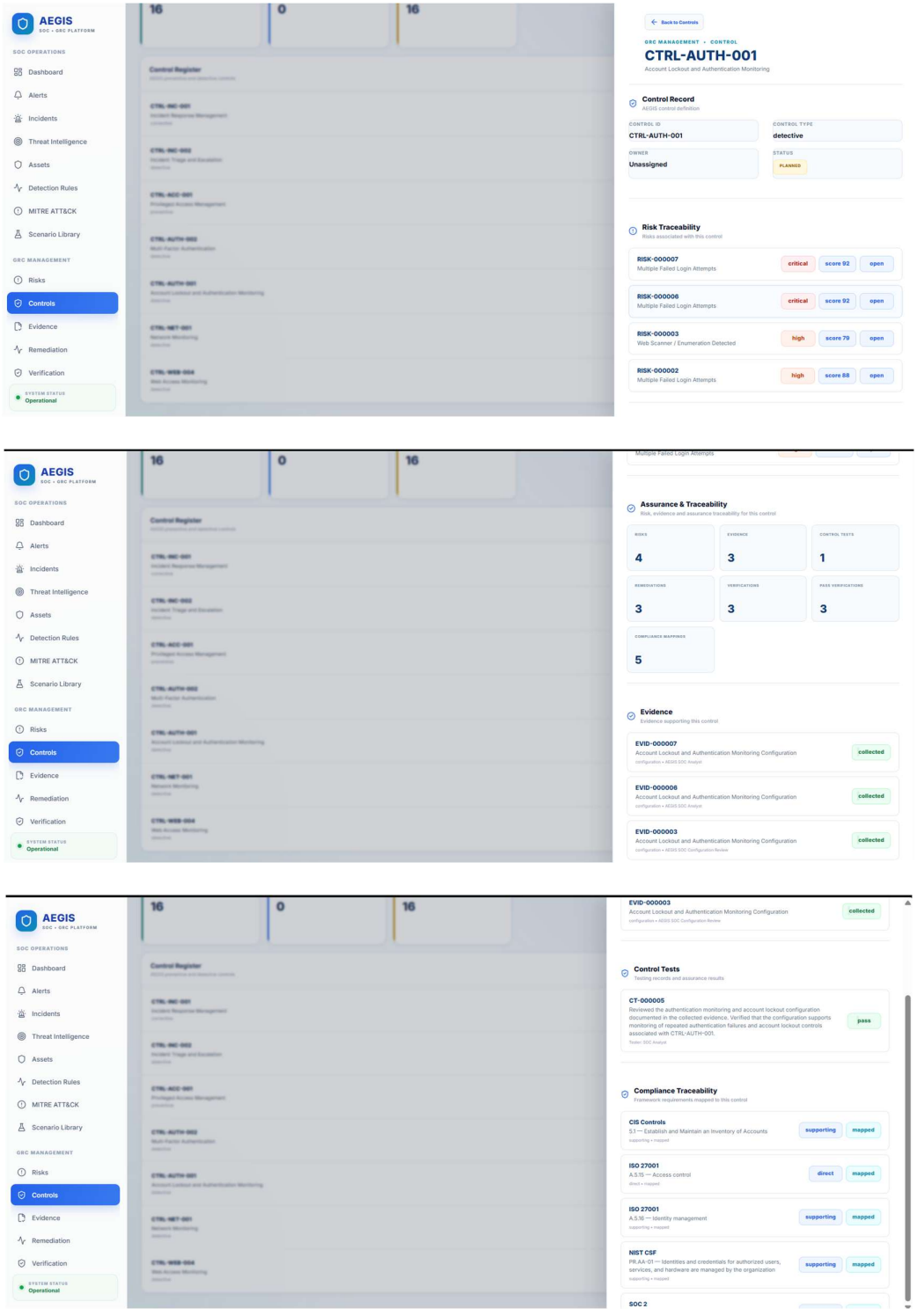
The screenshot shows:

- Risk ID
- Risk title
- Likelihood
- Impact
- Risk score
- Risk level
- Status
- Related incident
- Assigned control

What it demonstrates

This confirms the transition from SOC investigation to formal risk assessment.

34.10 Screenshot 8 — Security Control



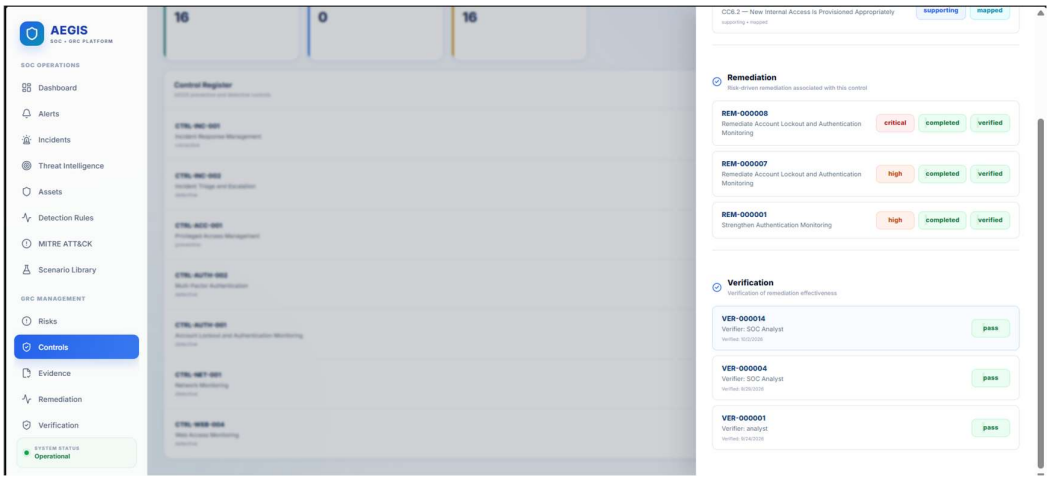


Figure 31 — Security control mapped to the identified authentication-related risk.

For example:

CTRL-AUTH-001

Show:

- Control ID
- Control name
- Description
- Related risk
- Implementation status
- Remediation relationship
- Evidence relationship
- Compliance relationship

What it demonstrates

This demonstrates how an identified risk is connected to a specific security control.

34.11 Screenshot 9 — Remediation

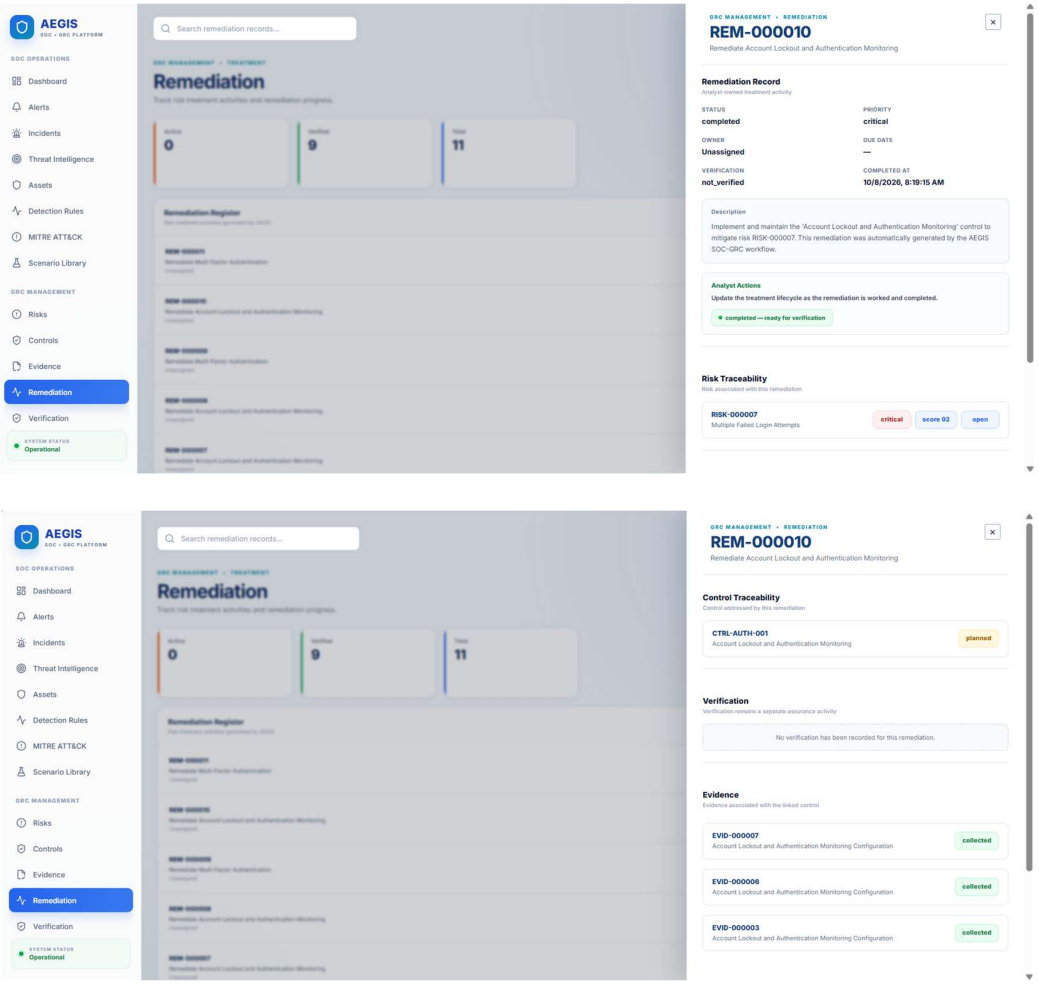


Figure 32 — AEGIS remediation workflow showing treatment of an identified security control requirement.

Show, where available:

- Remediation ID
- Description
- Status
- Assigned control
- Remediation action
- Completion information
- Verification status

What it demonstrates

This provides visual evidence that AEGIS does not stop at identifying a risk but also tracks its treatment.

34.12 Screenshot 10 — Evidence Collection

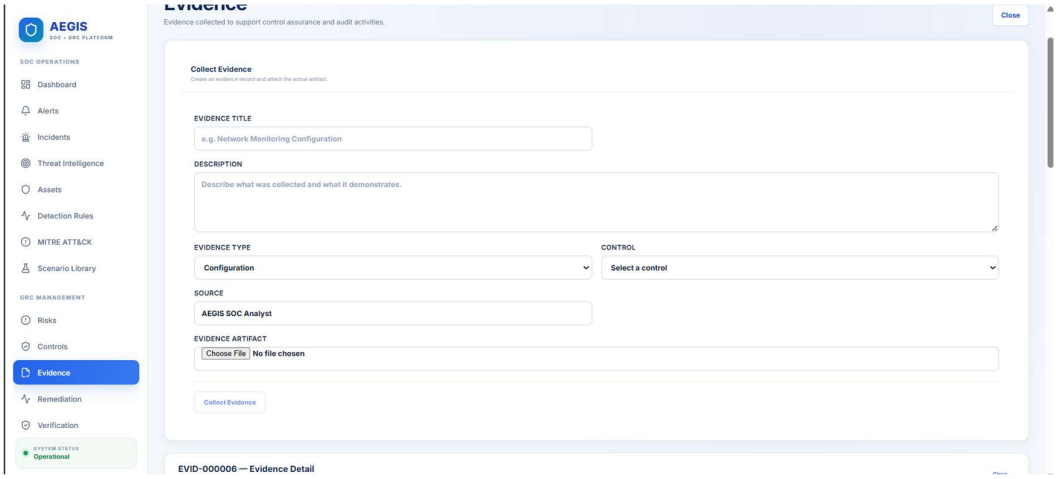


Figure 33 — Evidence records documenting authentication monitoring and account lockout configuration.

The project example can use:

Account Lockout and Authentication Monitoring Configuration

Include:

- Evidence number
- Evidence title
- Description
- Evidence type
- Associated control
- Source
- Artifact
- SHA-256 information

What it demonstrates

This confirms that remediation activities can be supported by documented evidence.

34.13 Screenshot 11 — Evidence Artifact Upload

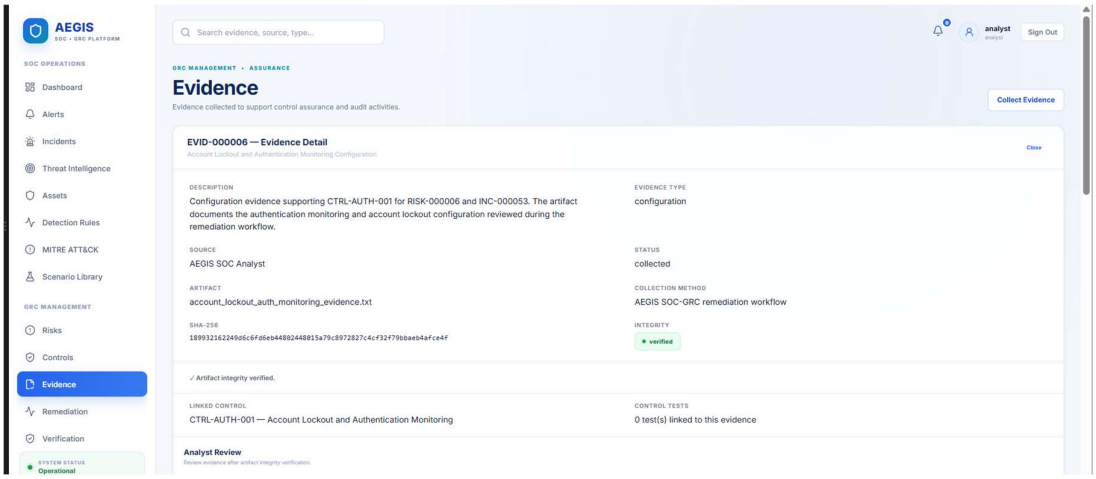


Figure 34 — Evidence artifact associated with the AEGIS control verification workflow.

The example artifact is:

`account_lockout_auth_monitoring_evidence.txt`

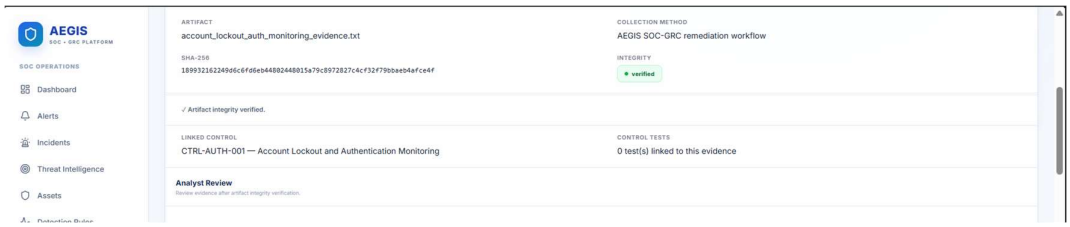
Where visible, include:

- Artifact filename
- Evidence number
- Artifact path/reference
- SHA-256 hash
- Upload status

What it demonstrates

This shows that evidence is associated with an actual artifact rather than existing only as descriptive metadata.

34.14 Screenshot 12 — SHA-256 Integrity Verification



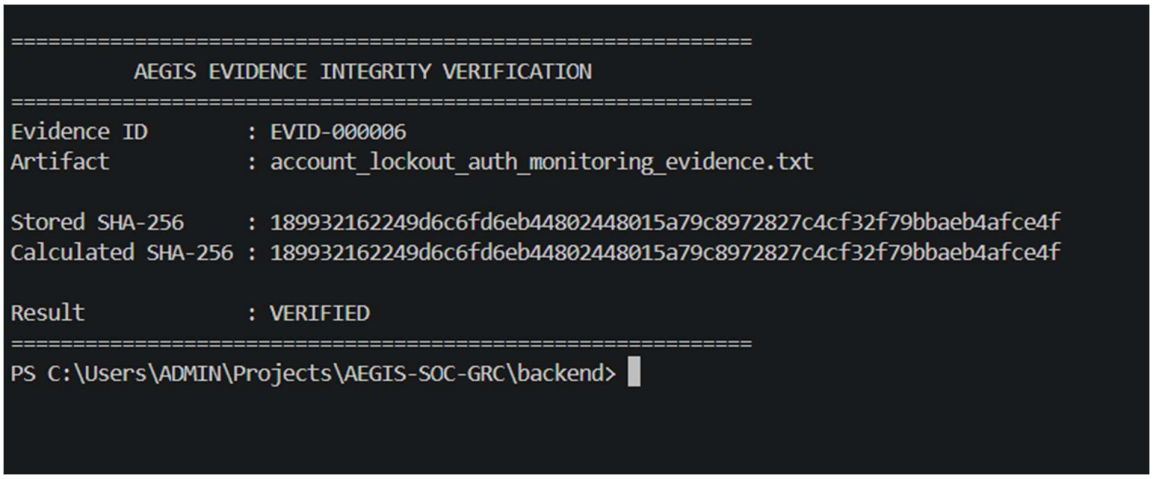


Figure 35 — SHA-256 integrity verification showing that the stored evidence hash matches the independently calculated artifact hash, resulting in a verified integrity status.

The screenshot shows:

- Evidence ID: EVID-000006
- Artifact: account_lockout_auth_monitoring_evidence.txt
- Stored SHA-256
- Calculated SHA-256
- Result: VERIFIED

Description

The screenshot shows the integrity verification performed on the collected evidence artifact. The SHA-256 hash stored with the evidence is compared with a newly calculated SHA-256 hash generated directly from the artifact. In this validation, both hash values are identical, resulting in a **VERIFIED** status.

What it demonstrates

This demonstrates that AEGIS includes an **evidence-integrity verification mechanism** rather than simply storing evidence as metadata. It confirms that the collected artifact currently produces the same cryptographic hash as the value recorded during evidence collection, providing assurance that the artifact has not changed since its recorded hash was established.

Technical significance

SHA-256 provides a deterministic fingerprint for the artifact. Even a small modification to the file would normally produce a different hash, causing the comparison to fail. Therefore, the matching values provide evidence of **integrity consistency** for the collected artifact.

34.15 Screenshot 13 — Control Test

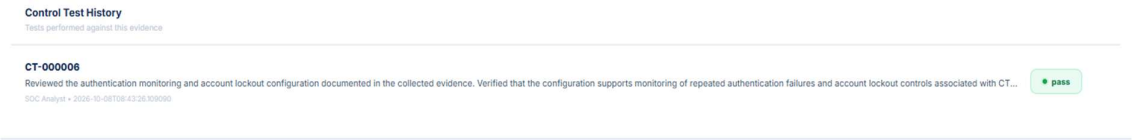


Figure 36 — Control testing performed against the collected authentication configuration evidence.

The screenshot shows:

- Test procedure
- Test result
- Findings
- Tester
- Notes

The authentication control test should show:

Result: PASS

What it demonstrates

This proves that collected evidence is actually evaluated against the associated control.

34.16 Screenshot 14 — Verification

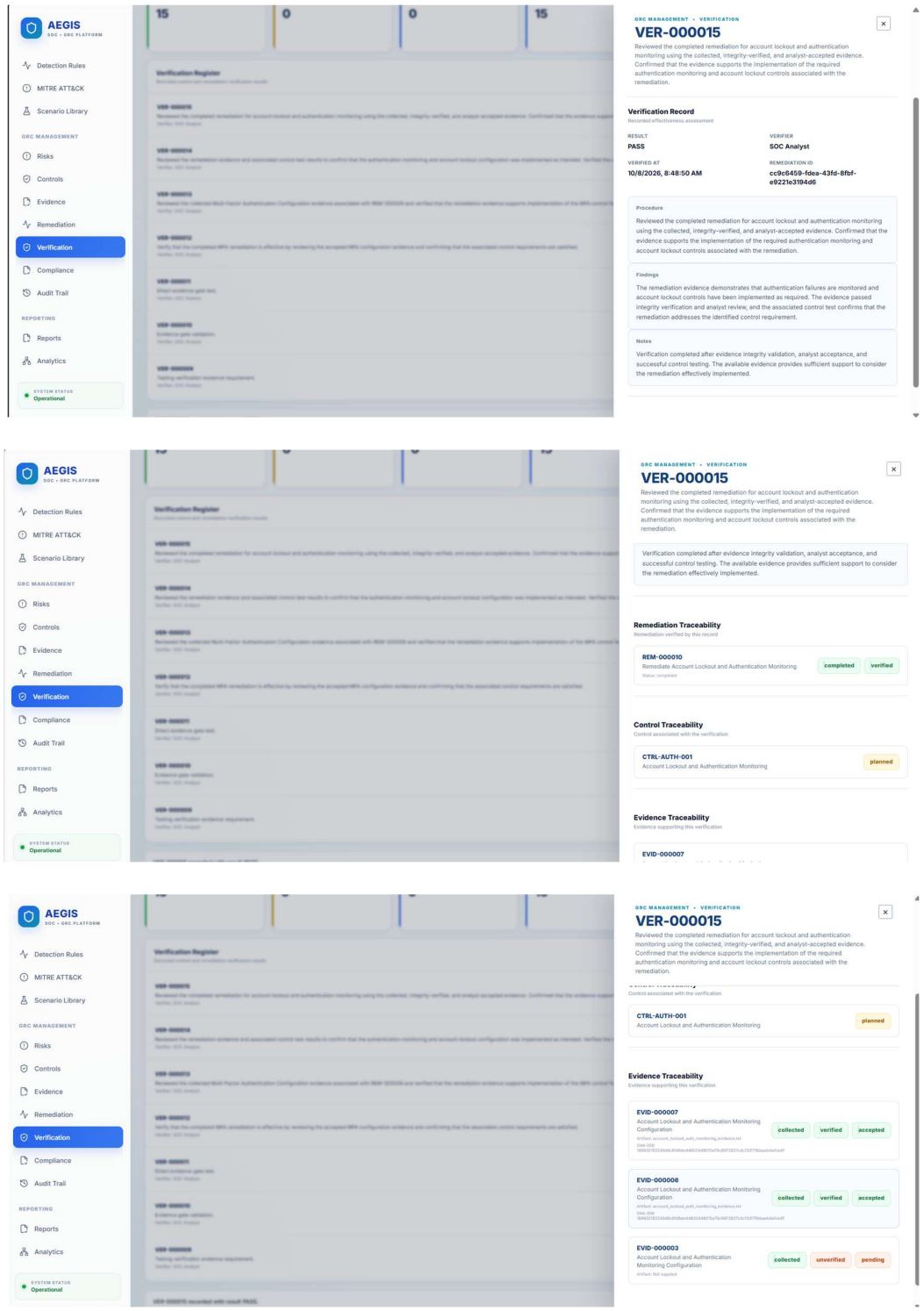


Figure 37 — Remediation verification confirming successful implementation of the associated control requirement.

Capture the Verification section showing:

- Verification procedure
- Verification result
- Findings
- Verifier
- Notes

The completed authentication workflow should show:

Result: PASS

What it demonstrates

This establishes the final validation step following evidence collection and control testing.

34.17 Screenshot 15 — Compliance

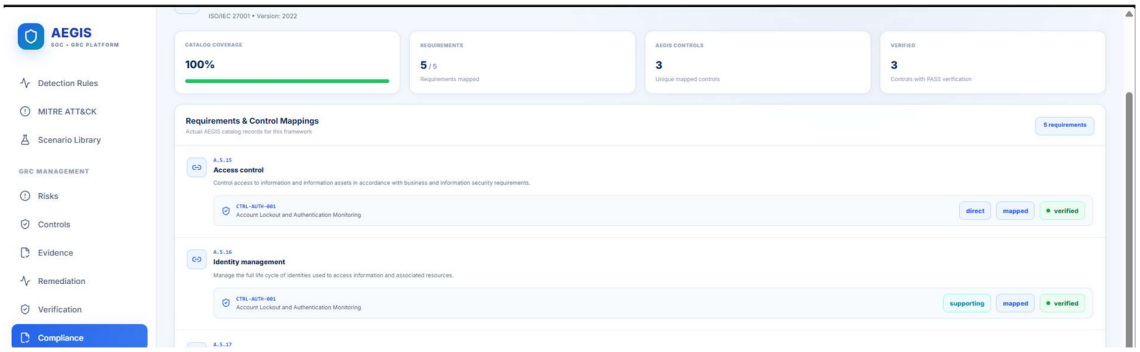


Figure 38 — AEGIS Compliance interface linking compliance requirements with mapped and verified security controls.

The screenshot shows the AEGIS Compliance interface with compliance requirements mapped to corresponding security controls. Each mapping identifies whether the control is directly or supportively associated with the requirement and displays the current verification status.

The visible example includes `A.5.15 Access control` mapped to CTRL-AUTH-001 — Account Lockout and Authentication Monitoring, with the control marked as **mapped** and **verified**.

What it demonstrates

This demonstrates that AEGIS connects its technical security controls with governance and compliance requirements. The verified status provides traceability from implemented controls and their supporting evidence/verification activities to the corresponding compliance requirements.

34.18 Screenshot 16 — Audit Trail

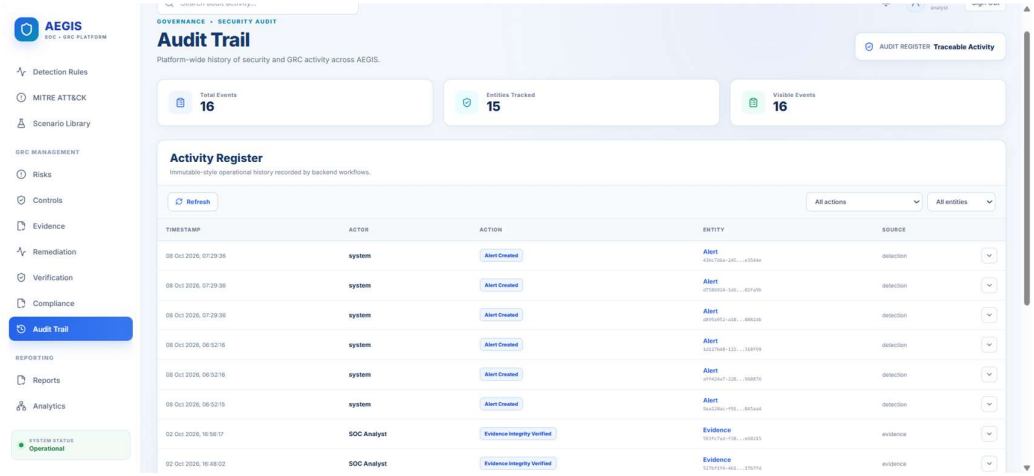


Figure 39 — AEGIS Audit Trail displaying system-generated security and GRC activity records.

The screenshot should preferably include records such as:

- Evidence Integrity Verified
- Alert Status Changed
- Alert Created

Important columns include:

- Timestamp
- Actor
- Action
- Entity Type
- Entity ID
- Description
- Old Value

- New Value
- Source

What it demonstrates

This is one of the most important screenshots because it proves that the platform records historical activity generated by the backend.

The Audit Trail is not simply a static frontend table. It contains records generated by actual platform operations.

34.19 Screenshot 17 — Reports

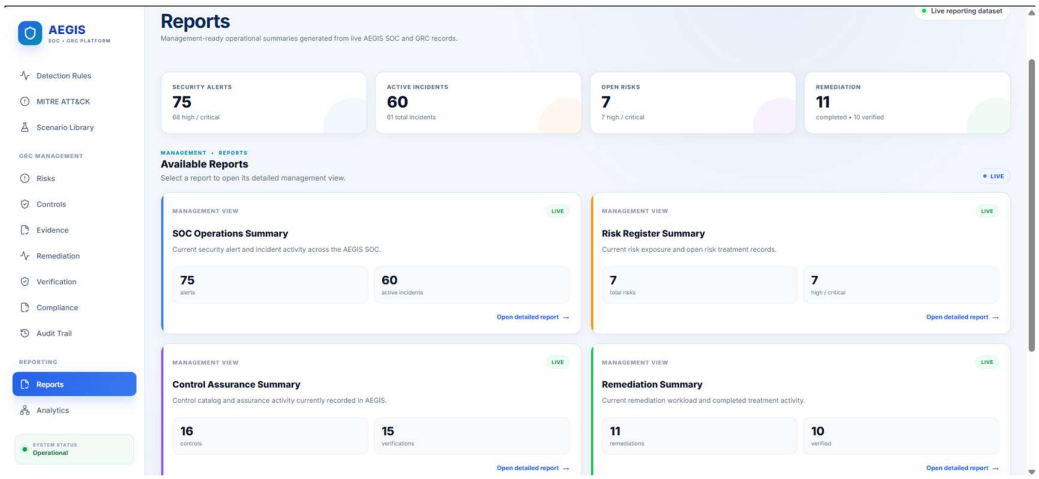


Figure 40 — AEGIS Reports interface providing consolidated security and GRC information.

Where applicable, include:

- Report categories
- Report summaries
- Security metrics
- GRC information
- Report detail view

What it demonstrates

This confirms that information generated throughout the platform can be presented for analysis and review.

34.20 Screenshot 18 — Report Detail View

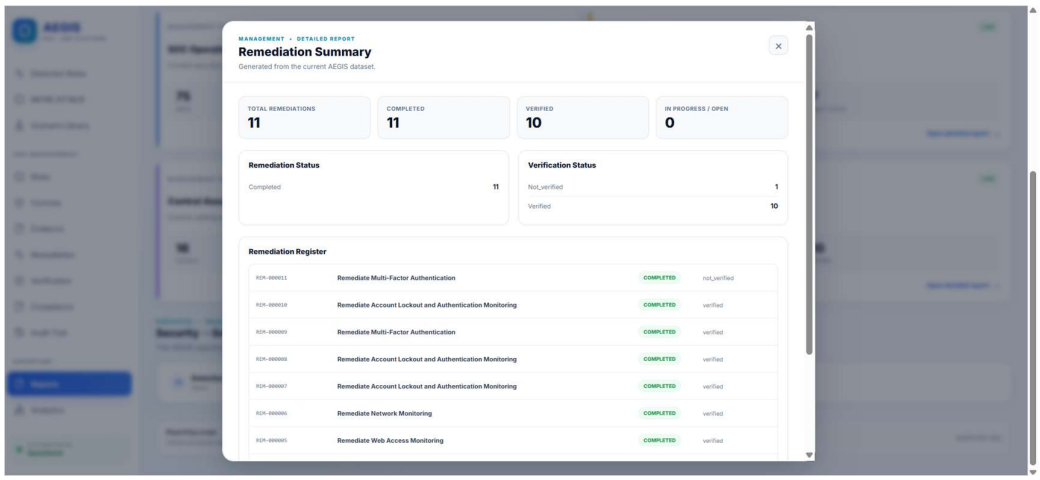


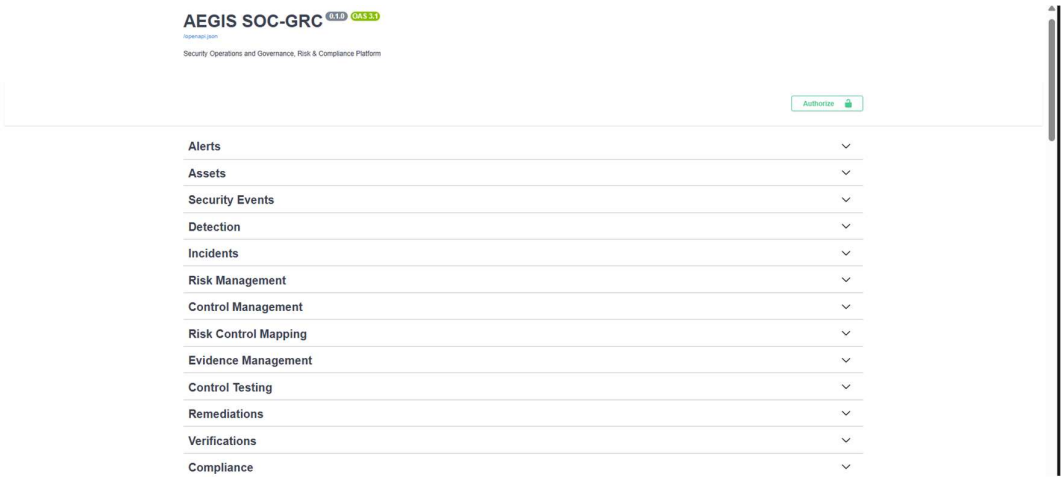
Figure 41 — Detailed AEGIS report view presented through the frontend report detail interface.

The screenshot should demonstrate the corrected modal/detail-panel behavior.

What it demonstrates

This provides evidence that detailed report information can be viewed without disrupting the main application layout.

34.21 Screenshot 19 — API Documentation



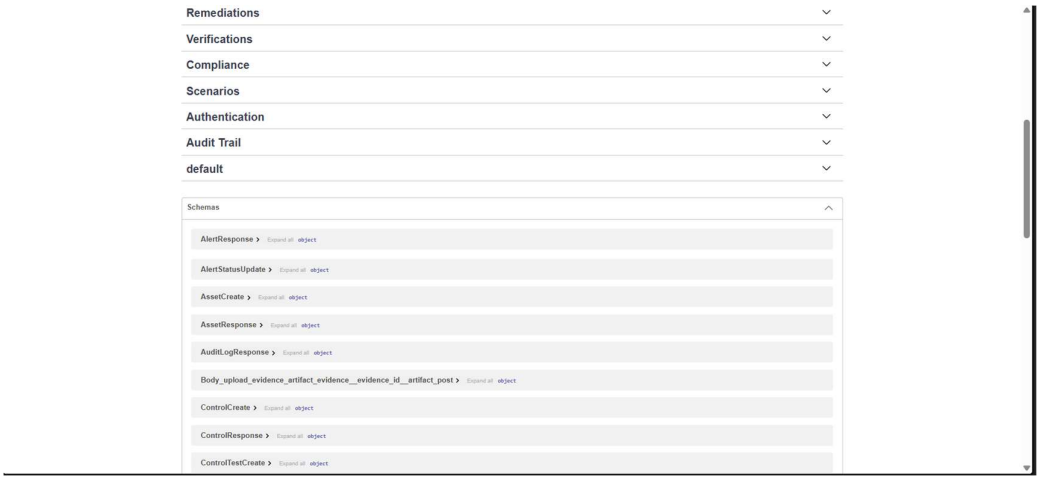


Figure 42 — FastAPI OpenAPI documentation displaying the AEGIS backend API resources.

Show representative endpoint groups such as:

- Events
- Detection
- Alerts
- Incidents
- Risks
- Controls
- Evidence
- Compliance
- Audit Trail

What it demonstrates

This provides direct evidence of the backend REST API structure.

34.22 Screenshot 20 — PostgreSQL Database

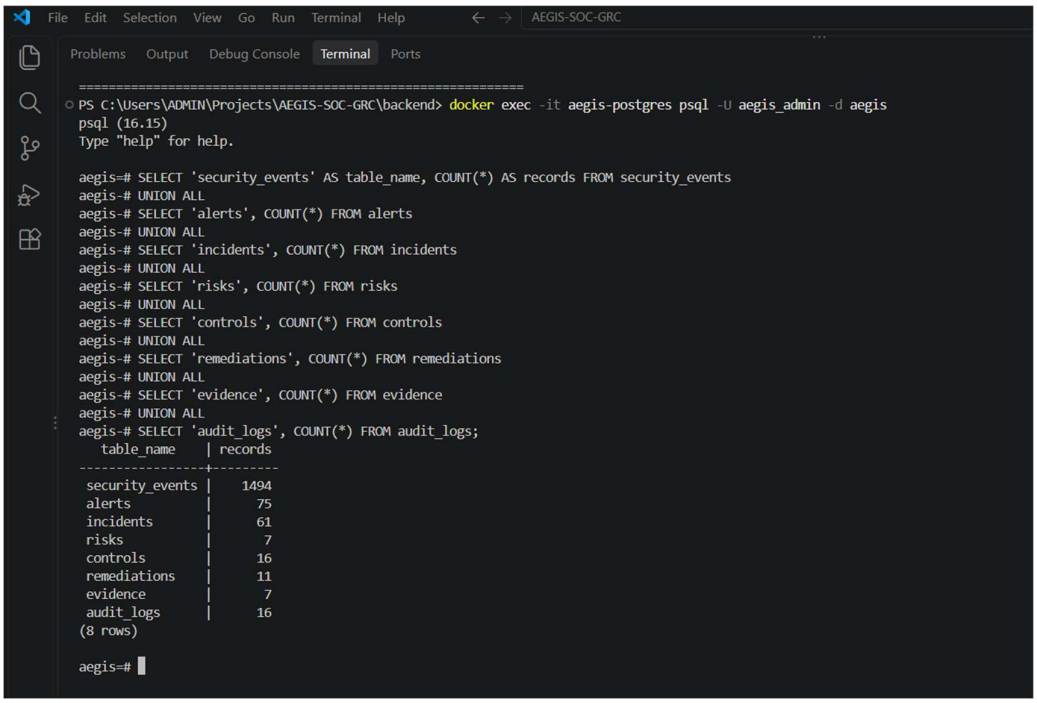


Figure 43 — PostgreSQL database records supporting the AEGIS-SOC-GRC platform.

Possible examples include:

- security_events
- alerts
- incidents
- risks
- controls
- evidence
- audit_logs

What it demonstrates

This confirms that the platform's data is persisted in PostgreSQL rather than existing only in frontend state.

34.23 Screenshot 21 — Docker Environment

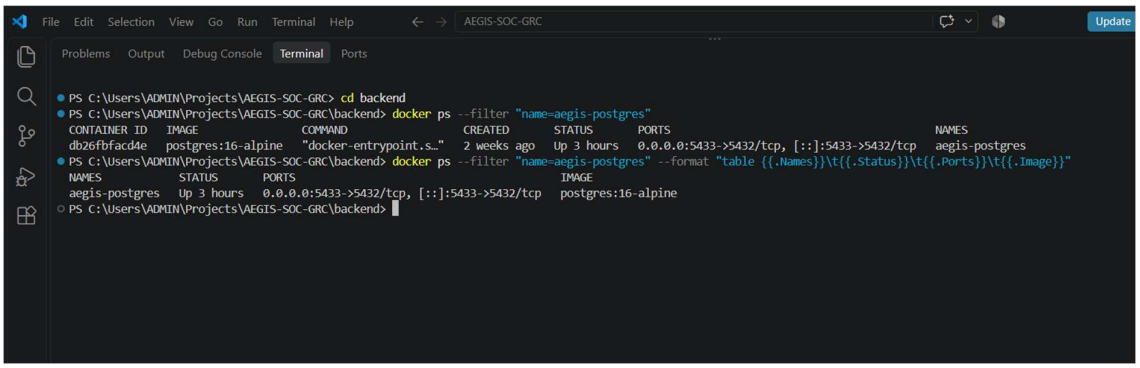


Figure 44 — Docker environment running the PostgreSQL database container used by AEGIS-SOC-GRC.

The screenshot can show:

- Container name
- Container status
- Port mapping
- PostgreSQL service

What it demonstrates

This documents the containerized database infrastructure used during development.

Suggested caption

34.24 Screenshot 22 — End-to-End Validation

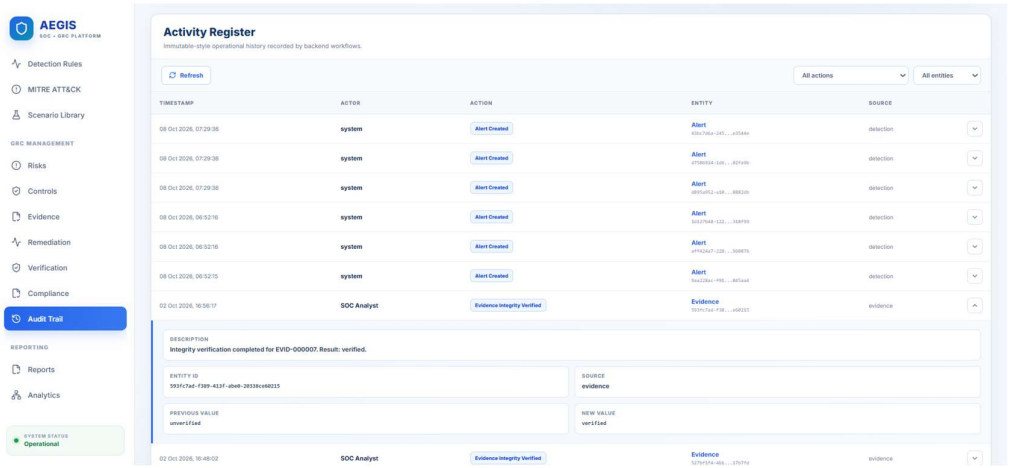


Figure 45 — End-to-end AEGIS-SOC-GRC validation demonstrating traceability from security simulation through audit recording.

The final Audit Trail view shows persistent records generated during the AEGIS security operations and GRC workflow. The recorded activities provide a historical trace of security and governance actions performed throughout the validation process.

What it demonstrates

This demonstrates that AEGIS maintains persistent auditability of operational and GRC activities. Combined with the preceding validation screenshots, the Audit Trail represents the final traceability layer of the end-to-end workflow.

34.24 Final Testing Evidence Statement

The screenshots included in this appendix are intended to complement the functional testing performed during development. They demonstrate that the major AEGIS-SOC-GRC components were exercised through the working application and that the resulting security and GRC information was persisted, displayed and traced across the platform.

> **The testing evidence supports the conclusion that AEGIS-SOC-GRC successfully connects security operations with risk management, remediation, evidence, compliance and auditability through a single traceable workflow.**

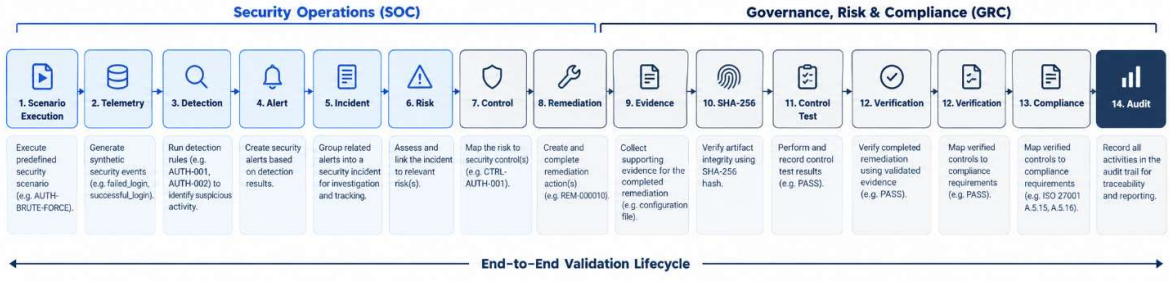


Figure 46 — **AEGIS-SOC-GRC** testing evidence sequence representing the complete end-to-end validation lifecycle.

35. Appendix D — Sample Security Scenarios and Detection Rules

35.1 Purpose of Appendix D

Appendix D documents representative security scenarios and detection rules implemented within the AEGIS-SOC-GRC platform. The purpose of the appendix is to show how controlled security scenarios are converted into synthetic telemetry and subsequently evaluated by detection logic.

The Scenario Engine provides repeatable test conditions for validating the SOC workflow without requiring real attacks or uncontrolled security activity.

The overall testing relationship is:

> **Scenario → Synthetic Telemetry → Detection Rule → Alert → Incident**

The resulting SOC activity can then continue into the GRC workflow through risk assessment, control assignment, remediation, evidence collection, verification, compliance mapping and audit recording.

35.2 AEGIS Scenario Engine

The AEGIS Scenario Engine is a controlled simulation component used to generate synthetic security events for testing the platform's detection and SOC workflows.

The engine allows predefined scenarios to be selected and executed repeatedly. Instead of performing real attacks, the scenarios create representative security telemetry that resembles events a SOC platform may receive during security monitoring.

This approach provides a controlled environment for testing detection rules, alert generation and downstream incident workflows.

Scenario execution model

Scenario Selection
↓
Scenario Execution
↓

Synthetic Security Events
↓
Security Event Storage
↓
Detection Rules
↓
Detection Result
↓
Alert
↓
Incident

The Scenario Engine therefore acts as the controlled input layer for the AEGIS security operations workflow.

35.3 Scenario Definition Structure

Each scenario is organized around several descriptive and operational properties.

Field	Purpose
Scenario ID	Unique identifier for the scenario
Category	Security domain represented by the scenario
Name	Human-readable scenario name
Description	Explanation of the simulated activity
Severity	Expected security severity
MITRE ATT&CK Technique	Relevant ATT&CK mapping where applicable
Detection Rules	Rules expected to identify the generated activity
Expected Outcome	Expected detection or alert result
Steps	Ordered simulation activities
Runner	Function responsible for generating the scenario telemetry.
Scenario Type	Classification of the scenario, such as atomic or attack-chain.

This structure makes scenarios easier to understand, execute and validate independently.

35.4 Authentication Scenario — AUTH-BRUTE-FORCE

One of the primary scenarios used during AEGIS validation is:

Scenario ID: `AUTH-BRUTE-FORCE`

Name: Brute Force → Successful Login

Category: Authentication

Severity: High

MITRE ATT&CK: `T1110.001 — Password Guessing`

Detection Rules:

- `AUTH-001`
- `AUTH-002`

Simulation behavior

The scenario generates repeated failed authentication events for a user and subsequently generates a successful login event.

A simplified representation is:

Failed Login
↓
Failed Login
↓
Failed Login
↓
Failed Login
↓
Failed Login
↓
Failed Login
↓
Successful Login

The generated events contain security telemetry such as:

- Username
- Source IP
- Asset
- Event type
- Timestamp

The purpose is to create a recognizable authentication pattern that can be evaluated by the corresponding detection rules.

35.5 Detection Rule — AUTH-001

Multiple Failed Login Attempts

Rule ID: `AUTH-001`

Purpose: Detect repeated failed authentication attempts within a defined time window.

The implemented rule evaluates failed-login security events and groups relevant events using attributes such as asset, username and source IP.

The configured threshold is:

> **5 or more failed login attempts within 5 minutes**

When the threshold is reached, the rule generates or updates a high-severity alert.

Detection flow

Failed Login Events
↓
5+ Failed Attempts
↓
5-Minute Detection Window
↓
AUTH-001 Condition Matched
↓
High-Severity Alert

This rule provides a basic detection mechanism for repeated authentication failures.

35.6 Detection Rule — AUTH-002

Successful Login After Repeated Failures

Rule ID: `AUTH-002`

This rule identifies a successful login that occurs after multiple failed authentication attempts.

The implemented rule evaluates failed-login and successful-login events within a five-minute window.

The configured condition requires:

> **At least 3 failed login attempts followed by a successful login**

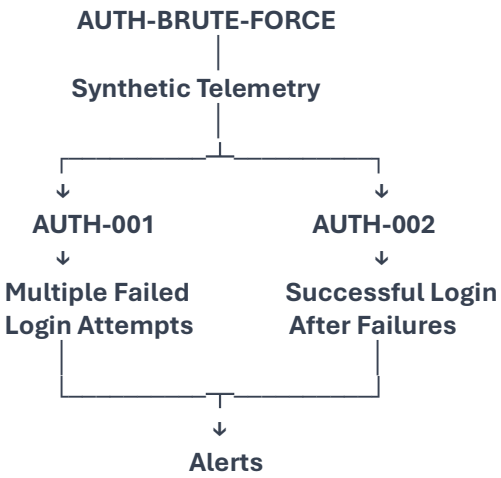
Detection flow

Multiple Failed Logins
↓
Successful Login
↓
AUTH-002 Condition Matched
↓
High-Severity Alert

This scenario is useful because a successful authentication following repeated failures provides additional context compared with detecting failed attempts alone.

35.7 Authentication Detection Relationship

The AUTH-BRUTE-FORCE scenario demonstrates how multiple detection rules can evaluate the same underlying telemetry.



This demonstrates that a single security scenario can produce multiple detection outcomes depending on the behavior represented in the telemetry.

35.8 Other Scenario Categories

The Scenario Engine is designed around multiple security domains rather than authentication alone.

The project includes scenario concepts covering areas such as:

- **Authentication**
- **Network activity**

- **Endpoint activity**
- **Phishing**
- **Web application security**
- **Malware-related activity**

This allows the detection workflow to be tested against different classes of synthetic security telemetry.

35.9 Network Security Scenarios

Network-oriented scenarios represent activities such as network reconnaissance and connection behavior.

Examples include simulated:

- Port scanning
- Repeated connection attempts
- Unusual destination-port activity
- High-volume connection behavior
- External destination activity

The purpose is to generate controlled network telemetry that can be evaluated by network detection rules.

The scenarios do not perform real network reconnaissance. They represent the expected telemetry patterns within the AEGIS simulation environment.

35.10 Endpoint Security Scenarios

Endpoint scenarios represent suspicious command, scripting and process activity.

Examples include simulated:

- Suspicious PowerShell activity
- Suspicious command execution

- LOLBin-related behavior
- Suspicious process spawning
- Persistence-related commands

These scenarios allow endpoint detection logic to be tested without executing actual malicious activity.

35.11 Phishing Scenarios

Phishing scenarios represent suspicious email and credential-harvesting activity.

Examples include:

- Suspicious email links
- Malicious attachments
- Credential-harvesting pages
- Suspicious sender domains

The purpose is to generate representative telemetry for testing phishing-related detection rules.

35.12 Web Application Scenarios

The Scenario Engine also provides scenarios representing common web application attack patterns.

Examples include simulated:

- SQL injection
- Cross-site scripting
- Directory traversal
- Web scanner activity

These scenarios provide controlled test inputs for web application detection logic.

35.13 Scenario-to-Detection Relationship

The relationship between scenarios and detection rules is an important part of the testing architecture.

A scenario does not directly create a final security alert. Instead, it generates telemetry that must pass through the detection layer.

Scenario
↓
Synthetic Security Events
↓
Detection Rule Evaluation
↓
Detection Condition
↓
Alert Creation / Update
↓
Incident Workflow

This separation makes the simulation more representative of an actual security monitoring architecture.

35.14 Scenario Execution

The Scenario Engine can be executed from the project environment using the scenario engine command-line interface.

Examples include:

```
python simulator/scenario_engine.py --list
```

to list available scenarios,

```
python simulator/scenario_engine.py --scenario AUTH-BRUTE-FORCE
```

to execute a specific scenario, and:

```
python simulator/scenario_engine.py --scenario ALL
```

to execute the available scenario collection.

The engine also supports scenario grouping and asset selection as part of the simulation workflow.

35.15 Safety and Simulation Boundary

The Scenario Engine was deliberately designed as a **synthetic telemetry generator** rather than an offensive security execution framework.

It does not perform real:

- Attacks
- Malware execution
- Network scanning
- Phishing campaigns
- SQL exploitation
- PowerShell attacks

Instead, it generates controlled security-event data representing the types of activity that detection rules are designed to identify.

This boundary makes the engine suitable for repeatable development and testing of the AEGIS SOC workflow.

35.16 Appendix Summary

The scenarios and detection rules documented in this appendix demonstrate how AEGIS transforms controlled security simulations into actionable SOC events.

The AUTH–BRUTE–FORCE scenario provides a representative example of the complete process:

> **Scenario → Synthetic Authentication Events → AUTH-001 / AUTH-002 → Detection → Alert**

The same design can be extended across network, endpoint, phishing and web-application security scenarios, allowing AEGIS to provide a broader controlled environment for testing security detection and downstream GRC workflows.

35.17 Detection Rule Catalog

The following table summarizes the detection rules implemented in AEGIS-SOC-GRC. The rules are organized by security category and are evaluated against security telemetry generated by the Scenario Engine or other event sources.

Rule ID	Rule Name	Category	Trigger	Time Window	Severity	Expected Alert
AUTH-001	Multiple Failed Login Attempts	Authentication	5 or more failed login attempts	5 minutes	High	Multiple Failed Login Attempts
AUTH-002	Successful Login After Repeated Failures	Authentication	At least 3 failed logins followed by a successful login	5 minutes	High	Successful Login After Repeated Failures
AUTH-003	Login From New Source IP	Authentication	Login detected from a previously unseen source IP	Rule-defined	Rule-defined	Login From New Source IP
AUTH-004	Multiple Accounts Targeted From One Source IP	Authentication	Multiple user accounts targeted from the same source IP	Rule-defined	Rule-defined	Multiple Accounts Targeted From One Source IP
AUTH-005	Privileged Account Login	Authentication	Privileged account authentication activity detected	Rule-defined	Rule-defined	Privileged Account Login
AUTH-006	Account Lockout Detected	Authentication	account_lockout security event detected	5 minutes	High	Account Lockout Detected
NET-001	Port Scan Detected	Network	Port-scan activity detected in network telemetry	Rule-defined	Rule-defined	Port Scan Detected

Rule ID	Rule Name	Category	Trigger	Time Window	Severity	Expected Alert
NET-002	Repeated Connection Attempts	Network	Repeated connection attempts detected	Rule-defined	Rule-defined	Repeated Connection Attempts
NET-003	Unusual Destination Port	Network	Connection to an unusual destination port	Rule-defined	Rule-defined	Unusual Destination Port
NET-004	High Volume Connections	Network	Abnormally high volume of network connections	Rule-defined	Rule-defined	High Volume Connections
NET-005	External Destination	Network	Connection to an external destination matching the rule condition	Rule-defined	Rule-defined	External Destination
PROC-001	Suspicious PowerShell Activity	Endpoint / Process	Suspicious PowerShell activity detected	Rule-defined	Rule-defined	Suspicious PowerShell Activity
PROC-002	Suspicious CMD Activity	Endpoint / Process	Suspicious command-shell activity detected	Rule-defined	Rule-defined	Suspicious CMD Activity
PROC-003	LOLBin Abuse	Endpoint / Process	Suspicious use of a Living-off-the-Land Binary	Rule-defined	Rule-defined	LOLBin Abuse
PROC-004	Suspicious Process Spawn	Endpoint / Process	Suspicious process creation/spawn activity	Rule-defined	Rule-defined	Suspicious Process Spawn
PROC-005	Persistence Command	Endpoint / Process	Suspicious persistence-related command activity	Rule-defined	Rule-defined	Persistence Command
MAL-001	Suspicious File Creation	Malware	Suspicious file creation event detected	Rule-defined	Rule-defined	Suspicious File Creation

Rule ID	Rule Name	Category	Trigger	Time Window	Severity	Expected Alert
MAL-002	Malicious Hash	Malware	File/hash matches a malicious indicator condition	Rule-defined	Rule-defined	Malicious Hash
MAL-003	Suspicious File Execution	Malware	Suspicious file execution activity detected	Rule-defined	Rule-defined	Suspicious File Execution
MAL-004	Malware Persistence	Malware	Malware-related persistence activity detected	Rule-defined	Rule-defined	Malware Persistence
PHISH-001	Suspicious Email Link	Phishing	Suspicious email-link activity detected	Rule-defined	Rule-defined	Suspicious Email Link
PHISH-002	Malicious Attachment	Phishing	Malicious/suspicious email attachment detected	Rule-defined	Rule-defined	Malicious Attachment
PHISH-003	Credential Harvesting Page	Phishing	Credential-harvesting page activity detected	Rule-defined	Rule-defined	Credential Harvesting Page
PHISH-004	Suspicious Sender Domain	Phishing	Suspicious sender-domain activity detected	Rule-defined	Rule-defined	Suspicious Sender Domain
WEB-001	SQL Injection	Web Application	SQL-injection activity detected	Rule-defined	Rule-defined	SQL Injection
WEB-002	XSS Attempt	Web Application	Cross-site scripting activity detected	Rule-defined	Rule-defined	XSS Attempt
WEB-003	Directory Traversal	Web Application	Directory-traversal activity detected	Rule-defined	Rule-defined	Directory Traversal

Rule ID	Rule Name	Category	Trigger	Time Window	Severity	Expected Alert
WEB-004	Web Scanner	Web Application	Web-scanner activity detected	Rule-defined	Rule-defined	Web Scanner

The authentication rule definitions explicitly identify AUTH-001 through AUTH-005, with AUTH-006 additionally implemented for account-lockout detection. Pasted text(20261008-072531) The detection endpoint invokes all six authentication rules, five network rules, five endpoint/process rules, four malware rules, four phishing rules and four web rules.

35.18 Important note about the catalog

> **Note:** `Rule-defined` is intentionally used where the available implementation extract does not expose a specific threshold, time window or severity value. This avoids documenting assumptions as implemented behavior. The explicitly validated authentication values are documented exactly where they were confirmed during project testing.

35.19 Detection Engineering Summary

The rule catalog demonstrates that AEGIS is not limited to a single authentication detection. The detection layer is organized into multiple security domains:

Authentication → Network → Endpoint/Process → Malware → Phishing → Web Application

This provides a reusable detection framework in which synthetic security scenarios can generate telemetry and multiple detection rules can independently evaluate the resulting events.

The architecture therefore follows:

Security Event → Detection Rule → Condition Match → Alert → Incident